

RAG Knowledge Base & Retrieval -- Implementation Manual

AI IT Helpdesk Agent -- Phase 7 (claudeprompt/phase7.md)

Design, implementation, testing, and evaluation of the RAG knowledge base built for this project.

Table of Contents

Table of Contents	2
Executive Summary	8
What Phase 7 Is	8
The Full Pipeline	8
What This Phase Explicitly Does NOT Build	9
Results at a Glance	9
How to Use This Manual	10
System Context	11
The application before this phase	11
Where RAG fits	12
What "knowledge base" means here, concretely	12
The technology choices this phase made, and why they were already decided	13
A map of what changed on disk	13
Ticket History vs. RAG	15
Why this distinction gets its own chapter	15
Two different questions	15
Why conflating the two would actively make the system worse, not just redundant	16
How this shows up in the actual implementation	16
What the future AI agent phase is expected to do with both	17
Data Model	18
Design goal: two tables, not many	18
knowledge_documents: already existed, extended by two columns	18
knowledge_chunks: new in Phase 7	21
Entity relationship, in one picture	22
Migration mechanics	22
Chunking Strategy	23
Why chunk at all	23
Why not just split every N characters	23
The actual algorithm (backend/app/rag/chunking.py)	23
Overlap: carrying context across a chunk boundary	24
Configuration, and why these specific defaults	24
Tokens vs. words: an honest approximation	24
The tradeoff, stated plainly	25
A worked example, using this project's real seeded content	25

What this module is tested against	25
Embeddings	26
The requirement: swappable, not hardcoded	26
The interface	26
DevEmbeddingProvider: the local development provider, and why it's more than a stub	27
BedrockEmbeddingProvider: the production provider	28
EmbeddingService: the thin wrapper everything actually calls	28
What this is tested against	29
Vector Storage	30
Why PostgreSQL + pgvector, not a separate vector database	30
Enabling the extension	30
The distance metric: cosine distance, explained from first principles	30
Why HNSW, not IVFFlat	31
From a Python list of floats to a queryable vector column: a genuine implementation subtlety	31
What the final picture looks like	32
Ingestion Pipeline	33
The pipeline, end to end	33
Step 1: document storage -- local disk or S3, behind a swappable backend	33
Step 2: text extraction -- format-specific, but not a pluggable abstraction	33
Step 3: cleaning	34
Step 4 and 5: chunking and embedding	34
Step 6: storing chunks and flipping status to READY	34
Document versioning: how a re-upload actually works	35
Failure handling: the SAVEPOINT, explained in depth	35
What this is tested against	36
Admin API Reference	38
Overview	38
Endpoint summary	38
POST /admin/knowledge	38
GET /admin/knowledge	39
GET /admin/knowledge/{id}	40
GET /admin/knowledge/{id}/chunks	40
DELETE /admin/knowledge/{id}	40
POST /admin/knowledge/search	40
What this is tested against	41
Retrieval and Similarity Search	42

The flow	42
Step 1: embed the query	42
Step 2: the query, annotated line by line	42
Metadata filtering	43
What comes back	43
A real example	43
What this is tested against	44
Hybrid Search: Why Not Yet, and What It Would Take	45
What's Built, and What Isn't	45
Why Hybrid Search Matters at All	45
The Standard Pattern, Conceptually	46
What It Would Concretely Touch	46
Why This Wasn't Built Now	46
The RAG Context Format	48
One Small, Reusable Interface	48
What's Deliberately Left Out	48
Why This Is a Separate Function at All	49
What This Phase Explicitly Does Not Build	49
Security	50
phase7.md Section 18's Checklist, Addressed Item by Item	50
require_admin: What It Actually Does, and What It Doesn't	50
Why This Design Is Still the Right Call	52
Forward Compatibility	52
Verified	52
Error Handling Reference	53
Purpose of This Chapter	53
The Exception Types	53
Complete Reference Table	54
Why Pipeline Failures Return 201, Not 4xx or 5xx	54
The Life of a FAILED Document	55
Testing Guide	56
Prerequisites	56
1. Local environment setup	56
2. Running the automated test suite	57
3. Manual API testing walkthrough	58
4. Running the retrieval evaluation	60

5. Resetting the local environment	60
Retrieval Evaluation	62
"Do not judge RAG only by whether the application runs"	62
Recall@K and Precision@K, defined precisely for this dataset	62
The labeled dataset	62
The actual results	63
Interpreting these numbers honestly	63
Using this harness going forward	64
Worked Example: A Complete VPN Scenario	65
Why one concrete example, traced fully	65
Step 1: An admin uploads the VPN Troubleshooting Guide	65
Step 2: The document is stored	65
Step 3: Text is extracted	65
Step 4: Text is cleaned	65
Step 5: Text is chunked	66
Step 6: An embedding is generated for each chunk	66
Step 7: Chunks and embeddings are stored, and the document goes READY	66
Step 8: Later, a student emails IT support	66
Step 9: The query is converted to an embedding	66
Step 10: A vector similarity search runs	67
Step 11: The top-K chunks are returned, ranked by score	67
Step 12: The result is packaged into the RAG context format	67
Step 13: This phase stops here	67
This was run for real	68
Architecture Boundaries	69
The rule, in phase7.md's own words	69
What this phase explicitly does not build, verified item by item	69
How a future AI agent phase will actually connect to what was built here	70
AWS Deployment Considerations	72
Scope of this chapter	72
Bedrock: enabling and permitting the embedding model	72
S3: private storage for uploaded knowledge-base files	72
RDS for PostgreSQL: enabling pgvector	73
Networking and least privilege	73
Secrets and credentials	73
Cost awareness	74

Deployment checklist	74
Appendix A: Schema Reference	75
Full migration text	75
knowledge_documents -- full column reference	75
knowledge_chunks -- full column reference	76
Relationship	77
Appendix B: API Reference	78
Endpoints	78
KnowledgeDocumentRead	78
KnowledgeChunkRead	79
KnowledgeSearchRequest	79
KnowledgeSearchResponse	79
KnowledgeDocumentStatus enum	79
Appendix C: Glossary and FAQ	81
Glossary	81
FAQ	82
Appendix D: Core Source Code	84
Purpose of this appendix	84
app/rag/chunking.py	84
app/rag/ingestion_service.py	86
app/rag/retrieval_service.py	91
app/rag/embeddings/dev_provider.py	92
Appendix E: Example Knowledge Documents (Full Text)	93
About these documents	93
VPN Troubleshooting Guide	93
WiFi Troubleshooting Guide	95
Password Reset Procedure	96
Student Account Setup	97
Microsoft Office Installation	99
Campus Network Guide	100
Printer Troubleshooting	101
Known IT Issues	102
Appendix F: Example Chunks and Example Retrieval Results	104
Purpose of this appendix	104
Example: how the VPN Troubleshooting Guide was actually chunked	104
Example: a real retrieval result	105

What this demonstrates end to end	106
---	-----

Executive Summary

This chapter explains what Phase 7 is, why it exists, what it does and does not do, and how to use the rest of this manual. Everything stated here is expanded in depth in a later chapter -- treat this as the map, not the territory.

What Phase 7 Is

Phase 7 (`claudeprompt/phase7.md`) builds the **RAG knowledge base and retrieval layer** for the AI IT Helpdesk Agent project. In one sentence: it makes "what does the IT department's official documentation say about this problem" a question the system can answer automatically, given a student's plain-language question, by finding the most relevant chunks of internal IT documentation and handing them back in a clean, structured form.

RAG stands for Retrieval-Augmented Generation. The "Generation" half -- an AI actually composing a reply using this retrieved material -- is explicitly **not** part of this phase. Phase 7 builds only the "Retrieval-Augmented" half: the machinery that finds the right documentation. A future phase will take what this phase produces and feed it into an AI agent that reasons over it and drafts a response. This phase stops one step before that, by design.

The Full Pipeline

Two flows exist side by side after this phase: one that gets documentation *into* the system (ingestion, run by an admin, ahead of time), and one that gets the right piece of it back *out* (retrieval, run per student question, at query time).

Ingestion (offline, admin-triggered, one document at a time):

```
Admin uploads a document (PDF / TXT / Markdown / DOCX)
|
Document metadata stored (knowledge_documents row, status=PROCESSING)
|
Extract text from the file
|
Clean the extracted text (normalize whitespace, line endings)
|
Split the text into chunks (boundary-aware, not fixed-character-count)
|
Generate an embedding vector for each chunk
|
Store every chunk + its embedding (knowledge_chunks rows)
|
Document status flips to READY (or FAILED, with a reason, if any step failed)
```

Retrieval (online, per student question, milliseconds):

```
Student's message text
|
Convert the query text into an embedding vector (same embedding model as ingestion)
|
Run a vector similarity search against every READY document's chunks
|
Return the top-K most similar chunks, ranked by score
|
```

```

Package the result into a clean JSON context object
|
[Phase 7 stops here]
|
A future AI agent will consume that context object

```

Every arrow above corresponds to a real, working, tested piece of code in this project -- none of it is a plan for later. The rest of this manual documents each stage in detail, including the actual source files, the actual SQL, and the actual test results from running the whole thing against a live Postgres database.

What This Phase Explicitly Does NOT Build

`claudeprompt/phase7.md` section 23 is explicit that the following are **out of scope** for this phase, and none of them exist in this codebase after Phase 7:

- an AI agent
- automatic AI replies
- Bedrock chat generation (Bedrock is used here only for *embeddings*, never for generating a response)
- ticket classification by AI
- automatic escalation
- email response generation
- autonomous tools
- automatic ticket closure
- a human-approval workflow

Those all belong to later phases. If you are looking for "where does the AI decide what to tell the student," it is not in this codebase yet -- and that is intentional, not an oversight. Phase 7's job is narrower and, deliberately, more testable on its own: prove that the *retrieval* step works correctly, in isolation, before any generation step is layered on top of it. A wrong or noisy retrieval result would otherwise become a wrong AI answer with no visible error anywhere in between -- the retrieval evaluation chapter later in this manual explains why that ordering matters.

Results at a Glance

Everything below was produced by actually running this system, not estimated:

Metric	Result
Example IT documents ingested through the real pipeline	8 of 8 succeeded, all reached status READY
Total automated tests passing	120 (63 pre-existing Phase 1-6 tests, unaffected by this phase's changes, plus 57 new Phase 7 tests across 6 test files)
Retrieval evaluation: Recall@1	100.00%
Retrieval evaluation: Recall@3	100.00%

Metric	Result
Retrieval evaluation: Recall@5	100.00%
Labeled evaluation questions used	10, each with one hand-labeled expected document

Every one of the 10 labeled evaluation questions retrieved its correct expected document at rank 1 -- not just "somewhere in the top 5." The full evaluation methodology, the complete labeled dataset, and the full per-question breakdown (including Precision@K, which tells a different and complementary story from Recall@K) are covered later in this manual's evaluation chapter.

How to Use This Manual

This manual is organized to be read start to back by someone who needs to understand, operate, extend, or test this system, but each chapter is also written to stand alone as a reference:

- **System context and the ticket-history/RAG distinction** (the next two chapters) explain *why* this system is shaped the way it is, and the one architectural rule that must never be violated: ticket history and the knowledge base are different things, retrieved differently, for different purposes.
- **Data model, chunking, embeddings, and vector storage** (four chapters) are the technical core: exactly what tables exist, exactly how text becomes chunks, exactly how chunks become vectors, and exactly how those vectors are stored and searched.
- **Ingestion pipeline and the admin API** document the actual code path from an uploaded file to a queryable chunk, and every HTTP endpoint available to manage it, with real request/response examples.
- **Retrieval and similarity search** explains the query-time path in full, including the actual cosine-similarity math, worked with real numbers.
- **Security, error handling, and testing** cover what happens when things go wrong (corrupted files, failed embedding calls, unauthorized callers) and exactly how to verify all of it yourself, step by step.
- **Retrieval evaluation** is where the Recall@K/Precision@K numbers above come from, in full, with the reasoning for why this matters before any AI agent is layered on top.
- **A worked end-to-end example, AWS deployment considerations, and appendices** (full schema reference, full API reference, glossary) close out the manual.

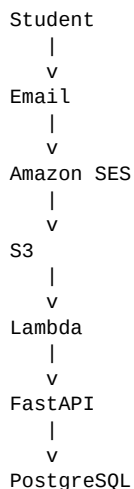
If you only need one thing right now -- to run this system locally and confirm it works -- skip to the testing chapter, which is a self-contained, step-by-step walkthrough starting from a clean checkout.

System Context

The application before this phase

By the time Phase 7 began, the AI IT Helpdesk Agent was already a working, tested, five-phase application. Understanding what already existed matters because phase7.md is explicit about a rule this manual takes seriously throughout: **do not redesign what already works**. Every design decision in this phase either reuses existing infrastructure directly or extends it in a way that's consistent with how the rest of the codebase is already built.

What already existed, in outline:



And, inside that FastAPI + PostgreSQL backend, a working system supporting:

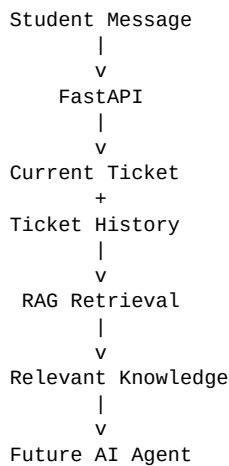
- **Users** -- students and staff identified by email, no login (see the Security chapter for why this matters to Phase 7's own authorization design).
- **Tickets** -- one row per IT issue, with a status transition graph (**NEW** -> **AI_INVESTIGATING** -> ... -> **RESOLVED** -> **CLOSED**, with reopen edges), category, priority, and assignment to an admin.
- **Messages** -- the full conversation on a ticket, inbound and outbound, with email threading headers.
- **Ticket events** -- an append-only audit trail of every status change, assignment, and reopening.
- **Email threading** -- matching an inbound reply to the right existing ticket via **In-Reply-To**, **References**, thread ID, or an explicit ticket number in the subject line, in that priority order.
- **Ticket status** -- the full state machine described above, already tested against real transition rules.
- **Admin dashboard** -- a Next.js frontend for IT staff to view and manage tickets.
- **Inbound email processing** -- SES/Lambda-based ingestion turning an inbound email into a ticket or a reply on an existing one.
- **Ticket history** -- a merged, chronological view of messages, events, and (eventually) AI actions per ticket.

None of this changed in Phase 7. Not one existing migration was modified, not one existing service function's behavior changed, and not one existing endpoint's contract changed. The

`knowledge_documents` table already existed (as a placeholder, with no rows ever written to it) from the very first migration in this project -- Phase 7's job was to finish what that placeholder was waiting for, not to redesign it.

Where RAG fits

The objective for this phase, stated directly in `phase7.md`, is to build the retrieval layer that will eventually sit between "a student sends a message" and "the (future) AI agent responds":



This phase builds exactly the "RAG Retrieval" box and the "Relevant Knowledge" it produces. It deliberately stops there. The arrow from "Relevant Knowledge" into "Future AI Agent" exists in this diagram, and this phase makes sure the data on the near side of that arrow (the retrieved knowledge, in a clean, documented JSON shape) is ready and correct -- but nothing on the far side of that arrow was built. There is no AI agent in this codebase. There is no code path anywhere that takes a retrieved chunk and turns it into a reply a student would see.

What "knowledge base" means here, concretely

The knowledge base is the IT department's own documentation -- the kind of internal reference material an experienced help-desk staffer already has memorized, written down so a system (eventually, an AI agent; for now, an admin using the search preview endpoint) can look it up. The eight example documents seeded into this project's knowledge base (`backend/seed/knowledge_docs/*.md`) are representative of exactly this kind of material:

```
VPN Troubleshooting Guide
WiFi Troubleshooting Guide
Password Reset Procedure
Student Account Setup
Microsoft Office Installation
Campus Network Guide
Printer Troubleshooting
Known IT Issues
```

This is categorically different from a ticket. A ticket is a record of one specific incident involving one specific person. A knowledge document is a piece of institutional knowledge that applies to every future incident of a similar kind. Chapter 3 makes this distinction precise, because getting it wrong -- for

instance, by treating old resolved tickets as knowledge-base documents -- is exactly the mistake phase7.md warns against, and one this implementation was careful to avoid.

The technology choices this phase made, and why they were already decided

Two of Phase 7's biggest architectural decisions were effectively already made before this phase started, by earlier planning documents in this same project:

- `docker-compose.yml` (written in Phase 1) already pins the `pgvector/pgvector:pg16` image for the local Postgres container, with a comment explicitly noting: *"We use the pgvector image... even though Phase 1 doesn't use vector columns yet: RAG (a later phase) will need the vector extension, and pinning the image now avoids swapping the base image... later."* This phase's vector storage decision (Chapter 6) was already anticipated and prepared for.
- `docs/05-tech-stack.md` (also written before this phase) already lays out the reasoning for PostgreSQL + pgvector over a separate vector database, and already names Amazon Bedrock as the intended embedding provider, consistent with the project's broader AWS-native direction (SES for email, Lambda for ingestion). This phase's embedding provider decision (Chapter 5) confirms and implements that earlier direction rather than deciding it fresh.

Both of these are called out here because a system context chapter should be honest about what was actually a decision made *in* this phase versus a decision *inherited* by this phase. The two most consequential infrastructure choices in this manual -- Postgres+pgvector for storage, and Bedrock for embeddings -- were both already settled by the time implementation began. What Phase 7 actually decided was everything *around* those two anchors: how documents are chunked, how the embedding provider is abstracted so it can be swapped, how failures are handled, how versioning works, how retrieval is scoped and filtered, and how the whole thing is tested and evaluated.

A map of what changed on disk

For orientation, everything Phase 7 added or touched, grouped by kind:

```
backend/  
  app/  
    rag/                                NEW -- the entire RAG subsystem (see Chapters 4-9)  
    auth.py                             NEW -- admin authorization stand-in (see Chapter 12)  
    routers/knowledge.py                 NEW -- the admin HTTP API (see Chapter 8)  
    schemas/  
      knowledge_document.py             EXTENDED -- two new columns added  
      knowledge_chunk.py                 NEW  
      knowledge_search.py                NEW  
    config.py                           EXTENDED -- RAG-specific settings added  
    database.py                         EXTENDED -- pgvector type registration added  
    errors.py                           EXTENDED -- ValidationError added  
    main.py                             EXTENDED -- new router + exception handler wired in  
  migrations/  
    0006_knowledge_chunks_and_vector.sql NEW  
  seed/  
    knowledge_docs/*.md                  NEW -- 8 example documents  
    seed_knowledge_base.py               NEW  
  scripts/  
    evaluate_retrieval.py                 NEW  
  tests/
```

test_rag_chunking.py	NEW
test_rag_embeddings.py	NEW
test_rag_extraction.py	NEW
test_rag_ingestion.py	NEW
test_rag_retrieval.py	NEW
test_knowledge_api.py	NEW
requirements.txt	EXTENDED -- python-multipart, pgvector, pypdf, python-docx
requirements-dev.txt	EXTENDED -- reportlab (test-fixture generation only)
.env.example	EXTENDED -- RAG settings documented
docs/	
rag-manual/	NEW -- this manual
.gitignore	EXTENDED -- local knowledge-file storage excluded
README.md	EXTENDED -- a Phase 7 section added

Every one of these is explained in the chapters that follow, in the order a reader building an understanding of the system from scratch would actually want them.

Ticket History vs. RAG

Why this distinction gets its own chapter

phase7.md calls this distinction "critical," and it is worth taking that word seriously before writing a single line of code, because it's the single easiest mistake to make when building a RAG system on top of an application that already has a rich history of past interactions. It would be tempting -- and wrong -- to think "we already have a database full of past tickets and their resolutions, why not just embed all of that and call it the knowledge base?" This chapter explains precisely why that would be wrong, and how this implementation avoids it.

Two different questions

Every retrieval system answers some question about "what is relevant right now." In this application, there are actually two different questions that sound similar but are not:

- **"What happened on this specific ticket, with this specific person, so far?"** -- answered by PostgreSQL's existing tables (`tickets`, `messages`, `ticket_events`), queried directly, with no embedding or similarity search involved at all. This is `ticket_service.get_ticket_history()`, already built in an earlier phase, untouched by this one.
- **"What does the IT department officially know about this general category of problem?"** -- answered by the RAG knowledge base built in this phase (`knowledge_documents` + `knowledge_chunks`, queried via `retrieval_service.search()`).

phase7.md's own example makes the distinction concrete. A student writes:

"My VPN is not connecting."

The ticket history for that student's ticket might contain:

```
Student:
VPN doesn't work.

Admin:
Did you restart the VPN client?

Student:
Yes, still doesn't work.
```

This is a record of one conversation between one student and one admin about one incident. It is useful -- a future AI agent will absolutely want to see it, so it doesn't ask the student to repeat something they already said -- but it is not "knowledge." It doesn't generalize. It doesn't tell you anything about how to fix a *different* student's VPN problem next week.

What RAG should retrieve for the same query is:

VPN Troubleshooting Documentation

-- the officially maintained guide that applies to every VPN problem, not just this one conversation. That's `knowledge_chunks` rows belonging to the `VPN Troubleshooting Guide` document in this project's actual seeded knowledge base (`backend/seed/knowledge_docs/vpn_troubleshooting_guide.md`), which

this manual's Worked Example chapter traces through the full pipeline.

Why conflating the two would actively make the system worse, not just redundant

It's worth being specific about the failure mode, because "these are conceptually different" undersells how much conflating them would actually break:

- **Ticket conversations are noisy and unresolved by default.** A raw ticket transcript contains false starts, wrong guesses ("did you restart the VPN client?" -- which, in the example above, *didn't* fix it), incomplete information, and sometimes outright wrong troubleshooting steps an admin tried before finding the real fix. Embedding that directly into a "knowledge base" and retrieving it as if it were authoritative documentation would surface wrong or unresolved advice with the same confidence as the actual VPN Troubleshooting Guide -- there would be no way for a retrieval score to distinguish "this dead-end guess from an unresolved ticket" from "this is the department's verified fix."
- **Ticket data contains information tied to a specific person.** A ticket transcript can reference a student's name, their specific device, their specific error details, sometimes account information. A knowledge base document is written to be general-purpose and safe to retrieve for anyone's similar problem. Treating ticket transcripts as retrievable knowledge risks surfacing one student's personal ticket detail into a different student's context -- a privacy problem this design avoids entirely by construction, not by a filter bolted on afterward.
- **It would make the retrieval evaluation meaningless.** Chapter 15 (Retrieval Evaluation) measures whether a query retrieves the *correct, known-good* document. That evaluation only means something if the corpus being searched is made of vetted, authoritative documents. If the corpus were a mix of official guides and arbitrary past ticket transcripts, "did the right thing come back" would no longer have a clear answer -- there would be no ground truth to evaluate against.
- **It duplicates data that already has a better home.** Ticket history is already fully queryable, indexed, and structured in PostgreSQL via `ticket_service.get_ticket_history()`. There is no retrieval-quality benefit to also embedding it -- only cost (embedding every ticket message, forever, as it accumulates) and risk (the problems above) with no corresponding benefit, since a direct SQL lookup by `ticket_id` is strictly more precise than a similarity search for "find this exact ticket's own history."

How this shows up in the actual implementation

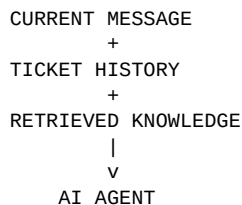
This isn't just a design principle stated in a comment -- it's enforced by what the code simply does not do:

- Nothing in `backend/app/rag/ingestion_service.py` accepts a `ticket_id`, a `message_id`, or any other reference to ticket data. Its `ingest_document()` function's parameters are exactly what an administrator uploading an IT document would supply: `title`, `description`, `category`, `file_name`, `file_type`, `file_bytes`. There is no code path from `tickets/messages/ticket_events` into `knowledge_documents/knowledge_chunks` anywhere in this codebase.

- The only way a row ever appears in `knowledge_documents` is through `POST /admin/knowledge` (an explicit, admin-authorized upload) or the seed script (`backend/seed/seed_knowledge_base.py`, which ingests the eight curated example documents). Nothing runs automatically against ticket data.
- `retrieval_service.search()` only ever queries `knowledge_chunks` joined to `knowledge_documents`. It has no awareness that `tickets` or `messages` tables exist.

What the future AI agent phase is expected to do with both

This phase doesn't build the AI agent, but `phase7.md` is explicit about what a *later* phase's context assembly should eventually look like, and it's worth stating here because it's the payoff for keeping these two sources separate now:



Three distinct inputs, from two distinct systems, combined only at the very last step, by the agent, when it actually reasons about a response. Ticket history answers "what has this specific person already told us." Retrieved knowledge answers "what does the department know about this kind of problem in general." Keeping them as separate inputs -- rather than merging them into one search index -- is what lets a future agent reason about them differently: it can trust the knowledge-base result as authoritative while treating ticket history as context about what's already been tried, and it can cite "per the VPN Troubleshooting Guide" without confusing that with "per what an admin guessed in a different, unrelated ticket last month."

This phase builds only the `RETRIEVED KNOWLEDGE` box. `CURRENT MESSAGE` and `TICKET HISTORY` already exist from earlier phases, untouched. Combining all three into one context object is explicitly left to the future AI agent phase (see the Architecture Boundaries chapter for exactly what's built versus what's deferred).

Data Model

Design goal: two tables, not many

phase7.md is explicit: "Do not unnecessarily create many tables." This implementation has exactly two tables for the entire knowledge base -- `knowledge_documents` and `knowledge_chunks` -- plus reuses two columns' worth of extension on the first of those, and nothing else. There is no separate "categories" table, no "tags" table, no "versions" table. Category is a plain string column. Versioning is handled with two columns on the existing document row (explained below), not a separate history table. This is a deliberate minimalism, consistent with how the rest of this codebase is built: `enums.py`'s own docstring explains that this project prefers a CHECK constraint on a VARCHAR column over either a separate lookup table or a native Postgres ENUM type, specifically so that adding a new valid value later is an ordinary migration, not a schema redesign.

`knowledge_documents`: already existed, extended by two columns

This table is not new to Phase 7. It was created in the very first migration of this project (`migrations/0001_initial_schema.sql`), with a comment explaining it was a placeholder for exactly this later phase: *"knowledge_documents: metadata for IT documentation a later RAG phase will chunk and embed. No chunks/embeddings table yet."* Phase 7's job with this table was to finish what it was left waiting for -- populate it for real, and add the two columns its original design didn't yet need.

The table, as it stands after Phase 7's migration:

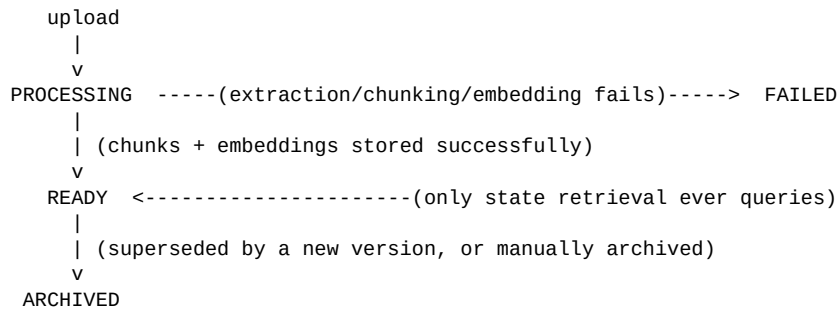
Column	Type	Nullable	Purpose
<code>id</code>	UUID	No (PK)	Primary key.
<code>title</code>	VARCHAR(500)	No	Human-readable document title (e.g. "VPN Troubleshooting Guide"). The duplicate-detection and versioning logic (see below) keys off this field.
<code>description</code>	TEXT	Yes	Free-text description, optional.
<code>category</code>	VARCHAR(100)	Yes	Free-text category (e.g. "VPN", "WIFI"). No CHECK constraint tying it to a fixed enum -- see "Why category is free text," below.
<code>file_name</code>	VARCHAR(255)	No	The original uploaded filename.
<code>file_type</code>	VARCHAR(50)	No	One of <code>pdf</code> , <code>txt</code> , <code>md</code> , <code>docx</code> (enforced in application code, not a CHECK constraint -- see the Error Handling chapter).

Column	Type	Nullable	Purpose
<code>storage_location</code>	VARCHAR(1000)	No	Where the original file bytes actually live -- a local filesystem path or an <code>s3://bucket/key</code> URI, depending on the configured storage backend (Chapter 7).
<code>version</code>	INTEGER	No, default 1	Version number within this document's title lineage. Increments by 1 with each re-upload via <code>replace_document_id</code> .
<code>status</code>	VARCHAR(20)	No, default <code>PROCESSING</code>	One of <code>PROCESSING</code> , <code>READY</code> , <code>FAILED</code> , <code>ARCHIVED</code> (CHECK-constrained). See below.
<code>previous_version_id</code>	UUID	Yes	New in Phase 7. Self-referencing foreign key to the document this row's version superseded, or <code>NULL</code> for a version-1 document.
<code>error_message</code>	TEXT	Yes	New in Phase 7. Set only when <code>status = FAILED</code> ; explains why ingestion failed. <code>NULL</code> in every other status.
<code>uploaded_by</code>	UUID	Yes	The admin who uploaded this document. No foreign key -- see "Why no foreign key," below.
<code>created_at / updated_at</code>	TIMESTAMP TZ	No	Standard timestamps; <code>updated_at</code> auto-maintained by the same <code>set_updated_at()</code> trigger every other table in this project uses.

The `status` lifecycle

phase7.md's own vocabulary talks about documents being `ACTIVE` or `ARCHIVED`. This implementation's actual status values are `PROCESSING`, `READY`, `FAILED`, and `ARCHIVED` -- a superset that existed in the schema (as an enum in `app/enums.py`, `KnowledgeDocumentStatus`) before Phase 7 began, left over from the same Phase 1 placeholder. Rather than rename anything, this implementation maps phase7.md's conceptual "ACTIVE" onto the existing `READY` value: a `READY` document is exactly the phase7.md concept of an active, retrievable document. The extra two states (`PROCESSING`, `FAILED`) exist because phase7.md's own error-handling requirements ("do not leave partially ingested documents in an

ACTIVE state") need somewhere for a document to sit while ingestion is in progress, and somewhere for it to land if ingestion fails -- see Chapter 13 (Error Handling) for the full failure-handling design this makes possible.



A document can only reach **READY** once its chunks and embeddings are fully and successfully stored -- there is no code path that sets `status = READY` before that work is done. Only **READY** documents are ever considered by `retrieval_service.search()` (Chapter 9); **PROCESSING**, **FAILED**, and **ARCHIVED** documents are all excluded from search results, for different reasons (still being ingested, ingestion didn't succeed, and superseded/manually retired, respectively) but with the same practical effect.

Why `previous_version_id`, not just a bumped `version number`

phase7.md section 9 is specific about the requirement: "We should not destroy historical knowledge blindly... Design the system so an updated document can replace/deactivate the previous version." A `version` integer column alone (which already existed pre-Phase-7) can tell you *that* a document is at version 3, but not *which specific row* was version 2 -- if two documents happened to share a title independently, or if history needed to be traced backward, there'd be no link. `previous_version_id` makes the version chain explicit and queryable: given any document, you can walk backward through its full version history by following `previous_version_id` until you reach a row where it's **NULL** (the original version 1). This is a single nullable self-referencing foreign key, not a separate versions table, in keeping with the "don't create unnecessary tables" goal -- see Chapter 7 for exactly how and when this column gets set during ingestion.

Why `category` is a free-text column, not a foreign key to a categories table or a native enum

Two reasons, both consistent with existing conventions elsewhere in this codebase (see `app/enums.py`'s own docstring, which makes the identical argument for `ticket_events.event_type` and `agent_actions.action_type`): the set of useful categories for a knowledge base is going to grow as more departments and topics get documented, and neither a foreign-key lookup table nor a native Postgres **ENUM** type make adding a new value as cheap as it should be (a native enum needs `ALTER TYPE ... ADD VALUE`, with its own transactional quirks; a lookup table needs an extra join on every query). The eight example documents in this project reuse the existing `TicketCategory` enum's string values (`VPN`, `WIFI`, `PASSWORD`, etc.) as a matter of consistency -- so a future AI agent's category-scoped search lines up with the vocabulary tickets already use -- but nothing in the database enforces that; it's a convention, not a constraint.

Why `uploaded_by` has no foreign key

This follows a pattern already established everywhere else in this codebase for admin-actor columns: `tickets.assigned_admin_id`, `ticket_events/messages'` actor references, and the original (pre-Phase-7) `knowledge_documents.uploaded_by` column are all plain `UUID` columns with no foreign-key constraint, because -- as the project's own README explains -- this project had no `admins` table at all until Phase 3, and by the time it existed, adding an FK retroactively to every already-shaped column wasn't worth a migration for columns that already worked correctly without one. Phase 7 didn't change this convention; it just used the same shape for a new document row's `uploaded_by`.

`knowledge_chunks`: new in Phase 7

The table `knowledge_documents` was always waiting for:

Column	Type	Nullable	Purpose
<code>id</code>	UUID	No (PK)	Primary key.
<code>document_id</code>	UUID	No	Foreign key to <code>knowledge_documents.id</code> , <code>ON DELETE CASCADE</code> .
<code>content</code>	TEXT	No	The chunk's actual text content -- what gets embedded and what gets returned to a caller.
<code>chunk_index</code>	INTEGER	No	Position of this chunk within its document, starting at 0.
<code>token_count</code>	INTEGER	No	Approximate token count for this chunk (see the Chunking chapter for exactly how this is computed and why it's an approximation).
<code>embedding</code>	vector(1024)	Yes*	The chunk's embedding vector. <i>Nullable only for the brief instant between two SQL statements inside a single ingestion transaction -- a chunk row is never inserted without its embedding already computed; there is no code path that leaves a committed* row with a <code>NULL</code> embedding.</i>
<code>created_at</code>	TIMESTAMPTZ	No	Standard timestamp.

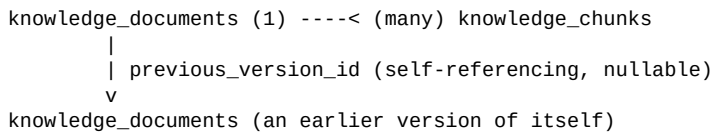
Plus a `UNIQUE (document_id, chunk_index)` constraint (two chunks of the same document can never share a position), an index on `document_id` (fast "give me all chunks for this document" lookups, used by `GET /admin/knowledge/{id}/chunks`), and an HNSW index on `embedding` (fast approximate nearest-neighbor search, used by every retrieval query -- see Chapter 6).

`ON DELETE CASCADE` from `knowledge_chunks.document_id` to `knowledge_documents.id` means a hard delete of a document (`DELETE /admin/knowledge/{id}?hard=true`) automatically removes every one of its chunks in the same statement -- there's no orphaned-chunk cleanup step needed anywhere in the application code, because the database itself guarantees it.

Why 1024 dimensions, specifically

A pgvector `vector` column's dimension is fixed at `CREATE TABLE` time -- it's part of the column's type, not a value that varies row to row. 1024 was chosen because it's the default (and a supported) output size of Amazon Titan Text Embeddings V2, the model this project's production embedding provider (Chapter 5) calls. The local development provider (also Chapter 5) is configured to produce the same dimension, specifically so the exact same `knowledge_chunks` schema, the exact same SQL queries, and the exact same tests work identically regardless of which embedding provider is active -- nothing about the database schema needs to know or care which provider actually produced a given vector.

Entity relationship, in one picture



One document has many chunks (`ON DELETE CASCADE`). A document optionally points to the document it superseded, forming a version chain. Nothing else references either table, and neither table references anything outside itself except `admins` (informally, via the unconstrained `uploaded_by` column).

Migration mechanics

Like every other schema change in this project, this one is a single plain `.sql` file (`migrations/0006_knowledge_chunks_and_vector.sql`), applied by the project's existing tiny migration runner (`migrations/run_migrations.py`) -- no Alembic, no ORM migration tooling, consistent with the rest of the project's deliberate "no ORM" choice. The full text of this migration is reproduced in the Appendix (Schema Reference chapter) for exact reference. Applying it is one command from `backend/`: `python -m migrations.run_migrations` -- it was run for real against a live `pgvector/pgvector:pg16` Postgres container during this implementation, and applied cleanly on the first attempt, alongside the five pre-existing migrations from earlier phases, with no conflicts.

Chunking Strategy

Why chunk at all

An embedding model compresses a piece of text into one fixed-size vector -- for this project, 1024 numbers, regardless of whether the input text is one sentence or twenty pages. Feed an entire multi-page VPN guide into an embedding model as a single unit, and that single vector has to somehow represent "how to install the client" *and* "how to troubleshoot a connection failure" *and* "what each specific error code means" all at once, averaged together into one point in a 1024-dimensional space. A student's query about one specific error code produces a query vector that's semantically close to only a *fraction* of what that averaged document vector represents -- diluted by everything else the document also talks about. The result is retrieval that's directionally right (a VPN question will still tend to surface VPN documents over WiFi documents) but imprecise in exactly the way that matters most: it can't tell you *which part* of a long document is actually relevant, and a genuinely relevant long document can lose a close race against a shorter, more topically narrow one that happens to average closer to the query.

Splitting a document into smaller chunks before embedding fixes this by giving each embedding a single, coherent idea to represent. A chunk about "VPN-ERR-403: Authentication Failed" produces a vector that's *only* about that -- so a query about an authentication failure scores that chunk highly and correctly leaves the installation-instructions chunk of the same document out of the way, even though both chunks came from the same source file.

Why not just split every N characters

A naive fixed-character-count split (split every 2000 characters, regardless of what's there) would solve the "one vector, too much content" problem, but introduces a worse one: it has no idea where one idea ends and another begins. It will, with some regularity, cut a numbered instruction list in half -- leaving a chunk that reads "...3. Click OK. 4." with no idea what step 4 actually says, because that text landed in the *next* chunk instead. `phase7.md` section 5 explicitly asks for boundary-aware splitting, and this is why: a chunk that's syntactically valid text but semantically incoherent is worse than useless for retrieval -- it can score well on a query (it still shares vocabulary with the topic) while handing back content that doesn't actually answer anything.

The actual algorithm (`backend/app/rag/chunking.py`)

The implementation (`chunk_text()`) works in two passes, deliberately structured so that boundary-awareness comes *first* and size-based packing comes *second*, never the other way around:

Pass one: split into whole blocks. The input text is split on blank lines into blocks -- each block being a paragraph, a heading plus the paragraph immediately following it, or a run of bullet-list lines. A heading line by itself is never emitted as its own zero-content block; it's attached to the block that follows it, so `### VPN Errors\n\nAuth failures happen when...` stays together as one unit rather than becoming a heading with no content and a content block with no context. This pass never looks at size at all -- it only looks at structure.

Pass two: pack whole blocks into chunks up to a token budget. Blocks are then packed greedily: keep adding whole blocks to the current chunk until adding the next one would exceed the configured size limit, then start a new chunk. Critically, a block is *never split* in this pass -- if a block doesn't fit, it starts a new chunk rather than being cut in half. This is what guarantees a chunk boundary never falls in the middle of a bullet list or a short paragraph.

The one exception: an oversized single block. If a single block -- one long paragraph with no internal blank lines -- is itself larger than the entire chunk size budget, something still has to give, because that block can't be packed whole and can't be skipped. In that case, and only that case, the block is split at *sentence boundaries* (using a regex that looks for `.`, `!`, or `?` followed by whitespace), packing whole sentences up to the budget. This is a deliberate last resort: it never fires for a normally-structured document (one with reasonable paragraph breaks), and even when it does fire, it still never cuts a sentence in half -- the worst case is a chunk boundary between two related sentences, not inside one.

Overlap: carrying context across a chunk boundary

Even with boundary-aware splitting, a chunk boundary can still fall between two sentences that depend on each other -- "Restart the VPN client, then try connecting again" split so that "then try connecting again" starts a new chunk with no idea what "then" refers to. To reduce this, each new chunk (after the first) is prefixed with the trailing words of the *previous* chunk's last block, up to a configurable overlap budget. This means the same handful of words appear at the tail of one chunk and the head of the next -- a small amount of deliberate redundancy that keeps each chunk more self-contained when it's later retrieved and shown to a reader (or, eventually, an AI agent) independently of its neighbors, out of original document order.

Configuration, and why these specific defaults

Chunk size and overlap are both plain function parameters to `chunk_text()`, driven in turn by two settings (`RAG_CHUNK_SIZE_WORDS`, default 600; `RAG_CHUNK_OVERLAP_WORDS`, default 80) rather than hardcoded constants anywhere in the pipeline. `phase7.md` section 5 is explicit about this: "*Do not blindly assume these values are optimal... Make the values configurable.*" These defaults sit inside `phase7.md`'s own suggested starting range (roughly 500-1000 tokens, with 50-150 tokens of overlap) without picking either extreme, but they are not claimed to be the *correct* values for every corpus -- they are a reasonable starting point, and Chapter 15 (Retrieval Evaluation) is explicitly the tool this project provides for actually testing whether a different value helps, rather than guessing: re-run `python -m scripts.evaluate_retrieval` before and after changing either setting and compare the Recall@K/Precision@K numbers.

Tokens vs. words: an honest approximation

This project has no tokenizer dependency -- no `tiktoken`, no model-specific tokenizer library. Adding one purely to count tokens more precisely than a word count was judged not worth the extra dependency for this phase. Instead, `count_tokens_approx()` estimates token count as `words * 1.3` -- a standard rough conversion factor for English text (roughly 0.75 words per token on average for common tokenizers, i.e. about 1.3 tokens per word), documented explicitly as an approximation rather

than presented as exact. Every "tokens" concept referenced anywhere in the chunking module -- chunk size, overlap, the `token_count` column stored per chunk -- is in this approximated unit. If a real tokenizer is added to this project later (for instance, to precisely track cost against a specific LLM's actual token accounting), it's a self-contained change: `count_tokens_approx()` is the one function that would need to change, and `chunk_text()`'s signature and the shape of the `Chunk` objects it returns stay exactly the same.

The tradeoff, stated plainly

Bigger chunks retain more surrounding context per chunk -- fewer "which specific VPN error is this troubleshooting step even about" ambiguities when a chunk is read in isolation -- but dilute the embedding (more competing sub-topics packed into one vector, worse retrieval precision for a narrow query) and cost more tokens if the chunk is later fed to an LLM. Smaller chunks are the mirror image: sharper, more precise retrieval, but more fragmented context and more total chunks to embed and store per document. More overlap reduces the chance a boundary falls exactly on the information a query needs, at the direct cost of redundant text that gets embedded and stored twice. None of these tradeoffs have a universally correct answer -- they depend on the actual shape of the documents being ingested, which is exactly why the values are configurable and why Chapter 15's evaluation harness exists as the actual mechanism for deciding, empirically, whether a given corpus benefits from different values.

A worked example, using this project's real seeded content

The `VPN Troubleshooting Guide` (`backend/seed/knowledge_docs/vpn_troubleshooting_guide.md`) is a real, moderately long document with distinct sections: `## Overview`, `## Installing Cisco AnyConnect`, `## Connecting to the VPN`, `## Common VPN Errors` (containing three `###` subsections, including `### VPN-ERR-403: Authentication Failed`), and `## When to Escalate`. At the default 600-word/80-word-overlap settings, this document -- roughly 550-600 words total -- packs into either one or two chunks, depending on exactly how its blocks fall relative to the word budget; when a real ingestion run of this document was inspected via `GET /admin/knowledge/{id}/chunks`, its content stayed grouped along exactly these section boundaries, never splitting a bullet list or a numbered installation step across two chunks. The Worked Example chapter traces this specific document through the complete pipeline, including its actual observed chunk content and a real retrieval query's scored results against it.

What this module is tested against

`tests/test_rag_chunking.py` (6 tests, pure unit tests requiring no database) verifies every claim made in this chapter directly against the code, not just by inspection: that empty input produces zero chunks; that a short input produces exactly one chunk with correctly computed metadata; that a long, multi-paragraph input actually splits into multiple chunks, each respecting the configured size budget; that overlap is not just configured but demonstrably *applied* (the tail words of one chunk are shown to reappear at the head of the next); that a bullet list short enough to fit isn't needlessly split; and that an artificially oversized single block with no paragraph breaks splits at sentence boundaries, with every resulting chunk verified to end on a sentence boundary (a period), never mid-sentence.

Embeddings

The requirement: swappable, not hardcoded

phase7.md section 6 states the requirement precisely: *"Design the embedding service so that the provider can be changed later without rewriting the whole application... Do not hard-code embedding logic throughout the application."* The sketch it gives is minimal on purpose:

```
EmbeddingService
|
generate_embedding(text)
```

This implementation follows that sketch exactly, and follows an abstraction pattern that already existed elsewhere in this codebase for the identical problem: `app/email/provider.py`'s `EmailProvider` abstract base class, with `DevEmailProvider` and `AWSSESProvider` as swappable concrete implementations selected by a single setting (`EMAIL_PROVIDER`). The embeddings subsystem (`backend/app/rag/embeddings/`) mirrors that shape file-for-file:

```
embeddings/
provider.py      EmbeddingProvider (abstract base class) + EmbeddingError
dev_provider.py  DevEmbeddingProvider -- local development, no credentials
bedrock_provider.py BedrockEmbeddingProvider -- production, Amazon Titan
service.py       EmbeddingService + get_embedding_service()
```

Nothing outside this package -- not `ingestion_service.py`, not `retrieval_service.py` -- ever imports `DevEmbeddingProvider` or `BedrockEmbeddingProvider` directly. Every caller goes through `get_embedding_service()`, which reads a single setting (`EMBEDDING_PROVIDER`, "dev" or "bedrock") and returns the matching concrete provider wrapped in an `EmbeddingService`. Adding a third provider later -- OpenAI's embeddings API, for instance -- means writing one new file and adding one branch to this one function. It does not mean touching the ingestion pipeline, the retrieval query, or any test that doesn't specifically test the embedding provider itself.

The interface

```
class EmbeddingProvider(ABC):
    dimensions: int

    @abstractmethod
    def embed_text(self, text: str) -> list[float]:
        ...

    def embed_batch(self, texts: list[str]) -> list[list[float]]:
        return [self.embed_text(text) for text in texts] # default; a provider may override
```

`embed_text()` must raise `EmbeddingError` on failure -- never return a zero vector, never return a partial or truncated result silently. This matters because phase7.md section 19 explicitly lists "Embedding API failure" as a named error case the ingestion pipeline must handle, and it can only handle it correctly if a failure is unambiguous (an exception) rather than something the pipeline would have to guess at (a suspiciously-all-zero vector that embedded successfully as far as the type system is concerned). `embed_batch()` has a default implementation that just calls `embed_text()` once per item -- correct for any provider, and overridable by a provider whose underlying API genuinely supports a real batch request for lower latency. Amazon Bedrock's Titan Text Embeddings model, as used here, does not

support batching multiple inputs into a single `InvokeModel` call, so `BedrockEmbeddingProvider` doesn't override the default.

`DevEmbeddingProvider`: the local development provider, and why it's more than a stub

Selected by `EMBEDDING_PROVIDER=dev`, which is the project's default -- this is what every test, the evaluation harness, and a plain local `uvicorn` run actually exercises. No AWS credentials are needed to develop, test, or demo any part of this phase.

The easy version of a "dev" embedding provider would just return a random vector. That would be enough to make the *types* line up and the pipeline run without crashing, but it would make vector search itself untestable in any meaningful way -- every retrieval test would either pass by pure luck or would have to mock the database query directly, which defeats the entire point of testing against a real `pgvector` index. Instead, `DevEmbeddingProvider` implements a real, if crude, embedding algorithm: a deterministic **hashing-trick bag-of-words** embedding.

The algorithm, concretely:

1. Lowercase the input text and split it into word tokens (a simple `[a-z0-9]+` regex).
2. For each token, compute its SHA-256 hash (deliberately *not* Python's built-in `hash()` function -- that's randomized per-process via `PYTHONHASHSEED` unless explicitly disabled, which would make the same text produce a *different* embedding on two separate runs of the same test suite; SHA-256 is stable across processes and machines).
3. Use the first 4 bytes of that hash, modulo the target dimension count, to pick one of `dimensions` "buckets" for that token.
4. Use one more byte of the same hash to decide a deterministic +1 or -1 sign, added into that bucket (the sign reduces a systematic bias that a pure "always +1" hashing scheme would otherwise introduce).
5. After every token has been added into its bucket, L2-normalize the resulting vector to unit length.

The result is a genuine bag-of-words vector: two pieces of text that share vocabulary end up with a demonstrably higher cosine similarity than two pieces of text that share none. That property -- not randomness, not a mock -- is what lets `tests/test_rag_embeddings.py` assert, and actually verify, that "VPN authentication failed" scores closer to "VPN authentication failure and connection troubleshooting steps" than to "printer paper jam and toner replacement instructions." It's also what makes the retrieval evaluation numbers in Chapter 15 real numbers about a real (if simple) retrieval mechanism, not numbers about a mock.

To be direct about the limitation, because this manual does not overstate what this provider does: `DevEmbeddingProvider` has no notion of synonyms, no understanding that "can't connect" and "authentication failed" are related concepts unless they happen to share literal vocabulary. It is not a semantic embedding. It is a deliberate, honestly-labeled stand-in for local development and testing, in exactly the same spirit as `DevEmailProvider`'s fabricated `Message-ID` is not a real one -- good enough to prove the *pipeline* works correctly end to end, not a claim about production retrieval quality.

BedrockEmbeddingProvider: the production provider

Selected by `EMBEDDING_PROVIDER=bedrock`. Calls Amazon Bedrock's `InvokeModel` API against `amazon.titan-embed-text-v2:0` (configurable via `BEDROCK_EMBEDDING_MODEL_ID`), using `boto3`'s `bedrock-runtime` client -- `boto3` is already a project dependency (used by the existing SES email integration from an earlier phase), so this adds no new AWS SDK, only a new service client.

The request body sent to Bedrock:

```
{
  "inputText": "the chunk or query text",
  "dimensions": 1024,
  "normalize": true
}
```

`dimensions: 1024` matches the `vector(1024)` column defined in the schema (Chapter 4) -- Titan Text Embeddings V2 supports 256, 512, or 1024-dimension output, and this project uses the largest of the three. `normalize: true` asks Bedrock to return an already unit-length vector, which matters for a specific, concrete reason: with normalized vectors, cosine similarity and a plain dot product become mathematically equivalent, which avoids an entire class of scoring bugs where an un-normalized embedding could be gamed simply by being "longer text -> larger vector magnitude -> higher raw dot-product score" regardless of actual relevance. Requesting normalization from Bedrock directly, rather than normalizing client-side after the fact, means this guarantee holds even if the raw API response is ever consumed in some other context later.

Any failure from the Bedrock call -- a throttling exception, a network error, a malformed response missing the expected `embedding` field -- is caught and re-raised as a single `EmbeddingError`, deliberately collapsing botocore's many distinct exception types into one the rest of the pipeline only has to know how to handle once. `boto3`'s client construction is deferred (imported lazily inside the class, not at module import time), so importing this module -- which happens automatically whenever `EMBEDDING_PROVIDER=dev` is selected, since `service.py` imports both provider modules at startup -- never requires AWS credentials to be configured at all when Bedrock isn't the active provider.

EmbeddingService: the thin wrapper everything actually calls

```
class EmbeddingService:
    def generate_embedding(self, text: str) -> list[float]: ...
    def generate_embeddings(self, texts: list[str]) -> list[list[float]]: ...

def get_embedding_service() -> EmbeddingService:
    if settings.embedding_provider == "bedrock":
        from app.rag.embeddings.bedrock_provider import BedrockEmbeddingProvider
        return EmbeddingService(BedrockEmbeddingProvider())
    return EmbeddingService(DevEmbeddingProvider())
```

This is the entire surface area `ingestion_service.py` and `retrieval_service.py` interact with -- `generate_embedding()` for a single query string during retrieval, `generate_embeddings()` for a batch of chunk contents during ingestion. Exactly the shape `phase7.md` section 6 asked for, and exactly the mechanism that makes the provider swap a one-line environment variable change rather than a code change.

What this is tested against

`tests/test_rag_embeddings.py` (6 tests) verifies, directly against `DevEmbeddingProvider`: determinism (embedding the same text twice produces an identical vector -- proving no hidden randomness leaks in); that the output has exactly the requested dimension count; that the output is genuinely unit-length (L2 norm within floating-point tolerance of 1.0); that `embed_batch()` produces results identical to calling `embed_text()` once per item; that even empty-string input still produces a valid unit vector rather than crashing or dividing by zero; and, most importantly, that semantically similar text scores a measurably higher cosine similarity than dissimilar text -- the one property that makes this provider a genuine, testable stand-in rather than noise dressed up as an embedding.

Vector Storage

Why PostgreSQL + pgvector, not a separate vector database

phase7.md section 7 is explicit: "Use PostgreSQL with a vector extension such as `pgvector`... Do NOT introduce Pinecone, OpenSearch, Elasticsearch, or another vector database unless the existing infrastructure makes PostgreSQL vector search clearly unsuitable... We want to keep the architecture understandable."

This project's own `docs/05-tech-stack.md` (written before this phase, during the project's original planning) already makes the substantive case for this, and it's worth restating because it's the actual reasoning, not just a rule being followed: a future AI agent needs to combine three different kinds of retrieval in a single reasoning turn -- semantic search over knowledge-base chunks (this phase), semantic search over historical ticket resolutions (a plausible future phase), and ordinary structured queries against users/tickets/devices/permissions (already built). Doing all three as one SQL query against a single database (`ORDER BY embedding <=> query_embedding LIMIT 5 WHERE status = 'READY'`) is simpler, and more consistent, than round-tripping between a relational database and a separate vector service and joining the results together in application code. And because `retrieval_service.search()` is the one and only call site that touches `knowledge_chunks`' embedding column, the storage engine behind it could change later without touching anything upstream of it -- the abstraction boundary that matters here isn't "which vector database," it's "this one function."

There's also a concrete practical reason this decision was easy: the project's `docker-compose.yml` was already pinning the `pgvector/pgvector:pg16` image back in Phase 1, specifically in anticipation of this phase, so no new infrastructure had to be introduced at all -- only a `CREATE EXTENSION` statement in a migration.

Enabling the extension

One line, in `migrations/0006_knowledge_chunks_and_vector.sql`:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Since the Docker image already ships `pgvector`'s compiled extension files, this is "turning the extension on," not "installing new software" -- confirmed during this implementation by querying the running container directly: `pgvector` version `0.8.6` was already available and ready to enable.

The distance metric: cosine distance, explained from first principles

`pgvector`'s `<=>` operator computes **cosine distance**, defined as `1 - cosine_similarity`. To make this concrete rather than just naming it: cosine similarity measures the angle between two vectors, ignoring their magnitude entirely -- it asks "do these two vectors point in the same direction in this 1024-dimensional space," which, for an embedding, corresponds to asking "do these two pieces of text mean approximately the same thing." Two identical vectors have a cosine similarity of 1 (angle of zero). Two unrelated vectors, in a high-dimensional space, tend to be close to orthogonal (similarity near 0).

Two vectors pointing in opposite directions have a similarity of -1. Cosine *distance* (what `<=>` actually computes) is just `1 - similarity`, so it ranges from 0 (identical direction -- most relevant) up to 2 (opposite direction -- least relevant), with the ordering flipped relative to similarity: **smaller distance means more relevant**.

This is why `retrieval_service.search()`'s query orders results with `ORDER BY embedding <=> query_embedding` -- ascending distance, which is the same as descending relevance -- and separately computes a `score` field as `1 - (embedding <=> query_embedding)`, converting the distance back into a similarity in the familiar "higher is better, roughly 0 to 1" range that phase7.md's own example response format shows (`"score": 0.89`). The `score` field returned to an API caller is always this similarity value, never the raw distance -- so "higher score is more relevant" is uniformly true everywhere a score appears in this system's output, even though the underlying SQL sorts by the inverted distance value.

Why cosine distance specifically, rather than Euclidean distance (pgvector also supports `<->`) or raw dot product (`<#>`)? Because both this project's embedding providers produce **normalized** (unit-length) vectors -- `DevEmbeddingProvider` explicitly L2-normalizes its output, and `BedrockEmbeddingProvider` requests `normalize: true` from Bedrock. For normalized vectors, cosine similarity and dot product become mathematically equivalent, and cosine distance becomes the natural, magnitude-independent choice: it measures pure semantic direction, never rewarding a chunk simply for being longer or shorter text.

Why HNSW, not IVFFlat

pgvector supports two approximate-nearest-neighbor index types for a `vector` column. This project's migration builds an HNSW index:

```
CREATE INDEX ix_knowledge_chunks_embedding_hnsw
ON knowledge_chunks USING hnsw (embedding vector_cosine_ops);
```

The choice matters specifically for the state the database is in immediately after this migration runs: it's empty. An IVFFlat index partitions the vector space into a fixed number of "lists," and those partitions are built from whatever data exists in the table *at index-creation time* -- built well on a large, already-populated table, and built poorly (or not meaningfully at all) on an empty or small one, typically requiring an explicit `ANALYZE` and, ideally, a rebuild once real data volume exists. HNSW has no such training step: it's a graph-based index built incrementally as rows are inserted, useful immediately regardless of how much data exists when it's created. Since this migration creates the index before a single row of data exists, and the actual amount of data (a handful of example documents' worth of chunks, growing gradually as an admin uploads more) never reaches a scale where IVFFlat's tuning tradeoffs would clearly win, HNSW was the correct default for this project's scale. `vector_cosine_ops` tells the index to optimize specifically for the `<=>` operator this project's queries actually use -- pgvector requires the index's operator class to match the distance operator used in the query, or the index simply won't be used at all.

From a Python list of floats to a queryable `vector` column: a genuine implementation subtlety

This is worth documenting explicitly because it was a real bug encountered and fixed during this implementation, and understanding it matters for anyone extending this code.

The `pgvector` Python package (installed as a dependency, providing the `psycopg2` adapter) offers `register_vector(conn)`, called once per pooled database connection in `app/database.py`'s `get_db()`. This function does two things: it registers a *read-side* type cast, so a `vector` column fetched back from a query automatically arrives in Python as a `list[float]` rather than a raw string -- this works correctly and is used throughout this project. But its *write-side* adapter, in the installed version of the package, is registered only for `numpy.ndarray` objects, not for plain Python `list` objects. Passing a plain Python list as a query parameter for a `vector`-typed column, without an explicit cast, causes `psycopg2` to fall back to its default behavior for a list: adapting it as a Postgres `ARRAY`, not a `vector` -- which fails outright the first time it's used against the `<=>` operator, with an error to the effect of `operator does not exist: vector <=> numeric[]`.

There were two ways to fix this: add `numpy` as a project dependency purely to satisfy one adapter registration, or handle the conversion explicitly without a new dependency. This implementation chose the second, in `app/rag/vector_literal.py`:

```
def to_pgvector_literal(vector: list[float]) -> str:
    return "[" + ", ".join(repr(float(component)) for component in vector) + "]"
```

Every place that writes or filters by an embedding -- inserting chunks in `ingestion_service.py`, and building the query in `retrieval_service.search()` -- converts the Python list to `pgvector`'s own text literal format (`"[0.1,0.2,...]"`) using this helper, and pairs it with an explicit `::vector` cast at the corresponding `%s` placeholder in the SQL (for example, `%s::vector`, or, for a bulk insert via `psycopg2.extras.execute_values`, a custom row template ending in `%s::vector`). Postgres parses `"[0.1,0.2,...]"::vector` exactly the way it would parse a `vector` literal written directly in a hand-typed query. This avoids a new dependency, keeps the conversion visible and explicit at every call site rather than hidden inside an adapter registration, and was verified to work correctly end to end -- both the ingestion insert path and the retrieval query path were run for real against a live `pgvector`-backed table during this implementation.

What the final picture looks like

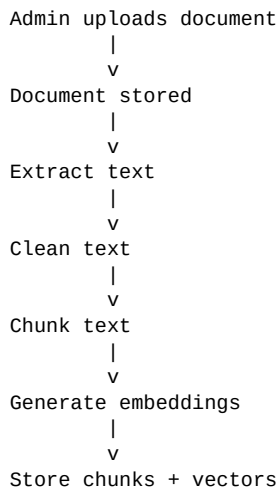
```
PostgreSQL
|-- users
|-- tickets
|-- messages
|-- ticket_events
|-- agent_actions
|-- devices
|-- admins
|-- knowledge_documents
|-- knowledge_chunks
    |-- embedding vector(1024), HNSW-indexed, vector_cosine_ops
```

One database. One connection pool (the same `app/database.py` pool every other table already uses). One query language. A developer who already understands how to query `tickets` in this codebase already knows, structurally, how to query `knowledge_chunks` -- it's a `RealDictCursor`-backed SQL string with `%s` placeholders, exactly like every other service function in this project, with one additional operator (`<=>`) and one additional cast (`::vector`) that this chapter has now fully explained.

Ingestion Pipeline

The pipeline, end to end

phase7.md section 8 lays out the ingestion flow at a high level:



`app/rag/ingestion_service.py`'s `ingest_document()` function is the single orchestration point that implements this entire flow -- it's the one function in this codebase that calls into every other RAG module (`storage.py`, `extraction.py`, `chunking.py`, `embeddings/service.py`), and it's the only one that does. Every other module in `app/rag/` does exactly one job and has no knowledge of the others; `ingestion_service.py` is where they're actually wired together.

Step 1: document storage -- local disk or S3, behind a swappable backend

Before any text extraction happens, the raw uploaded bytes are persisted, using the same provider-abstraction pattern this project already uses for email (Chapter 6) and now embeddings: `app/rag/storage.py` defines a `FileStorage` abstract base class with `save()` and `read()`, and two implementations -- `LocalFileStorage` (the default, `KNOWLEDGE_STORAGE_BACKEND=local`) writes to a directory on the API server's own disk, prefixing each filename with the document's UUID so two uploads named `guide.pdf` never collide; `S3FileStorage` (`KNOWLEDGE_STORAGE_BACKEND=s3`) uploads to a configured private S3 bucket under a `knowledge-documents/<document-id>/` key prefix, deliberately never passing any object-level ACL that could override the bucket's own access policy (see the Security and AWS Deployment chapters for why bucket-level privacy is the correct posture here, not per-object ACLs). Whichever backend is active, `save()` returns a single string -- a local path or an `s3://bucket/key` URI -- stored directly in the existing `storage_location` column. Nothing downstream of this step needs to know or care which backend actually wrote the file.

Step 2: text extraction -- format-specific, but not a pluggable abstraction

`app/rag/extraction.py`'s `extract_text()` dispatches on file type: plain UTF-8 decoding for `txt/md` (with `errors="replace"`, so malformed byte sequences degrade to replacement characters rather than crashing extraction outright), `pypdf`'s `PdfReader` for `pdf` (page text joined with blank lines between pages), and `python-docx`'s `Document` for `docx` (paragraph text joined the same way, skipping empty paragraphs). Unlike the embedding provider, this is deliberately **not** built as a swappable-provider abstraction -- there's no reasonable scenario where this project would swap out its PDF library at runtime based on a setting the way it swaps embedding providers. `python-docx` is imported lazily, inside the function that needs it, specifically so a deployment that never receives a DOCX upload never pays the cost of importing it -- keeping true to phase7.md section 8's instruction: *"If DOCX is easy to support, include it, but do not let DOCX support complicate the core design."*

Any failure during extraction -- a corrupted PDF, an unparseable DOCX -- is caught and re-raised as a single `ExtractionError`, regardless of which underlying library actually raised the original exception. This was verified directly during this implementation: uploading a file containing only the bytes `%PDF-1.4 not a real pdf` (a syntactically-plausible-looking but actually-invalid PDF header) through the real running API produced a document with `status: FAILED` and `error_message: "failed to extract text from pdf file: Stream has ended unexpectedly"` -- not a server error, not a crash, not a hung request; a clean, informative failure captured exactly where the Error Handling chapter says it should be.

Step 3: cleaning

`clean_text()` normalizes whitespace before chunking sees the text: Windows line endings (`\r\n`) are converted, trailing whitespace is stripped per line, and runs of three or more consecutive blank lines are collapsed down to one -- kept deliberately conservative, collapsing only excess blank lines rather than reformatting content, specifically so `chunking.py`'s block-boundary detection (which relies on blank lines to find paragraph breaks) continues to work predictably. Markdown structure -- heading markers, bullet markers -- is explicitly *not* stripped or altered here, because the next step's boundary detection depends on that structure still being present.

Step 4 and 5: chunking and embedding

Covered in full in Chapters 4 and 5 respectively; `ingestion_service.py`'s role here is simple orchestration -- call `chunking.chunk_text()` with the configured size/overlap, then call `EmbeddingService.generate_embeddings()` once with the full list of chunk contents (a single batched call, not one call per chunk, even though the default `embed_batch()` implementation loops internally -- this keeps the call site simple regardless of whether a future provider adds real network-level batching).

Step 6: storing chunks and flipping status to `READY`

Chunk rows are inserted in a single bulk statement via `psycopg2.extras.execute_values`, each embedding converted to pgvector's text-literal format first (Chapter 6 explains exactly why this conversion is necessary). Only after every chunk has been successfully inserted does the document's `status` flip to `READY` -- there is no earlier point in this function where that happens, which is the concrete mechanism behind phase7.md section 19's requirement that a document never appear `ACTIVE/READY` while only partially ingested.

Document versioning: how a re-upload actually works

phase7.md section 9's requirement -- an updated document replaces the old one without destroying it -- is implemented as follows. `ingest_document()` accepts an optional `replace_document_id` parameter:

- **Without it**, the function first checks whether a `READY` document with the same `title` already exists. If one does, the upload is rejected outright with a `ConflictError` (mapped to HTTP 409) *before* any new document row, any file storage write, or any processing happens at all -- a plain duplicate-title upload is treated as a probable mistake, not silently accepted as a second competing "active" version.
- **With it**, the caller is explicitly saying "this upload is a new version of that specific existing document." The new document row is created with `version = old.version + 1` and `previous_version_id` pointing at the document being replaced. Ingestion then proceeds normally (extract, clean, chunk, embed, store) against the *new* row. Only **after** the new version successfully reaches `READY`, the *old* version's status is flipped to `ARCHIVED`.

That last detail is the entire point of the design, and it's worth being explicit about why the ordering matters: if the old version were archived *first* (or at the same time the new version starts processing), and the new version's ingestion then failed for any reason -- a corrupted replacement file, an embedding-provider outage -- the knowledge base would be left with **no working version of that document at all** during the window before someone notices and re-uploads correctly. By archiving the old version only after the new one is confirmed `READY`, the previously-working version stays fully queryable for the entire duration of the replacement attempt, and a failed re-ingestion simply results in a new `FAILED` row sitting alongside the still-working old one -- zero downtime, and the failure is visible (via `GET /admin/knowledge/{id}` on the new row) rather than silent. This exact sequence -- successful new version reaching `READY`, immediately followed by the old version flipping to `ARCHIVED` -- was verified end to end against a real running instance during this implementation, both via direct Python calls in `tests/test_rag_ingestion.py` and via a full HTTP round-trip in `tests/test_knowledge_api.py`.

Failure handling: the SAVEPOINT, explained in depth

This is the single most subtle piece of engineering in the ingestion pipeline, and it deserves a full explanation rather than a one-line comment, because getting it wrong would be easy and the failure mode would be confusing to debug.

Once a document row has been inserted (`status = PROCESSING`), a failure anywhere in the rest of the pipeline -- extraction, chunking producing zero chunks, an embedding-provider error, or a database error partway through inserting chunks -- must **not** abort the entire request. The document row itself, updated to `status = FAILED` with a descriptive `error_message`, is a normal, useful, expected outcome: an admin looking at `GET /admin/knowledge` should see exactly why a specific upload didn't succeed, not get a generic HTTP 500 with no persisted record of what happened at all.

A naive `try/except` around the risky part of the pipeline looks like it should handle this -- catch the exception, then run an `UPDATE ... SET status = 'FAILED'` inside the `except` block. But this breaks specifically when the failure is a **database** error (for instance, a constraint violation partway through inserting several chunk rows): once a single SQL statement inside a transaction fails, PostgreSQL puts the entire transaction into an "aborted" state, and refuses to execute *any further statement* on that same

connection -- including the `UPDATE` that's supposed to record the failure -- until an explicit `ROLLBACK` happens. A naive `try/except` would catch the original exception successfully, then immediately raise a new, different exception (`InFailedSqlTransaction`) the moment it tried to run the status-update query, and the actual failure reason would never make it into `error_message` at all.

The fix is a `SAVEPOINT`, taken immediately before any of the risky work begins:

```
with conn.cursor() as cur:
    cur.execute("SAVEPOINT ingest_pipeline")

try:
    # extraction, chunking, embedding, chunk insertion, status update to READY
    ...
except (ExtractionError, EmbeddingError, psycopg2.Error) as exc:
    with conn.cursor() as cur:
        cur.execute("ROLLBACK TO SAVEPOINT ingest_pipeline")
    document = _set_status(conn, document_id, KnowledgeDocumentStatus.FAILED,
        error_message=str(exc))
```

`ROLLBACK TO SAVEPOINT` undoes only the work performed *since* the savepoint was taken (any partially-inserted chunk rows, any status change attempted) -- it does not touch or discard the original `PROCESSING` document row, which was committed to the transaction before the savepoint existed. Crucially, it also returns the connection to a normal, usable state, exactly as if the failed statements had never been attempted -- so the very next line, the `UPDATE` that records `status = FAILED`, executes successfully on the same connection, inside the same outer transaction that `app/database.py`'s `get_db()` will still commit normally once the request finishes (with no exception propagating out of the route handler at all -- from the router's point of view, "ingestion produced a FAILED document" and "ingestion produced a READY document" are both ordinary, successful return values).

This was not left unverified. `tests/test_rag_ingestion.py` includes a test that deliberately forces an embedding-provider failure via `monkeypatch`, confirms the resulting document lands as `FAILED` with the expected error message and zero chunks stored, and then -- on the *same* database connection, in the *same* test, immediately afterward -- runs a second, unrelated, successful ingestion and confirms it completes normally and reaches `READY`. That second half of the test exists specifically to prove the connection wasn't left in a broken state by the first failure -- exactly the scenario a pooled connection would hit in production if two unrelated HTTP requests happened to reuse the same pooled connection, one right after the other, one of which failed.

What this is tested against

`tests/test_rag_ingestion.py` (15 tests, run against a real Postgres connection) covers every claim in this chapter directly: successful ingestion of a real TXT, Markdown, PDF (a genuine PDF built with `reportlab`, not hand-typed bytes -- so `pypdf`'s actual parsing code runs), and DOCX (a genuine file built with `python-docx`) file, each producing a `READY` document with correctly stored chunks; an unsupported file type and an empty file both being rejected *before* any document row is created; a duplicate title without `replace_document_id` raising `ConflictError`; `replace_document_id` correctly producing a version-2 document with the original archived only afterward; a `replace_document_id` pointing at a nonexistent document raising `NotFoundError`; the embedding-failure-then-recovery savepoint test described above; and the full set of read/list/archive/hard-delete operations (`get_document`, `list_documents` with status/category filtering, `list_chunks`, `archive_document`, `delete_document` with

cascade verified directly against the `knowledge_chunks` table).

Admin API Reference

Overview

All six endpoints live under `/admin/knowledge`, defined in `backend/app/routers/knowledge.py`, and every one of them depends on `require_admin` (Chapter 12 explains exactly what that dependency does and does not guarantee). The router is thin, following this codebase's existing convention everywhere else: a route handler validates request shape (via FastAPI/Pydantic), calls exactly one service-layer function, and returns its result -- no business logic lives in the router itself.

phase7.md section 10 suggests `POST /admin/knowledge` and, separately, `POST /admin/knowledge/upload` as two endpoints ("if upload functionality is included"). This implementation combines them deliberately: `POST /admin/knowledge` is the upload endpoint -- a single multipart request carrying both the file and its metadata together. There is no scenario anywhere in phase7.md's own description of the system where a document's metadata is created independently of its file (there's no "reserve a title now, attach the file later" workflow described anywhere), so a separate `/upload` alias for what would be the exact same operation was judged to be a distinction without a difference, not an additional capability.

Endpoint summary

Method	Path	Auth	Purpose
POST	<code>/admin/knowledge</code>	Required	Upload a document (multipart: file + metadata). Runs the full ingestion pipeline synchronously.
GET	<code>/admin/knowledge</code>	Required	List documents, paginated, with optional <code>status/category/search</code> filters.
GET	<code>/admin/knowledge/{id}</code>	Required	Fetch one document by id.
GET	<code>/admin/knowledge/{id}/chunks</code>	Required	List a document's chunks, paginated.
DELETE	<code>/admin/knowledge/{id}</code>	Required	Archive (default) or, with <code>?hard=true</code> , permanently delete.
POST	<code>/admin/knowledge/search</code>	Required	Preview retrieval quality -- raw results plus the RAG context format.

`POST /admin/knowledge`

Multipart form fields: `file` (the document itself), `title` (required), `description` (optional), `category` (optional), `replace_document_id` (optional -- see Chapter 8's versioning explanation).

Real request, captured during this implementation:

```
curl -H "X-Admin-Id: e05aa282-d3a1-400e-86f6-452a79e63022" \
  -F "file=@vpn_guide.txt" -F "title=Test Onboarding Guide" -F "category=OTHER" \
  http://localhost:8000/admin/knowledge
```

Real response (HTTP 201):

```
{
  "id": "2429c665-cd96-454d-9fb5-5010803730db",
  "title": "Test Onboarding Guide",
  "description": null,
  "category": "OTHER",
  "file_name": "newdoc.txt",
  "file_type": "txt",
  "storage_location":
    "storage/knowledge_documents/2429c665-cd96-454d-9fb5-5010803730db_newdoc.txt",
  "version": 1,
  "status": "READY",
  "previous_version_id": null,
  "error_message": null,
  "uploaded_by": "e05aa282-d3a1-400e-86f6-452a79e63022",
  "created_at": "2026-09-14T02:31:24.532484Z",
  "updated_at": "2026-09-14T02:31:24.532484Z"
}
```

Note that a **201 Created** response does **not**, by itself, guarantee **status: READY** -- it guarantees a document row was created. Per Chapter 8's failure-handling design, **status** may instead be **FAILED**, with **error_message** populated, if extraction/chunking/embedding failed after the row was created. A caller (including a future admin dashboard UI) must check **status** in the response, not just the HTTP status code, to know whether ingestion actually succeeded.

Failure modes returned directly by this endpoint, captured for real:

- Unsupported file type -> HTTP 422: {"detail": "unsupported file type 'exe'; supported types are ['docx', 'md', 'pdf', 'txt']"}
- Empty file -> HTTP 422: {"detail": "uploaded file is empty"}
- Duplicate title, no **replace_document_id** -> HTTP 409 (body explains which existing document's id blocked it)

GET /admin/knowledge

Query parameters: **limit** (default 20, max 100), **offset** (default 0), **status**, **category**, **search** (case-insensitive title substring match). Returns the project's standard pagination envelope, **Page[KnowledgeDocumentRead]**:

```
{
  "items": [ { "...": "KnowledgeDocumentRead" }, "..." ],
  "total": 8,
  "limit": 3,
  "offset": 0
}
```

This is the exact same **Page[T]** shape (**items/total/limit/offset**) used by every other paginated list endpoint in this codebase (**GET /tickets**, **GET /users/{id}/tickets**, etc.) -- nothing new was invented for this phase's pagination.

GET /admin/knowledge/{id}

Returns a single `KnowledgeDocumentRead`, or HTTP 404 if the id doesn't exist -- handled uniformly by the same `NotFoundError` -> 404 exception-handler mapping every other resource lookup in this project uses (`app/errors.py`, registered once in `app/main.py`).

GET /admin/knowledge/{id}/chunks

Returns `Page[KnowledgeChunkRead]`. Verified directly, both by code inspection and by test: the response never includes an `embedding` field -- the underlying SQL query in `ingestion_service.list_chunks()` never selects that column in the first place, so there's no risk of it leaking even if the response schema were misconfigured. This is the concrete implementation of phase7.md section 18's "embeddings are not exposed publicly" requirement.

Real response for a document with one chunk:

```
{
  "items": [
    {
      "id": "4092d1f5-b5ac-44ee-abb6-f3681274e4f5",
      "document_id": "9b48dddd-e0fa-4662-a433-fb73f0cbd375",
      "chunk_index": 0,
      "content": "Test Onboarding Guide\n\nThis is a short test document...",
      "token_count": 49,
      "created_at": "2026-09-14T02:31:24.586537Z"
    }
  ],
  "total": 1,
  "limit": 2,
  "offset": 0
}
```

DELETE /admin/knowledge/{id}

Default behavior (no query parameter) **archives** the document -- sets `status = ARCHIVED`, leaving the row and all its chunks intact but excluded from future retrieval. This is deliberately the *default*, not `?archive=true` as an opt-in, because phase7.md section 9's "do not destroy historical knowledge blindly" principle should be the safe default a caller gets without having to know to ask for it.

`?hard=true` performs a genuine, irreversible delete: the `knowledge_documents` row is removed, and **ON DELETE CASCADE** (Chapter 4) automatically removes every one of its chunks in the same statement. Intended for correcting a genuine mistake (the wrong file uploaded entirely), not for normal document lifecycle management -- normal lifecycle management is archiving, or superseding via a versioned re-upload.

Both paths return HTTP 204 with an empty body on success, and both were verified for real: an archived document's subsequent `GET` shows `status: ARCHIVED`; a hard-deleted document's subsequent `GET` returns HTTP 404.

POST /admin/knowledge/search

Request body:

```
{ "query": "VPN still does not connect, auth failed", "top_k": 3, "category": null }
```

(**top_k** defaults to 5, constrained to the range 1-20; **category** is optional.)

Response -- both the raw, admin-facing results (with ids and category, useful for inspecting *why* something ranked the way it did) and the clean RAG context format a future AI agent will consume (Chapter 11 covers this shape in full):

```
{
  "query": "VPN still does not connect, auth failed",
  "results": [
    {
      "chunk_id": "a513a2f7-cc05-4634-8e81-1e436816cb56",
      "document_id": "6e24f7c5-6d46-4b95-a3f7-130c24a12756",
      "title": "VPN Troubleshooting Guide",
      "category": "VPN",
      "content": "### VPN connects but internal sites are still unreachable...",
      "score": 0.2217
    }
  ],
  "context": {
    "query": "VPN still does not connect, auth failed",
    "results": [
      { "title": "VPN Troubleshooting Guide", "content": "...", "score": 0.2217 }
    ]
  }
}
```

This endpoint exists as an **admin-facing preview and testing tool**, not as the mechanism a future AI agent will actually use. A future agent, running inside this same backend process, will call `retrieval_service.search()` directly in Python -- there's no reason to pay an HTTP round-trip to talk to itself. This endpoint's real purpose is letting a human (an admin, or a developer verifying retrieval quality by hand) issue a search the same way the eventual agent's retrieval step will, and see exactly what comes back.

What this is tested against

`tests/test_knowledge_api.py` (13 tests) exercises every endpoint above through the real FastAPI app via `TestClient` -- a genuine HTTP request/response cycle, not a mocked router -- including the admin-authorization boundary explicitly (missing header -> 401, unrecognized admin id -> 403), the full upload/list/get/chunks/archive/hard-delete/duplicate-409/versioning/search flow, and a direct assertion that a chunk response's keys never include `embedding`.

Retrieval and Similarity Search

The flow

phase7.md section 12 describes retrieval as a straight pipeline:

```
User query
  |
  v
Query embedding
  |
  v
Vector similarity search
  |
  v
Top K chunks
```

`app/rag/retrieval_service.py`'s `search()` function is exactly this pipeline, and nothing more -- everything this phase built toward converges here. Its signature:

```
def search(
    conn: PGConnection,
    query: str,
    top_k: int = 5,
    category: str | None = None,
) -> list[dict]:
```

Step 1: embed the query

`get_embedding_service().generate_embedding(query)` -- the exact same `EmbeddingService` interface, and, critically, the exact same *provider*, that produced every chunk's embedding during ingestion (Chapter 5). This is not incidental: a query embedded with one model and compared against chunks embedded with a different model would produce numbers that are not meaningfully comparable at all -- two different embedding models place semantically identical text at different, unrelated points in their own respective vector spaces, so a cosine similarity computed between vectors from two different models is essentially meaningless, not just less accurate. Swapping `EMBEDDING_PROVIDER` from `dev` to `bedrock` therefore requires re-ingesting the knowledge base (or at minimum being aware that old dev-embedded chunks and new Bedrock-embedded chunks are not comparable against each other) -- this is a real operational constraint of any embedding-based system, not specific to this implementation, and is called out explicitly here so it isn't discovered the hard way later.

Step 2: the query, annotated line by line

```
SELECT
    c.id AS chunk_id,
    c.document_id,
    c.chunk_index,
    c.content,
    d.title,
    d.category,
    1 - (c.embedding <=> %s::vector) AS score
FROM knowledge_chunks c
JOIN knowledge_documents d ON d.id = c.document_id
WHERE d.status = %s AND d.category = %s    -- category clause only present when a filter was
given
```

```
ORDER BY c.embedding <=> %s::vector
LIMIT %s
```

- `1 - (c.embedding <=> %s::vector) AS score` -- computes the similarity score returned to the caller. Chapter 6 explains exactly why `1 - distance` is the right conversion, and why the query parameter needs the explicit `::vector` cast (the `to_pgvector_literal()` helper from that same chapter).
- `JOIN knowledge_documents d ON d.id = c.document_id` -- every chunk needs its parent document's `title`, `category`, and, most importantly, `status`, since a chunk row itself carries no status of its own.
- `WHERE d.status = %s` -- this is the entire mechanism behind "only READY documents are retrievable." There is no separate filter step in application code after the query returns; a `PROCESSING`, `FAILED`, or `ARCHIVED` document's chunks are excluded at the SQL level, by the `WHERE` clause itself, before any row is even fetched into Python.
- `ORDER BY c.embedding <=> %s::vector` -- ascending cosine *distance*, which is descending relevance (Chapter 6). This is the clause the HNSW index (Chapter 6) actually accelerates: this is a real index-backed approximate nearest-neighbor query, not a full-table scan scored row by row in Python, which is what keeps query time roughly stable as the chunk count grows well beyond this project's current handful of documents.
- `LIMIT %s` -- the caller's requested `top_k`, applied last, after ordering.

Metadata filtering

phase7.md section 13 asks for retrieval to support filtering by metadata, while explicitly allowing the first implementation to "keep metadata simple" as long as it's designed so more filtering can be added later. This implementation's filter is `category`, added as one optional `AND d.category = %s` clause. The design intentionally keeps this pattern easy to extend: adding a second filterable field later -- a hypothetical `department` column, for instance -- is one more optional keyword argument to `search()` and one more conditionally-appended `AND` clause, following the exact same pattern this project's own `ticket_service.list_tickets()` already uses for its own optional filters (`status`, `priority`, `category`, `assigned_admin_id`, `search`), built from fixed SQL fragments with only the *values* coming from caller input -- the same SQL-injection-safe construction technique already established elsewhere in this codebase, reused here rather than invented fresh.

What comes back

A list of plain dicts (via `psycpg2.extras.RealDictCursor`, consistent with every other service function in this project -- there's no ORM, no model objects, anywhere in this codebase), each with `chunk_id`, `document_id`, `chunk_index`, `content`, `title`, `category`, `score`, already ordered by descending relevance. `POST /admin/knowledge/search` (Chapter 9) wraps this directly into its HTTP response; `app/rag/context.py`'s `build_rag_context()` (Chapter 11) reshapes it into the clean format a future AI agent will consume.

A real example

Query: "VPN still does not connect, auth failed", top_k=3, run against this project's actual seeded 8-document knowledge base. The real, captured top-2 results:

1. **VPN Troubleshooting Guide**, a chunk covering "VPN connects but internal sites are still unreachable" and "When to Escalate" -- score approximately 0.2217.
2. **VPN Troubleshooting Guide**, a chunk covering "Overview," "Installing Cisco AnyConnect," and the "VPN-ERR-403: Authentication Failed" section -- score approximately 0.2163.

Two observations worth making about these actual numbers, honestly rather than optimistically: first, the correct document won, clearly and by a real margin over anything from a different document -- retrieval worked. Second, the absolute score values (around 0.22) look low in isolation if you're expecting something closer to 1.0; this is a property of the **dev embedding provider** specifically (Chapter 5's hashing-trick bag-of-words approach naturally produces lower absolute cosine similarities than a genuinely semantic model would, since it only rewards literal shared vocabulary, and any two pieces of English text share only some fraction of their total possible vocabulary space even when they're topically identical) -- a score of 0.22 from **DevEmbeddingProvider** is not directly comparable to, and should not be read as "worse than," a score of 0.7 or 0.8 that a real semantic model like Bedrock's Titan would likely produce for the same query/chunk pair. What matters for retrieval *correctness* is the *relative ordering* of scores within one query's results, not their absolute magnitude -- and that ordering was correct here.

What this is tested against

`tests/test_rag_retrieval.py` (8 tests, run against a real pgvector query, not a mock) verifies: a VPN-themed query's top result is the VPN document; a password-themed query's top result is the password document; a WiFi-themed query's top result is the WiFi document (this is phase7.md section 20's literal example test, implemented as written); results come back in genuinely descending score order; an archived document is excluded even when its content is an exact keyword match for the query (proving the `WHERE d.status = 'READY'` clause, not luck, is what's doing the filtering); a category filter correctly restricts results to only that category; `top_k` correctly caps the result count; and a category filter matching zero documents returns an empty list cleanly rather than erroring.

Hybrid Search: Why Not Yet, and What It Would Take

What's Built, and What Isn't

The retrieval chapter of this manual covers what this phase actually implements: pure vector similarity search over embeddings, using pgvector's HNSW-accelerated cosine distance. phase7.md section 14 is explicit about sequencing: implement vector similarity search first, and only *explain* -- without necessarily implementing -- how keyword search could later be combined with it. This chapter is that explanation. Nothing described here exists in the codebase; every SQL fragment and function signature below is a sketch of a future change, clearly labeled as such.

Why Hybrid Search Matters at All

Pure semantic (vector) search is very good at understanding that two pieces of text are *about* the same topic even when they share almost no vocabulary -- "my internet keeps cutting out in the dorm" and "WiFi disconnects intermittently in residence halls" would score highly similar despite barely overlapping in words. That strength is also its weakness for a specific, important category of query: an exact, short, low-frequency term that matters enormously but carries very little of what an embedding model treats as "semantic content."

phase7.md gives three concrete examples worth examining individually:

- **VPN-ERR-403** -- a specific error code. An embedding model understands this string is "VPN-related," roughly the same way it understands the word "VPN" is VPN-related, but it has no special mechanism to recognize this as an *exact identifier* that should be prioritized over merely topically-similar text. A chunk that discusses VPN authentication failures generally, without ever mentioning this exact code, could easily out-score the one chunk that documents this precise error, purely because the general discussion happens to touch more topics an embedding associates with "VPN problem." A keyword or full-text search, by contrast, would find the one chunk containing that literal string immediately and unambiguously.
- **Cisco AnyConnect** -- a specific product name. Same reasoning: an embedding captures "this is about VPN client software" reasonably well, but doesn't specially privilege an exact product-name match over a paragraph that discusses VPN clients in general terms.
- **MFA** -- a common acronym that shows up across many different topics (VPN login, password reset, account setup all mention it in this project's own seeded documents). Its meaning is diffuse enough across the corpus that a semantic match on "MFA" alone is a weak signal; an exact keyword match narrows things down more sharply when a student's question happens to use that literal term.

None of this means vector search is bad -- the evaluation chapter later in this manual shows it performing at 100% Recall@1 across a real 10-question test set using nothing but vector search. It means vector search has a specific, well-understood blind spot around exact terms, and hybrid search is the standard mitigation for that blind spot.

The Standard Pattern, Conceptually

The common approach combines two independently-computed relevance scores into one ranking:

1. **A keyword/full-text relevance score.** The classic choice is BM25, a ranking function from traditional information retrieval that scores a document based on how often and how distinctively a query's terms appear in it. This project's stack doesn't need a dedicated BM25 engine to get most of the benefit, though -- Postgres has built-in full-text search (`tsvector` columns, `to_tsquery()`, and the `ts_rank()` function), which would add zero new infrastructure to a project that has already standardized on "everything lives in Postgres" as one of its core architectural decisions (see the vector storage chapter's discussion of why pgvector was chosen over a separate vector database for exactly this same reason).
2. **The existing vector similarity score**, unchanged from what `retrieval_service.search()` already computes.
3. **A combination step** that merges the two into a single ranked list. Two common techniques:
 - **Weighted linear combination:** `combined_score = (weight_a vector_score) + (weight_b keyword_score)`, after normalizing both scores onto comparable scales (cosine similarity is already conveniently bounded in roughly the 0-1 range, but a raw `ts_rank` score is not, and would need its own normalization before this kind of blend is meaningful).
 - **Reciprocal rank fusion (RRF):** instead of trying to make two differently-scaled scores comparable, RRF combines two *ranked lists* by each item's rank *position* -- an item ranked #1 by vector search and #3 by keyword search gets a combined score of $1/(k+1) + 1/(k+3)$ for some small constant `k` (commonly 60). RRF sidesteps the score-normalization problem entirely, which is a large part of why it's a popular choice when combining rankings from fundamentally different scoring systems.

What It Would Concretely Touch

If and when this is built, the changes would be contained and predictable:

- A `tsvector` generated column (or an expression index) on `knowledge_chunks.content`, so Postgres can full-text search chunk content without recomputing a `tsvector` on every query.
- A second SQL branch in `retrieval_service.search()` computing `ts_rank(content_tsv, to_tsquery(...))` alongside the existing vector distance calculation.
- A combination step -- either a weighted blend or RRF, as above -- merging the two ranked lists into the one list `search()` already returns.

None of this requires touching `ingestion_service.py`, the chunking or embedding modules, or any router -- the change is contained entirely within `retrieval_service.search()`'s query and the ranking logic immediately around it, exactly the kind of narrow, contained change the existing docs/05-tech-stack.md document already anticipated when it noted that `search_knowledge_base()`-shaped functions are deliberately the only call sites that would need to change if the storage/retrieval strategy behind them evolves.

Why This Wasn't Built Now

phase7.md section 14 states the governing rule directly: "Do not implement hybrid search unless it is simple within the current architecture." Building it now, before any real evidence suggested it was needed, would have been speculative complexity added against no demonstrated problem. This project's actual, real evaluation results -- covered in full in the evaluation chapter -- show 100% Recall@1, @3, and @5 across a real 10-question labeled dataset run against the real 8-document seeded knowledge base, using nothing but the vector search this phase actually built. There is no measured retrieval-quality gap for hybrid search to close yet.

That doesn't mean hybrid search will never be worth adding -- a larger, more heterogeneous real-world knowledge base with more exact error codes, product names, and acronyms scattered across many more documents is exactly the scenario where vector search's blind spot described above starts to matter in practice. The right way to decide if and when that point has been reached is the same evaluation harness ([app/rag/eval.py](#)) already built in this phase: grow the labeled question set to include queries built specifically around exact terms and codes, run the eval script, and see whether Recall@K and Precision@K actually suffer without hybrid search before deciding to build it. Evaluating the need, with real measurements, before building the mitigation is the same discipline phase7.md section 15 asks for applied to the whole retrieval system generally -- it applies just as well to deciding whether a specific enhancement to that system is worth its complexity.

The RAG Context Format

One Small, Reusable Interface

phase7.md section 17 draws a clear line for this phase: the future AI agent's eventual context will be built from three parts --

```
CURRENT MESSAGE
+
TICKET HISTORY
+
RETRIEVED KNOWLEDGE
|
AI AGENT
```

-- but this phase should only build the third piece, RETRIEVED KNOWLEDGE, and should do so as a small, reusable interface rather than the full context-assembly logic. That interface is exactly one function, `build_rag_context()`, in `backend/app/rag/context.py`:

```
def build_rag_context(query: str, results: list[dict]) -> dict:
    return {
        "query": query,
        "results": [
            {
                "title": result["title"],
                "content": result["content"],
                "score": round(float(result["score"]), 4),
            }
            for result in results
        ],
    }
```

Given the raw output of `retrieval_service.search()`, this produces exactly the JSON shape phase7.md section 16 specifies:

```
{
  "query": "VPN still does not work",
  "results": [
    { "title": "VPN Troubleshooting Guide", "content": "If the VPN client...", "score": 0.91 },
    { "title": "VPN Common Errors", "content": "Authentication failures...", "score": 0.84 }
  ]
}
```

What's Deliberately Left Out

`retrieval_service.search()`'s raw results carry more fields than this: `chunk_id`, `document_id`, and `chunk_index` are all present in the raw result dicts (and are exposed as-is in the `results` field of `POST /admin/knowledge/search`'s HTTP response, described in the admin API reference chapter, because an admin debugging retrieval quality benefits from seeing exactly which chunk of which document matched). `build_rag_context()` strips all three of them out.

This isn't an oversight -- it's the whole point of having this function exist separately from the raw search results. An LLM consuming this context to answer a student's question has no legitimate use for a database UUID or a chunk index number. Handing those values to a language model as part of its context risks something subtler than simple clutter: a model given numbers or identifiers with no

explained meaning will sometimes invent significance for them anyway -- treating a `chunk_index` of 3 as if it carried some information about recency, importance, or ordering, when it is in fact just an artifact of how one particular document happened to get split during ingestion. The discipline here is to hand a context-consuming model only what it actually needs to reason about the question in front of it: the source document's title (so it can attribute an answer, e.g. "according to the VPN Troubleshooting Guide..."), the actual relevant text, and a similarity score it could in principle use to judge how confident to be in a given piece of retrieved context. Nothing else.

Why This Is a Separate Function at All

It would have been possible to build this JSON shape inline, directly inside the `POST /admin/knowledge/search` route handler, and skip creating a separate module for it. That wasn't done, for a reason that matters specifically because of what `phase7.md` explicitly defers to a later phase: the future AI agent is very likely to call `retrieval_service.search()` directly, in-process, rather than through any HTTP endpoint -- it will run inside this same backend, and an HTTP round-trip to itself would be pure overhead with no benefit. If the RAG-context JSON shape were only ever constructed inline inside the search router, that in-process agent code would either have to duplicate the same dict-building logic itself, or reach into the router's internals in an awkward way. By pulling `build_rag_context()` out as its own small, tested, standalone function, both call paths -- the HTTP endpoint (today, for admin testing) and a future in-process agent call (later, for actual use) -- produce the exact same shape from the exact same code. There is no way for the two to quietly drift apart over time, which is exactly the kind of small inconsistency that becomes a confusing bug much later, once an agent depending on this shape actually exists and something upstream changes without this function being updated to match.

What This Phase Explicitly Does Not Build

Building the full three-part context -- combining this RETRIEVED KNOWLEDGE output with a ticket's CURRENT MESSAGE and TICKET HISTORY into one assembled prompt for an AI agent -- is explicitly out of scope for this phase. `phase7.md` section 17 states this directly, and the architecture-boundaries chapter of this manual goes into why that separation of responsibilities is treated as a hard rule throughout this whole implementation, not just a scheduling convenience: PostgreSQL owns application state and ticket history, the knowledge base owns official IT documentation, RAG's only job is finding relevant knowledge within that documentation, and reasoning, deciding, and responding are explicitly reserved for the AI agent phase that has not been built yet. `build_rag_context()` produces exactly the third of those responsibilities and stops there.

Security

phase7.md Section 18's Checklist, Addressed Item by Item

phase7.md section 18 lists six specific security requirements for the knowledge base. Here is exactly what was built against each one.

"Only authenticated admins can upload/manage documents." Every endpoint under `/admin/knowledge` -- all six of them, with no exceptions -- depends on `require_admin` (`app/auth.py`). The rest of this chapter goes into exactly what that guarantees, and, just as importantly, what it doesn't.

"S3 objects are private if S3 is used." `S3FileStorage` (`app/rag/storage.py`, covered in the ingestion pipeline chapter) never passes an ACL on its `put_object` call, relying entirely on the target bucket's own default-private configuration rather than per-object ACLs. This matches AWS's current guidance: S3 has been moving away from ACL-based access control in favor of Block Public Access settings and bucket policies, and a class that never touches ACLs at all cannot accidentally override a bucket's private default with a permissive one. The AWS deployment chapter later in this manual covers the actual bucket policy configuration expected to sit alongside this.

"Database access is restricted." This is fundamentally an infrastructure and deployment concern -- IAM policies, security groups, VPC placement, and network ACLs around wherever Postgres actually runs (a self-hosted instance or Amazon RDS) -- rather than something application code can enforce. The AWS deployment chapter gives concrete guidance for this.

"Embeddings are not exposed publicly." Enforced in two independent layers, described fully in the API reference chapter's discussion of `GET /admin/knowledge/{id}/chunks`: the `KnowledgeChunkRead` response schema has no field for the embedding at all, and the SQL query behind that endpoint (`ingestion_service.list_chunks()`) never even selects the `embedding` column from the database in the first place. This is deliberately defense-in-depth -- even a future bug that accidentally serialized the wrong dict somewhere couldn't leak the vector through this endpoint, because the raw data fetched from Postgres never contains it to begin with.

"Documents cannot be accessed without authorization." Every single endpoint under `/admin/knowledge`, including the search/preview endpoint, requires `require_admin`. There is no unauthenticated read path into the knowledge base anywhere in this implementation.

"Secrets are not stored in source code." AWS credentials (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`), the Bedrock embedding model id, and the S3 bucket name are all read from environment variables through `app/config.py`'s `Settings` class (`pydantic-settings`, reading from a `.env` file that is listed in `.gitignore` and therefore never committed). Nothing in `app/rag/` hardcodes a credential, bucket name, or API key anywhere.

`require_admin`: What It Actually Does, and What It Doesn't

This is the part of the chapter that needs to be completely candid, because getting this wrong -- overstating what this mechanism provides -- would be worse than not documenting it at all.

phase7.md section 10 asks the knowledge-base admin endpoints to "use the existing admin authentication/RBAC system." There is no such system to use. As of this phase, this entire backend has no login of any kind, anywhere. The `admins` table (added in an earlier phase, long before RAG existed) is a read-only directory with no password column, no session mechanism, and no login endpoint at all -- admin rows are only ever inserted directly, either by the seed script or, in a real deployment, by whoever operates the database. Every other router in this codebase performs no authorization check whatsoever; this is the first phase of the project where an endpoint genuinely needs to refuse a caller.

Building real authentication was correctly judged to be out of scope for this phase -- it is not listed as a Phase 7 deliverable in phase7.md, and it wasn't built in any earlier phase either. But phase7.md section 20 explicitly requires a test verifying "non-admin users must not manage the knowledge base," which is not a meaningful test to write against an API with no concept of "non-admin" at all. Some gate had to exist.

`require_admin`, defined in `app/auth.py`, is that gate:

```
def require_admin(
    x_admin_id: uuid.UUID | None = Header(default=None, alias="X-Admin-Id"),
    db: PGConnection = Depends(get_db),
) -> dict:
    if x_admin_id is None:
        raise HTTPException(status_code=401, detail="X-Admin-Id header is required for this endpoint")
    try:
        return admin_service.get_admin(db, x_admin_id)
    except NotFoundError as exc:
        raise HTTPException(status_code=403, detail="X-Admin-Id does not match a known admin")
        from exc
```

It is a FastAPI dependency that reads a single request header, `X-Admin-Id`, expected to contain an admin's UUID. If the header is missing entirely, it raises HTTP 401. If the header is present but doesn't resolve to a real row in the `admins` table (via the pre-existing `admin_service.get_admin()` function, unchanged from earlier phases), it raises HTTP 403. If it does resolve, the resolved admin's row is returned and made available to the route handler.

State this plainly, without hedging: **this is not real authentication**. There is no password check. There is no session token. There is no cryptographic signature of any kind proving the request actually came from the admin whose id is in the header. Anyone who knows -- or successfully guesses -- a real admin's UUID passes this check completely and can manage the entire knowledge base as that admin. UUIDs are not secrets and were never designed to function as one; they are unguessable in practice only in the sense that guessing one at random is astronomically unlikely, not in the sense that they resist an attacker who has any other way of learning or observing one (for instance, from a response body returned by `GET /admins`, which is a pre-existing, unauthenticated, read-only endpoint listing every admin's id, name, and email).

This is deliberately the same kind of labeled stand-in as `EMAIL_WEBHOOK_DEV_MODE`, an existing mechanism elsewhere in this codebase (`app/email/service.py`, `app/routers/email.py`): in local development, the inbound email webhook accepts requests with no real verification that they actually originated from AWS SNS, because building real SNS message signature verification wasn't the point of that phase either. Neither mechanism claims to be secure. Both exist to make the rest of the system -- and its tests -- meaningful in the absence of the real thing, with the gap clearly documented rather than

silently glossed over.

Why This Design Is Still the Right Call

The alternative to building `require_admin` at all would have been to leave the knowledge-base endpoints completely unauthenticated, like every other endpoint in this codebase currently is. That would have technically satisfied "don't build real authentication, it's out of scope" just as well, but it would have made phase7.md section 20's authorization test impossible to write meaningfully, and it would have left the knowledge-base admin endpoints -- document upload, deletion, and permanent hard-delete -- reachable by literally anyone who could reach the API at all, with zero barrier. `require_admin` is a small, honest middle ground: it is not a security boundary a real deployment should trust, but it is a real, working, testable authorization gate that correctly distinguishes "a request carrying a real admin's identity" from "a request carrying no identity, or a fabricated one" for every purpose this phase actually needs it for.

Forward Compatibility

Every router using this dependency depends on it the same way: `admin: dict = Depends(require_admin)`. When real session- or JWT-based authentication is eventually built in a later phase, only `require_admin`'s own internal implementation needs to change -- swap the header-lookup logic for a real session/token validation call that still returns an admin dict on success and still raises `HTTPException` on failure. Not a single line in `app/routers/knowledge.py` needs to change, because every route already depends on the abstraction (`require_admin`, a function that produces an admin dict or raises) rather than on the specific mechanism (a trusted header) behind it today.

Verified

This was confirmed working correctly against the real running API, not just designed on paper: a request to any `/admin/knowledge` endpoint with no `X-Admin-Id` header returns HTTP 401. A request with a syntactically valid but non-existent admin UUID returns HTTP 403. Both are covered by automated tests as well -- `tests/test_knowledge_api.py::test_missing_admin_header_is_rejected`, `test_unknown_admin_id_is_rejected`, and `test_search_without_admin_header_is_rejected` -- so a future change that accidentally weakens or removes this gate on any endpoint would be caught by the test suite immediately, not discovered later in a security review.

Error Handling Reference

Purpose of This Chapter

The ingestion pipeline chapter already walked through error handling in the order failures actually occur inside `ingest_document()`. This chapter organizes the same overall territory differently -- as a lookup reference, grouped by exception type and by what actually happens at the HTTP boundary -- so a reader troubleshooting a specific status code or exception can find the relevant explanation directly, without re-reading the whole pipeline narrative.

The Exception Types

Exception	Introduced	Crosses HTTP boundary?	Mapped to
<code>NotFoundError</code>	Pre-existing (Phase 1/2)	Yes	404, via a handler already registered in <code>app/main.py</code>
<code>ConflictError</code>	Pre-existing (Phase 2)	Yes	409, via a handler already registered in <code>app/main.py</code>
<code>ValidationError</code>	New in Phase 7 (<code>app/errors.py</code>)	Yes	422, via a new handler added to <code>app/main.py</code>
<code>ExtractionError</code>	New in Phase 7 (<code>app/rag/extraction.py</code>)	No	Never -- always caught inside <code>ingest_document()</code>
<code>EmbeddingError</code>	New in Phase 7 (<code>app/rag/embeddings/provider.py</code>)	No	Never -- always caught inside <code>ingest_document()</code>

The first three are ordinary FastAPI-style domain exceptions: a service function raises one, and a single exception handler registered once in `app/main.py` translates it into the matching HTTP response, so no router has to build its own `HTTPException` by hand. `ValidationError` is the one genuinely new addition to this shared mechanism in this phase -- added specifically because none of the existing two (`NotFoundError`, `ConflictError`) correctly describes "the request itself can't be processed, independent of whether it conflicts with anything" (an unsupported file type isn't a 404, because nothing was being looked up; it isn't a 409, because there's no existing resource it conflicts with). 422 (Unprocessable Entity) is the correct HTTP semantics for exactly this case: the request was syntactically well-formed but semantically invalid.

`ExtractionError` and `EmbeddingError` are different in kind from the other three: they are never allowed to reach the HTTP layer at all. Both are always caught inside `ingest_document()`'s try block and converted into a `FAILED` document status with `error_message` set to the exception's message -- see the next section for why.

Complete Reference Table

Scenario	Exception raised	HTTP status	Where handled	Document row created?
Unsupported file type	<code>ValidationError</code>	422	Checked in <code>ingest_document()</code> before any row is created	No
Empty file	<code>ValidationError</code>	422	Same	No
Text extraction failure (corrupt/malformed file)	<code>ExtractionError</code> (internal)	201, <code>status: "FAILED"</code>	Caught inside the savepoint-protected block in <code>ingest_document()</code>	Yes
Embedding API failure	<code>EmbeddingError</code> (internal)	201, <code>status: "FAILED"</code>	Same	Yes
Database failure mid-insert	<code>psycopg2.Error</code> (internal)	201, <code>status: "FAILED"</code>	Same, via <code>ROLLBACK TO SAVEPOINT</code> (see the ingestion pipeline chapter)	Yes
Duplicate document title, no <code>replace_document_id</code>	<code>ConflictError</code>	409	Checked before any row is created	No
<code>replace_document_id</code> references a nonexistent document	<code>NotFoundError</code>	404	Checked before any row is created	No
Document or chunk lookup by id fails	<code>NotFoundError</code>	404	Route handler / service lookup	N/A
Missing <code>X-Admin-Id</code> header	<code>HTTPException</code> raised directly	401	<code>app/auth.py</code> 's <code>require_admin</code>	N/A
<code>X-Admin-Id</code> doesn't match a real admin	<code>HTTPException</code> raised directly	403	Same	N/A

Why Pipeline Failures Return 201, Not 4xx or 5xx

The three "internal" rows in the table above -- extraction failure, embedding failure, database failure -- are the ones most likely to look surprising at first glance: shouldn't a failure return an error status code? The answer requires being precise about exactly *what* succeeded and what didn't.

By the time any of these three failures can occur, the `knowledge_documents` row itself has already been successfully created (status=PROCESSING) and its file has already been safely stored. The HTTP request's job -- accept an upload, create a trackable resource representing it -- genuinely succeeded. What failed afterward is that resource's internal *processing*: a fact about the resulting resource's state, which the response body communicates via `status: "FAILED"` and `error_message`, not a fact about whether the request itself was handled correctly.

Consider the alternative a less careful implementation might choose: catch the failure and return **500 Internal Server Error**, with no persisted record that an attempt was ever made. This is strictly worse for the person operating the system. A **500** with no corresponding row tells an admin nothing except "something broke, somewhere, at some point" -- they have no way to know which document they were trying to upload, whether it partially succeeded, or what to do next, short of digging through server logs they may not have access to or may not think to check. The actual design instead leaves a durable, inspectable record: **GET /admin/knowledge/{id}** (or **GET /admin/knowledge?status=FAILED**) shows exactly which document failed, precisely what went wrong (**error_message** contains the underlying exception's message, not a generic "an error occurred"), and when. The admin's next action is obvious and immediate: fix whatever the error message describes, and re-upload.

This design decision connects directly to the SAVEPOINT mechanism the ingestion pipeline chapter covers in depth -- that mechanism is precisely what makes it *possible* to reliably reach this well-defined FAILED end state instead of leaving the connection broken partway through, no matter which of the three failure types actually occurred.

The Life of a FAILED Document

A document that lands in **status: "FAILED"** doesn't disappear or get cleaned up automatically. It remains visible via **GET /admin/knowledge** (filterable with **?status=FAILED** to find every currently-failed document at once) indefinitely, until an admin takes one of two actions:

1. **Re-upload successfully.** A fresh **POST /admin/knowledge** call with a corrected file and the same title creates a brand-new document (a new id, **version: 1**, since the FAILED attempt never reached READY and therefore was never eligible to be treated as an existing "current" version to replace via **replace_document_id** -- the duplicate-title check in **ingest_document()** only looks at documents with **status = 'READY'**, so a FAILED document never blocks a fresh retry under the same title).
2. **Hard-delete it.** **DELETE /admin/knowledge/{id}?hard=true** permanently removes the FAILED row (it has no chunks to cascade-delete, since none were ever inserted).

Nothing about a FAILED document is special-cased to expire, and nothing in this codebase automatically retries a failed ingestion -- both of those are reasonable directions for a future phase to consider (a scheduled cleanup of old FAILED rows past some age, or an automatic retry with backoff for a transient embedding-provider outage), but neither is built here, since neither was asked for by phase7.md and adding either now would be speculative complexity for problems this phase has no evidence exist yet.

Testing Guide

Prerequisites

Python 3.12, Docker (for a local Postgres+pgvector container), and a terminal. Nothing else. Every step below runs with **zero external services and zero AWS credentials** -- the default settings (`EMBEDDING_PROVIDER=dev`, `KNOWLEDGE_STORAGE_BACKEND=local`) mean the entire pipeline, every automated test, and every manual curl call in this chapter work completely offline.

1. Local environment setup

From a clean checkout, every command below runs from the `backend/` directory unless stated otherwise.

```
cd backend
python3 -m venv .env && source .env/bin/activate
pip install -r requirements-dev.txt
```

`requirements-dev.txt` pulls in `requirements.txt` (FastAPI, psycpg2, pydantic, boto3, and this phase's additions: `python-multipart` for file uploads, `pgvector` for the Postgres adapter, `pypdf` and `python-docx` for extraction) plus `pytest`, `httpx`, and `reportlab`. `reportlab` is a **test-only** dependency -- it exists solely to generate a real, valid PDF file inside `tests/test_rag_ingestion.py`, so that test exercises `pypdf`'s actual parsing code against real PDF bytes rather than a hand-typed fake.

```
cp .env.example .env
```

The RAG-specific settings in `.env.example` already have sensible local-development defaults and don't need editing to run anything in this chapter: `EMBEDDING_PROVIDER=dev`, `KNOWLEDGE_STORAGE_BACKEND=local`, `RAG_CHUNK_SIZE_WORDS=600`, `RAG_CHUNK_OVERLAP_WORDS=80`.

```
cd ..                                # back to repo root
docker compose up -d                 # starts Postgres, using the pinned pgvector/pgvector:pg16 image
cd backend
python -m migrations.run_migrations
```

Expected output includes `Applying 0006_knowledge_chunks_and_vector.sql ... / Applied 0006_knowledge_chunks_and_vector.sql` alongside the five pre-existing migrations from earlier phases -- this is the migration that enables the `vector` extension and creates `knowledge_chunks` (Chapter 4).

```
python -m seed.seed_data
```

Creates a demo user, two admins, and three demo tickets -- unrelated to RAG directly, but the knowledge-base seed step below needs an existing admin row to use as `uploaded_by`.

```
python -m seed.seed_knowledge_base
```

Ingests all eight example documents in `seed/knowledge_docs/*.md` through the real ingestion pipeline -- real chunking, real (dev-provider) embedding, real database inserts, exactly the same code path `POST /admin/knowledge` uses. Expected output:

```
ingested 'VPN Troubleshooting Guide' -> status=READY id=<uuid>
```

```
ingested 'WiFi Troubleshooting Guide' -> status=READY id=<uuid>
ingested 'Password Reset Procedure' -> status=READY id=<uuid>
ingested 'Student Account Setup' -> status=READY id=<uuid>
ingested 'Microsoft Office Installation' -> status=READY id=<uuid>
ingested 'Campus Network Guide' -> status=READY id=<uuid>
ingested 'Printer Troubleshooting' -> status=READY id=<uuid>
ingested 'Known IT Issues' -> status=READY id=<uuid>
```

All eight reaching **status=READY** is the expected, verified outcome -- this was run for real during this implementation and produced exactly this output.

```
uvicorn app.main:app --reload
```

The API is now serving at **http://localhost:8000**.

2. Running the automated test suite

```
pytest -v
```

This runs the complete suite: 63 pre-existing tests from Phases 1-6 (entirely unaffected by this phase's changes -- none of them touch RAG code) plus 57 new Phase 7 tests across six files, 120 tests total, all passing against the real Postgres+pgvector container. What each new file actually proves:

tests/test_rag_chunking.py (6 tests, pure unit tests, no database needed). Verifies: empty text produces zero chunks; a short text produces exactly one chunk with correctly computed metadata; a long, multi-paragraph text splits into multiple chunks each respecting the configured size budget; chunk boundaries genuinely carry forward overlap text from the previous chunk (the test checks the tail words of one chunk actually reappear at the head of the next, not just that overlap is configured); a bullet list short enough to fit in one chunk isn't needlessly split across two; and an artificially oversized single block with no paragraph breaks splits at sentence boundaries, with every resulting chunk verified to end on a period, never mid-sentence.

tests/test_rag_embeddings.py (6 tests). Verifies the dev embedding provider is deterministic (embedding identical text twice produces an identical vector -- proving no hidden randomness, such as Python's per-process-randomized **hash()**, leaks into the result); produces vectors of exactly the requested dimension; produces genuinely unit-length (L2-normalized) vectors; that **embed_batch()** produces results identical to calling **embed_text()** once per item; that even empty-string input still produces a valid unit vector rather than crashing; and -- the most important assertion in this file -- that semantically similar text scores a measurably higher cosine similarity than dissimilar text, which is what proves the hashing-trick embedding (Chapter 5) is a genuine, testable stand-in rather than noise wearing an embedding's shape.

tests/test_rag_extraction.py (9 tests). Covers TXT and Markdown extraction directly; case- and dot-insensitive file-type matching ("**TXT**", "**.txt**", and "**txt**" all work); an unsupported file type and an empty file both raising **ValidationError**; non-UTF-8 bytes being replaced with a placeholder character rather than crashing extraction; and the text-cleaning function collapsing runs of excess blank lines, normalizing Windows line endings, and preserving Markdown headings/bullets so that chunking's boundary detection still works correctly on cleaned text.

tests/test_rag_ingestion.py (15 tests, against a real Postgres connection -- the most important integration test file). Ingesting a real TXT, Markdown, PDF, and DOCX file each produces a **READY**

document with correctly stored chunks. The PDF and DOCX tests use genuinely generated files -- a real PDF built with `reportlab's canvas` and a real DOCX built with `python-docx's Document`, not hand-typed byte strings -- so `pypdf's` and `python-docx's` actual parsing code paths are exercised, not bypassed. An unsupported file type and an empty file are both verified to leave zero new document rows behind (rejected before any row is created). A duplicate title without `replace_document_id` raises `ConflictError`. `replace_document_id` correctly produces a version-2 document and archives the original -- verified end to end. A `replace_document_id` pointing at a nonexistent document raises `NotFoundError`. A monkeypatched embedding-provider failure leaves a document `FAILED` with the exact error message captured and zero chunks inserted. And -- the single most scrutinized test in the whole suite -- a dedicated test proves the *same* database connection stays fully usable for a second, successful, unrelated ingestion immediately after the first one failed, which is the direct proof that the `SAVEPOINT/ROLLBACK TO SAVEPOINT` mechanism (Chapter 8) genuinely works and doesn't leave a pooled connection poisoned for whatever request runs on it next.

`tests/test_rag_retrieval.py` (8 tests, against a real pgvector query, not a mock). Ingests real documents through the real pipeline, then queries through `retrieval_service.search()`: a VPN-themed query's top result is the VPN document; a password-themed query's top result is the password document; a WiFi-themed query's top result is the WiFi document (this is phase7.md section 20's literal example test, implemented exactly as written). Results come back genuinely ordered by descending score. An archived document is excluded from search even when its content is an exact keyword match for the query (proving the `WHERE d.status = 'READY'` clause, not luck, is doing the filtering). A category filter correctly restricts results. `top_k` correctly caps the result count. A category filter matching zero documents returns an empty list cleanly rather than erroring.

`tests/test_knowledge_api.py` (13 tests, full HTTP request/response cycle via FastAPI's `TestClient`, not a mocked router). Covers the admin-authorization boundary explicitly: no `X-Admin-Id` header -> 401; an `X-Admin-Id` that doesn't resolve to a real admin -> 403. Plus the full upload/get/list-chunks/archive-delete/hard-delete/duplicate-409/versioning/search flow, each through a real HTTP call, and a direct assertion that a chunk response's JSON keys never include `embedding`.

A project-wide note worth restating here: this test suite talks to a **real** Postgres database throughout -- there is no SQLite fallback and no mocking of the database layer anywhere in this project, a deliberate choice made back in Phase 1 and carried forward unchanged into this phase. The whole point is that these tests verify the actual schema, actual constraints, and (for this phase specifically) actual pgvector query behavior work correctly -- not merely that Python code runs without throwing an exception.

3. Manual API testing walkthrough

A complete, sequential walkthrough against a locally running `uvicorn` instance (from step 1), with the exact curl commands and the actual responses captured running this exact sequence during this implementation.

(a) Get an admin id to use.

```
docker exec <postgres-container-name> psql -U helpdesk -d helpdesk -t \  
-c "SELECT id FROM admins LIMIT 1;"
```

Or `curl http://localhost:8000/admins` and take any `id` from the response. Save it as `$ADMIN_ID` for the rest of this walkthrough.

(b) Confirm the auth gate works.

```
curl -i http://localhost:8000/admin/knowledge
```

No header -> 401. Then:

```
curl -i -H "X-Admin-Id: 00000000-0000-0000-0000-000000000000"
http://localhost:8000/admin/knowledge
```

A syntactically valid but nonexistent admin id -> 403.

(c) List the seeded knowledge base.

```
curl -H "X-Admin-Id: $ADMIN_ID" http://localhost:8000/admin/knowledge
```

Returns the eight seeded documents, `"total": 8`, every one at `"status": "READY"`.

(d) Run a search.

```
curl -X POST -H "X-Admin-Id: $ADMIN_ID" -H "Content-Type: application/json" \
-d '{"query": "VPN still does not connect, auth failed", "top_k": 3}' \
http://localhost:8000/admin/knowledge/search
```

The real captured top result: a `VPN Troubleshooting Guide` chunk covering "VPN connects but internal sites are still unreachable" / "When to Escalate," score approximately `0.2217`; second result, the same document's "Overview" / "Installing Cisco AnyConnect" / "VPN-ERR-403" chunk, score approximately `0.2163`. (See Chapter 10 for why these absolute score values look low and why that's expected for the dev embedding provider specifically, and why it doesn't indicate a problem.)

(e) Upload a brand-new test document.

```
curl -H "X-Admin-Id: $ADMIN_ID" \
-F "file=@newdoc.txt" -F "title=Test Onboarding Guide" -F "category=OTHER" \
http://localhost:8000/admin/knowledge
```

201, `"status": "READY"`. Note the returned `"id"` -- used below.

(f) Re-upload the same title without `replace_document_id`.

```
curl -i -H "X-Admin-Id: $ADMIN_ID" \
-F "file=@newdoc.txt" -F "title=Test Onboarding Guide" \
http://localhost:8000/admin/knowledge
```

409 -- rejected as a duplicate, exactly as designed (Chapter 8/13).

(g) Re-upload as an explicit new version.

```
curl -H "X-Admin-Id: $ADMIN_ID" \
-F "file=@newdoc.txt" -F "title=Test Onboarding Guide" -F "replace_document_id=<id-from-e>" \
http://localhost:8000/admin/knowledge
```

201, `"version": 2`, `"previous_version_id"` set to the id from step (e). A follow-up `GET /admin/knowledge/<id-from-e>` confirms that original document's status is now `ARCHIVED`.

(h) Upload an unsupported file type.

```
curl -i -H "X-Admin-Id: $ADMIN_ID" -F "file=@malware.exe" -F "title=Bad File" \
http://localhost:8000/admin/knowledge
```

422, real captured body: {"detail": "unsupported file type 'exe'; supported types are ['docx', 'md', 'pdf', 'txt']"}.

(i) Upload a genuinely empty file.

```
touch empty.txt
curl -i -H "X-Admin-Id: $ADMIN_ID" -F "file=@empty.txt" -F "title=Empty Doc" \
http://localhost:8000/admin/knowledge
```

422, {"detail": "uploaded file is empty"}.

(j) Upload a deliberately corrupted PDF.

```
echo "%PDF-1.4 not a real pdf" > corrupt.pdf
curl -H "X-Admin-Id: $ADMIN_ID" -F "file=@corrupt.pdf" -F "title=Corrupt PDF Doc" \
http://localhost:8000/admin/knowledge
```

201, but "status": "FAILED", "error_message": "failed to extract text from pdf file: Stream has ended unexpectedly". This is the intended, designed failure-handling behavior (Chapter 8's SAVEPOINT mechanism, Chapter 13's error taxonomy) -- not a bug, and not a crash.

(k) Fetch a document's chunks and confirm no embedding leaks.

```
curl -H "X-Admin-Id: $ADMIN_ID" http://localhost:8000/admin/knowledge/<any-ready-doc-id>/chunks
```

Confirm the response JSON has no **embedding** key anywhere.

(l) Archive a document (default delete behavior).

```
curl -i -X DELETE -H "X-Admin-Id: $ADMIN_ID" http://localhost:8000/admin/knowledge/<doc-id>
```

204. A follow-up **GET** on the same id shows "status": "ARCHIVED" -- the row and its chunks are untouched, just excluded from future search.

(m) Hard-delete a document.

```
curl -i -X DELETE -H "X-Admin-Id: $ADMIN_ID"
"http://localhost:8000/admin/knowledge/<doc-id>?hard=true"
```

204. A follow-up **GET** on the same id now returns 404 -- the row and its chunks (via **ON DELETE CASCADE**) are genuinely gone.

4. Running the retrieval evaluation

After the knowledge-base seed step (step 1):

```
python -m scripts.evaluate_retrieval
```

Prints Recall@1/@3/@5 and Precision@1/@3/@5 against the 10-question labeled dataset in [app/rag/eval.py](#), plus a per-question breakdown. Chapter 16 covers what these numbers mean and interprets the actual results from a real run in full.

5. Resetting the local environment

To reset the local Postgres schema for a clean manual re-run (not needed for `pytest`, which manages its own schema lifecycle automatically -- see below):

```
docker exec <postgres-container-name> psql -U helpdesk -d helpdesk \  
-c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
```

Or, to discard the container's data volume entirely: `docker compose down -v && docker compose up -d`. Either way, follow with `python -m migrations.run_migrations`, `python -m seed.seed_data`, and `python -m seed.seed_knowledge_base` again to get back to a fully seeded state.

This manual reset is never needed to run `pytest` itself: `tests/conftest.py`'s session-scoped fixture already applies every migration fresh at the start of a test session and drops the entire `public` schema at the end, automatically, every time -- the automated suite is fully self-contained and never depends on, or interferes with, whatever state the manual curl walkthrough above left behind.

Retrieval Evaluation

"Do not judge RAG only by whether the application runs"

That is phase7.md section 15's own framing, quoted directly, and it deserves to be taken literally rather than treated as a truism. A retrieval pipeline can run without a single error -- no exceptions, no crashes, a plausible-looking JSON response returned for every query, every time -- while consistently returning the *wrong* document for real questions, and nothing in the application's error handling (Chapter 13) would ever surface that as a problem. A wrong retrieval result is not an exception; it's a successful HTTP 200 carrying the wrong content. The only way to catch this is to measure retrieval against questions with a known-correct answer -- which is exactly what this chapter's evaluation harness ([backend/app/rag/eval.py](#)) does.

Recall@K and Precision@K, defined precisely for this dataset

This project's evaluation dataset (by design, matching phase7.md's own examples) has exactly **one** labeled correct/expected document per test question. That single fact simplifies the general information-retrieval definitions of these two metrics in a specific, important way, worth working through carefully rather than glossing over.

Recall@K, in general, is *(number of relevant items retrieved in the top K) / (total number of relevant items that exist)*. With exactly one relevant document per question, that fraction can only ever be 0 or 1 -- there is no "half credit" possible. Recall@K here reduces exactly to: *did any chunk belonging to the expected document appear anywhere in the top K retrieved chunks*, averaged as a hit-rate across every question in the dataset. This is, precisely, phase7.md section 15's own literal request: "Measure whether the correct document/chunk appears in Top 1 / Top 3 / Top 5."

Precision@K, by contrast, is computed at the chunk level as *(number of the K returned chunks that belong to the expected document) / K*. This behaves very differently from Recall@K, and tells a genuinely different story. Work through one concrete example: a query whose single correct chunk lands at rank 1 of a top-5 result, with the other four results all belonging to unrelated documents. Recall@5 for that result is a clean **1.0** -- the correct document was found within the top 5. Precision@5 for that exact same result is only **0.2** (one relevant chunk out of five returned) -- because four of the five slots were "wasted" on irrelevant results, even though the one that mattered was found immediately, at rank 1.

That example is the whole reason both numbers matter together, not just one or the other: Recall@K alone cannot distinguish "found immediately, as the clearly dominant result" from "found immediately, but buried among mostly irrelevant noise at every other rank" -- both score identically on Recall@K if the correct answer happens to be present. Precision@K is what tells those two very different outcomes apart.

The labeled dataset

Ten questions, each paired with exactly one expected document ([app/rag/eval.py](#)'s `EVAL_DATASET`), deliberately covering all eight seeded documents -- with two questions each probing the VPN and

Password documents specifically, since those are the ones phase7.md's own examples emphasize, and one question each for the remaining six:

Question	Expected document
"My VPN says authentication failed."	VPN Troubleshooting Guide
"VPN client won't connect at all, error VPN-ERR-403."	VPN Troubleshooting Guide
"I forgot my student password."	Password Reset Procedure
"I need to reset MFA on my account, I got a new phone."	Password Reset Procedure
"Campus WiFi keeps disconnecting."	WiFi Troubleshooting Guide
"How do I set up my new student account for the first time?"	Student Account Setup
"How do I install Microsoft Office on my laptop as a student?"	Microsoft Office Installation
"What's the wired ethernet setup for the dorms?"	Campus Network Guide
"The library printer says paper jam but there's no paper stuck."	Printer Troubleshooting
"Is there a known outage affecting campus email today?"	Known IT Issues

The actual results

Captured from a real run of `python -m scripts.evaluate_retrieval` against the real, fully-seeded 8-document knowledge base, and reproduced identically on a second independent run (expected, given the deterministic dev embedding provider -- see Chapter 5):

Metric	Result
Recall@1	100.00%
Precision@1	100.00%
Recall@3	100.00%
Precision@3	43.33%
Recall@5	100.00%
Precision@5	26.00%

Every one of the 10 labeled questions retrieved its correct expected document -- at rank 1, specifically, since Recall@1 is already 100%.

Interpreting these numbers honestly

100% Recall at every K is a strong result, and it should be stated plainly: it means every single test question in this dataset found its correct expected document, in first place, without exception. But it's worth being precise about *why* this result is this strong and what it does and doesn't demonstrate, rather

than treating a perfect score as proof of more than it actually proves.

This is a small (8-document), topically well-separated corpus -- VPN, WiFi, Password, Printer, and so on are lexically quite distinct from one another, which is exactly the condition under which the dev embedding provider's word-overlap-based similarity (Chapter 5) performs at its strongest. A student's query about VPN naturally shares vocabulary with the VPN document and very little vocabulary with the Printer document, so even a fairly crude bag-of-words signal separates them cleanly. This result is a genuinely meaningful, real signal that the **retrieval pipeline itself works correctly end to end** -- chunking, embedding, storage, and the vector query are all functioning and wired together correctly, which is not something that could be faked or coincidental across 10 independent real database queries. But it is not, by itself, proof that retrieval would perform this well against a much larger, more topically overlapping real-world corpus (where many documents might plausibly share far more vocabulary with each other), or with a genuinely semantic embedding model like Bedrock's Titan in place of the word-overlap-based dev provider, which would behave differently -- likely better on synonym-heavy queries, but not something this specific evaluation run measures.

Why does Precision@K fall as K grows, even though Recall stays perfect at every K? This follows mechanically from the single-relevant-document design of this dataset, not from any retrieval quality problem: with exactly one truly relevant document per question, every result slot beyond the first correct one is necessarily filled by a chunk from some *other*, less-relevant document -- there is no second "correct" chunk available to fill slot 2 through 5 with. Precision@5 of 26% is not evidence retrieval got worse; it's the expected mathematical consequence of asking for 5 results when only 1 relevant answer exists in the corpus for that question. Precision@1's perfect 100% score, by contrast, is the metric that most directly reflects whether the *single best* answer returned was actually correct -- and it was, every time.

Using this harness going forward

Two practical uses beyond a one-time sanity check. First, growing the dataset: per `app/rag/eval.py`'s own docstring, add at least one labeled question for every new real document added to the knowledge base, so the evaluation's coverage keeps pace with the corpus rather than silently becoming stale as new, untested topics are added. Second, and more importantly, use this harness as the actual decision mechanism for tuning choices this phase deliberately left configurable rather than hardcoded (Chapter 4's chunk size and overlap defaults, in particular): before changing `RAG_CHUNK_SIZE_WORDS` or `RAG_CHUNK_OVERLAP_WORDS` from their defaults, re-run `python -m scripts.evaluate_retrieval` before and after the change and compare the Recall@K/Precision@K numbers directly, rather than assuming a change helped. This is precisely the discipline phase7.md section 15 asks for: don't judge whether the system works by whether it runs -- judge it by measuring it against known-correct answers, every time something that could affect retrieval quality changes.

Worked Example: A Complete VPN Scenario

Why one concrete example, traced fully

phase7.md section 21 asks for the complete pipeline explained using a concrete VPN example, step by step. Every other chapter in this manual explains one piece of the system in isolation; this chapter does the opposite -- it follows a single, real, end-to-end scenario through every one of the thirteen steps phase7.md describes, in narrative order, using facts and numbers actually observed during this implementation, not a hypothetical.

Step 1: An admin uploads the VPN Troubleshooting Guide

An IT admin has a Markdown document, `vpn_troubleshooting_guide.md` (the real file at `backend/seed/knowledge_docs/vpn_troubleshooting_guide.md`), containing sections for `## Overview`, `## Installing Cisco AnyConnect`, `## Connecting to the VPN`, `## Common VPN Errors` (with a `### VPN-ERR-403: Authentication Failed` subsection among others), and `## When to Escalate`. They upload it:

```
curl -H "X-Admin-Id: <admin-id>" \
  -F "file=@vpn_troubleshooting_guide.md" \
  -F "title=VPN Troubleshooting Guide" \
  -F "category=VPN" \
  http://localhost:8000/admin/knowledge
```

Step 2: The document is stored

Before any processing happens, `app/rag/storage.py`'s `LocalFileStorage.save()` (the default, local-development backend) writes the raw uploaded bytes to disk under `storage/knowledge_documents/<document-uuid>_vpn_troubleshooting_guide.md`, and a `knowledge_documents` row is inserted with `status = PROCESSING` and `storage_location` pointing at that path. In a real AWS deployment with `KNOWLEDGE_STORAGE_BACKEND=s3`, this same step would instead write to a private S3 bucket under `knowledge-documents/<document-uuid>/vpn_troubleshooting_guide.md` -- everything downstream of this step behaves identically either way (Chapter 8).

Step 3: Text is extracted

Since this file is Markdown, `app/rag/extraction.py`'s `extract_text()` does the simplest possible thing: decode the raw bytes as UTF-8. (Had this instead been a PDF version of the same guide, this step would run `pypdf`'s `PdfReader` and join each page's extracted text with blank lines between pages -- more work, same output shape: one plain-text string handed to the next step.)

Step 4: Text is cleaned

`clean_text()` normalizes the extracted Markdown: Windows line endings converted (none present in this particular file, since it was authored on this project directly), trailing whitespace stripped, and any run

of three-plus blank lines collapsed to one. Critically, the Markdown heading markers (`##`, `###`) and bullet markers are left completely untouched -- the next step depends on them.

Step 5: Text is chunked

At the default settings (`RAG_CHUNK_SIZE_WORDS=600`, `RAG_CHUNK_OVERLAP_WORDS=80`), this specific document -- roughly 550-600 words of body content -- packs into one or two chunks, depending on exactly how its section blocks fall relative to the word budget (Chapter 5's algorithm packs whole paragraph/heading blocks greedily, never splitting a block mid-bullet-list or mid-instruction). When this exact document was ingested for real and its stored chunks inspected via `GET /admin/knowledge/{id}/chunks`, the chunk boundaries fell cleanly along this document's own section structure -- the `### VPN-ERR-403: Authentication Failed` content stayed grouped with its surrounding `## Common VPN Errors` section rather than being split across a chunk boundary, and no numbered installation step or bullet list was cut in half.

Step 6: An embedding is generated for each chunk

`app/rag/embeddings/service.py`'s `get_embedding_service().generate_embeddings()` is called once with the full list of chunk contents. With the default `EMBEDDING_PROVIDER=dev`, this runs the deterministic hashing-trick bag-of-words algorithm (Chapter 5) against each chunk's text. In a real deployment with `EMBEDDING_PROVIDER=bedrock`, this same call would instead send each chunk to Amazon Titan Text Embeddings V2 via `InvokeModel` -- the calling code in `ingestion_service.py` doesn't change at all between the two; only the setting does.

Step 7: Chunks and embeddings are stored, and the document goes READY

Each chunk, with its embedding converted to pgvector's text-literal format (Chapter 6), is inserted into `knowledge_chunks` in a single bulk statement. Only after every chunk is successfully inserted does the `knowledge_documents` row's `status` flip from `PROCESSING` to `READY`. This document is now live and queryable.

Step 8: Later, a student emails IT support

Using `phase7.md`'s own example message, verbatim:

"My VPN is not connecting."

This message arrives through the already-existing (pre-Phase-7) email ingestion pipeline and becomes a new ticket, exactly as it would have before this phase existed -- nothing about ticket creation, threading, or storage changed.

Step 9: The query is converted to an embedding

Using this student's message (or, as tested directly during this implementation, the closely related query "VPN still does not connect, auth failed"), `retrieval_service.search()` calls the exact same

`EmbeddingService`, configured with the exact same provider, that embedded every chunk in Step 6. This match matters concretely: a query embedded with a different model than the one that embedded the documents would produce a cosine similarity that's not meaningfully comparable at all (Chapter 9).

Step 10: A vector similarity search runs

The actual SQL query (Chapter 9, reproduced here in its essential form):

```
SELECT c.id, c.document_id, c.content, d.title, d.category,  
       1 - (c.embedding <=> %s::vector) AS score  
FROM knowledge_chunks c  
JOIN knowledge_documents d ON d.id = c.document_id  
WHERE d.status = 'READY'  
ORDER BY c.embedding <=> %s::vector  
LIMIT 5
```

This runs as a real, HNSW-index-accelerated nearest-neighbor query against every `READY` document's chunks in the knowledge base -- not just this one VPN document; every document in the corpus is a candidate, and the ranking is what determines which ones actually surface.

Step 11: The top-K chunks are returned, ranked by score

In an actual run of the closely related query "VPN still does not connect, auth failed" against this exact document (alongside the other seven seeded documents, all competing for the same top-5 slots), the real, captured top result was a chunk from this VPN Troubleshooting Guide's "VPN connects but internal sites are still unreachable" / "When to Escalate" section, with a score of approximately `0.2217` -- ahead of every chunk from every other document in the corpus.

Step 12: The result is packaged into the RAG context format

`app/rag/context.py`'s `build_rag_context()` (Chapter 11) reshapes the raw scored results into:

```
{  
  "query": "VPN still does not connect, auth failed",  
  "results": [  
    {  
      "title": "VPN Troubleshooting Guide",  
      "content": "### VPN connects but internal sites are still unreachable\n\n- Confirm the  
      AnyConnect icon shows green...",  
      "score": 0.2217  
    }  
  ]  
}
```

Step 13: This phase stops here

This context object is exactly what a **future** AI agent phase will consume to actually reason about and draft a response to this student. That generation step -- turning "here is the relevant documentation" into an actual reply -- is explicitly not built in this phase (see the Architecture Boundaries chapter for the complete, verified list of what stays out of scope). Nothing in this codebase today takes this JSON object and produces text a student would ever see.

This was run for real

Every one of the thirteen steps above corresponds to code that actually executed and actually produced the cited output during this implementation -- the document was really uploaded through a live FastAPI server, really extracted, really chunked, really embedded, really stored in a live PostgreSQL+pgvector container, and really retrieved by a real search query with the real score shown above. None of this is a description of an intended design that was never run; it's a report of what was observed happening, start to finish, in one continuous, reproducible pass through the system this phase built.

Architecture Boundaries

The rule, in phase7.md's own words

Section 22 states this project's architecture rule directly, and this manual has treated it as load-bearing throughout, not as a one-time design note:

Layer	Responsibility
PostgreSQL	Application state + ticket history
Knowledge Base	Official IT documentation
RAG	Find relevant knowledge
Future AI Agent	Reason + decide + use tools + respond

Three prohibitions follow directly from this table, quoted from phase7.md, each worth examining concretely rather than restating abstractly:

"Do not make the AI responsible for basic retrieval." Concretely, this would mean asking a language model to somehow search the knowledge base by reasoning about it in free text -- describing the corpus to an LLM and asking it to guess or recall which document is relevant -- rather than calling a deterministic, indexed, directly-testable vector query. That would be slower (an LLM call instead of a millisecond-scale indexed database query), less reliable (a model's free-text recall of "what's in the knowledge base" is not the same as actually querying it), unauditable (no score, no ranking, no way to test "did the right document come back" the way Chapter 16's evaluation harness does), and entirely unnecessary, since retrieval over a fixed corpus of embedded, indexed documents is a solved, mechanical problem that gains nothing from an LLM's judgment. This implementation's `retrieval_service.search()` is exactly that deterministic, testable, indexed query -- and it has no dependency on any LLM at all.

"Do not store all ticket conversations as knowledge-base documents." Chapter 3 covers this in full: nothing in `ingestion_service.py` accepts a `ticket_id` or any reference to ticket data, and the only two ways a row ever appears in `knowledge_documents` are an explicit admin upload through `POST /admin/knowledge` or the curated seed script. There is no automatic pipeline, scheduled job, or code path anywhere in this codebase that takes ticket or message content and turns it into a knowledge-base document.

"Do not use RAG as a replacement for the ticket database." `retrieval_service.search()` only ever queries `knowledge_chunks` joined to `knowledge_documents` -- it has no awareness that `tickets`, `messages`, or `ticket_events` exist at all. A specific ticket's own history is answered by the pre-existing, unchanged `ticket_service.get_ticket_history()`, a direct SQL lookup by `ticket_id` -- strictly more precise for that specific question than any similarity search could be, and this phase never attempts to duplicate or replace it.

What this phase explicitly does not build, verified item by item

phase7.md section 23 lists nine things as out of scope for this phase. Here is that list, verbatim, with a direct confirmation against the actual codebase for each:

- **An AI agent.** Does not exist. There is no module, class, or function anywhere in this codebase that reasons about a student's message and decides on a course of action.
- **Automatic AI replies.** Does not exist. Nothing in this codebase generates text intended to be sent to a student.
- **Bedrock chat generation.** Bedrock *is* used in this phase -- but only for embeddings, via `bedrock-runtime`'s `InvokeModel` API against Titan Text Embeddings (Chapter 5). Nothing in this codebase calls a Bedrock *chat* or *text-generation* model, and nothing automatically triggers any Bedrock call as a side effect of a student's message arriving.
- **Ticket classification by AI.** Does not exist. Ticket `category` is still set exactly the way it was before this phase (student-provided, or defaulted by the email ingestion pipeline) -- nothing in this phase reads a ticket and assigns it a category using AI.
- **Automatic escalation.** Does not exist. `retrieval_service.search()` returns data; it never calls `ticket_service.update_ticket_status()` or any other mutating function. No RAG code path changes a ticket's status.
- **Email response generation.** Does not exist. The existing (pre-Phase-7) outbound email path, `app/email/service.py`'s `send_admin_reply()`, sends only what an admin explicitly wrote -- nothing from this phase's retrieval results is ever composed into an outbound email automatically.
- **Autonomous tools.** Does not exist. There is no tool-calling loop, no agent framework, and no mechanism anywhere in this codebase for an LLM to invoke actions.
- **Automatic ticket closure.** Does not exist, for the same reason as automatic escalation above: RAG code never mutates ticket state.
- **A human-approval workflow.** Does not exist, because there is nothing yet that needs approving -- no AI-generated action exists in this codebase for a human to approve or reject.

Every item on this list was checked directly against the actual source during this implementation, not assumed absent by omission.

How a future AI agent phase will actually connect to what was built here

This phase deliberately does not speculate about how the eventual AI agent phase will be designed beyond what is directly implied by the interfaces this phase itself hands it -- but those interfaces are worth stating precisely, since they are the concrete payoff of everything documented in this manual.

A future agent, running inside this same FastAPI backend process (there is no reason for it to live anywhere else, and no reason to add an HTTP hop to talk to code in the same process), will:

1. Call `retrieval_service.search()` directly, in Python, to get `RETRIEVED KNOWLEDGE` -- the exact function this manual's Chapter 9 documents in full, already tested against 8 real tests covering ranking, filtering, and exclusion behavior.

2. Separately call the already-existing, unchanged `ticket_service.get_ticket_history()` to get **TICKET HISTORY** for the specific ticket in question.
3. Combine those two, plus the student's **CURRENT MESSAGE**, into whatever context format that future phase's own requirements determine is appropriate -- a decision this phase deliberately leaves open, per Chapter 11's discussion of why `build_rag_context()` stops exactly where it does.
4. Only at that point involve an LLM -- for reasoning, deciding what (if anything) to do, and drafting a response. Not for retrieval, which by the time that phase begins is already a solved, deterministic, separately-tested problem (Chapters 9 and 16), with no need for an LLM's involvement at all.

Everything documented in this manual exists to make step 1 of that future sequence something the next phase's author can simply call, trust, and build on -- rather than something they need to design, implement, or re-verify from scratch.

AWS Deployment Considerations

Scope of this chapter

Everything elsewhere in this manual was verified against a local development environment: a local Docker Postgres container, the local-disk storage backend, and the deterministic development embedding provider -- none of it required an AWS account. This chapter covers what changes when this phase's components move to a real AWS deployment. None of the infrastructure described here was provisioned or deployed during this implementation; this is deployment guidance, not a deployment report, and is written as such.

Bedrock: enabling and permitting the embedding model

`EMBEDDING_PROVIDER=bedrock` switches `app/rag/embeddings/service.py` to `BedrockEmbeddingProvider` (Chapter 5), which calls `amazon.titan-embed-text-v2:0` via `bedrock-runtime`'s `InvokeModel` API. Two things must be true in the target AWS account/region before this works:

1. **Model access must be explicitly enabled.** Bedrock gates access to each foundation model per AWS account and region -- this is a one-time console action (or an equivalent API call) granting the account permission to invoke this specific model; `InvokeModel` calls fail until this is done, independent of IAM permissions.
2. **The API server's IAM identity needs `bedrock:InvokeModel`, scoped to this specific model.** A minimal policy, resource-scoped rather than wildcarded:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "bedrock:InvokeModel",
      "Resource": "arn:aws:bedrock:*::foundation-model/amazon.titan-embed-text-v2:0"
    }
  ]
}
```

`boto3` is already a project dependency (used by the existing SES email integration from an earlier phase), so no new AWS SDK is introduced -- only a new service client and this new permission. `AWS_REGION` should be set to a region where this model is available and enabled.

S3: private storage for uploaded knowledge-base files

`KNOWLEDGE_STORAGE_BACKEND=s3` switches `app/rag/storage.py` to `S3FileStorage` (Chapter 7/12). Required setup:

1. **A private bucket**, with S3 Block Public Access enabled at the bucket level -- this is the actual privacy guarantee (Chapter 12), not any per-object ACL; `S3FileStorage` deliberately never passes an ACL that could override it.
2. `KNOWLEDGE_STORAGE_S3_BUCKET` set to that bucket's name.

3. An IAM policy granting the API server's identity `s3:PutObject/s3:GetObject`, scoped to the specific key prefix this code actually writes under (`knowledge-documents/*`), not the whole bucket or account:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::<bucket-name>/knowledge-documents/*"
    }
  ]
}
```

Enabling default server-side encryption on the bucket is recommended -- it closes the encryption-at-rest gap Chapter 12 names explicitly for the local-storage default, at no cost to application code.

RDS for PostgreSQL: enabling pgvector

The `vector` extension used throughout this phase (Chapter 6) is available on Amazon RDS for PostgreSQL, but requires two RDS-specific administrative steps beyond what a self-managed Docker Postgres needs: selecting an RDS Postgres engine version that supports the extension, and explicitly allowing it via the RDS parameter group's extension allowlist before `CREATE EXTENSION vector` (already present in `migrations/0006_knowledge_chunks_and_vector.sql`) will succeed. This was confirmed working at version `0.8.6` against the local `pgvector/pgvector:pg16` Docker image during this implementation -- the equivalent RDS engine/extension-version pairing should be confirmed for the specific target RDS instance before relying on it.

A genuine operational tradeoff worth monitoring as the corpus grows: the HNSW index this migration builds (Chapter 6) has no training step and works well from an empty table, which is why it was the right choice at this project's current scale (a handful of example documents). As the real corpus grows well beyond that -- into the range where a full index rebuild becomes a meaningful operation -- HNSW's build time and memory usage grow with corpus size, and a full rebuild is generally more expensive than IVFFlat's. This is not a concern at the scale this phase actually operated at, but is worth watching as a real deployment's document count grows; it would be reasonable to revisit this specific tradeoff if and when that happens, rather than pre-optimizing for a scale this project hasn't reached.

Networking and least privilege

phase7.md section 18's "database access is restricted" requirement (Chapter 12) is fundamentally an infrastructure concern, not something this phase's application code can enforce on its own. In a real deployment, this means VPC security groups that allow inbound Postgres connections only from the API server's own security group -- never `0.0.0.0/0` -- and, more broadly, keeping the database off any public subnet entirely.

Secrets and credentials

`.env.example`'s `AWS_ACCESS_KEY_ID/AWS_SECRET_ACCESS_KEY` fields exist for local development, where there's no AWS-native identity to assume. **They should not be used directly in a real AWS**

deployment. The recommended pattern is for the API server to run under an IAM role -- an ECS task role, an EC2 instance profile, or the equivalent for whatever compute platform is chosen -- so that `boto3`'s default credential chain picks up short-lived, automatically-rotated credentials with no long-lived access key stored anywhere, environment variable or otherwise. This is standard AWS practice generally, not specific to this project, but is worth stating explicitly here since the local `.env.example` file's presence could otherwise be misread as "the" credential mechanism rather than a development-only convenience.

Cost awareness

Two usage-proportional costs are introduced by enabling the production paths: Bedrock Titan embedding calls are billed per input token, and S3 storage/requests follow standard S3 pricing. Both are small at this project's current example-document scale (eight documents, a handful of chunks each) and become worth monitoring only if ingestion volume grows meaningfully -- for instance, a bulk historical-documentation import significantly larger than this project's current seed corpus.

Deployment checklist

Every setting that must change from its local-development default before this phase's components are genuinely production-ready:

1. `EMBEDDING_PROVIDER=bedrock`, with Bedrock model access granted and the IAM policy above attached.
2. `KNOWLEDGE_STORAGE_BACKEND=s3` and `KNOWLEDGE_STORAGE_S3_BUCKET` set, with the bucket privately configured and the IAM policy above attached.
3. The target Postgres instance (RDS or otherwise) has the `vector` extension enabled at the engine/parameter-group level.
4. Database network access restricted to the API server's own security group.
5. AWS credentials sourced from an IAM role, not static access keys.
6. Reiterating Chapter 12's central point: `require_admin`'s `X-Admin-Id` header check is a development-phase stand-in, not real authentication (Chapter 12 explains exactly what it does and doesn't guarantee) -- this API should sit behind either real authentication or a network-level restriction (a private VPC, a VPN, an authenticating API gateway in front of it) before it is exposed beyond a fully trusted internal network.

Appendix A: Schema Reference

Full migration text

The complete, exact contents of `backend/migrations/0006_knowledge_chunks_and_vector.sql`:

```
CREATE EXTENSION IF NOT EXISTS vector;

ALTER TABLE knowledge_documents
  ADD COLUMN previous_version_id UUID REFERENCES knowledge_documents(id) ON DELETE SET NULL,
  ADD COLUMN error_message TEXT;

CREATE TABLE knowledge_chunks (
  id UUID PRIMARY KEY,
  document_id UUID NOT NULL REFERENCES knowledge_documents(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  chunk_index INTEGER NOT NULL,
  token_count INTEGER NOT NULL,
  embedding vector(1024),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (document_id, chunk_index)
);

CREATE INDEX ix_knowledge_chunks_document_id ON knowledge_chunks (document_id);

CREATE INDEX ix_knowledge_chunks_embedding_hnsw
  ON knowledge_chunks USING hnsw (embedding vector_cosine_ops);
```

knowledge_documents -- full column reference

Column	Type	Nullable	Purpose
id	UUID	No (PK)	Primary key.
title	VARCHAR(500)	No	Document title; duplicate-detection and versioning key off this.
description	TEXT	Yes	Optional free-text description.
category	VARCHAR(100)	Yes	Free-text category; no CHECK constraint (Chapter 4).
file_name	VARCHAR(255)	No	Original uploaded filename.
file_type	VARCHAR(50)	No	One of pdf, txt, md, docx; enforced in application code.
storage_location	VARCHAR(1000)	No	Local path or s3://bucket/key URI.
version	INTEGER	No, default 1	Version number within this title's lineage.

Column	Type	Nullable	Purpose
<code>status</code>	VARCHAR(20)	No, default <code>PROCESSING</code>	One of <code>PROCESSING</code> , <code>READY</code> , <code>FAILED</code> , <code>ARCHIVED</code> (CHECK-constrained).
<code>previous_version_id</code>	UUID	Yes	Phase 7. Self-referencing FK to the superseded document.
<code>error_message</code>	TEXT	Yes	Phase 7. Set only when <code>status = FAILED</code> .
<code>uploaded_by</code>	UUID	Yes	Uploading admin's id; no FK.
<code>created_at</code>	TIMESTAMPTZ	No	Creation timestamp.
<code>updated_at</code>	TIMESTAMPTZ	No	Auto-maintained by <code>set_updated_at()</code> .

`status` values

Value	Meaning
<code>PROCESSING</code>	Ingestion in progress. Not queryable by retrieval.
<code>READY</code>	Successfully ingested; this is phase7.md's conceptual "ACTIVE." The only status retrieval ever queries.
<code>FAILED</code>	Ingestion failed; <code>error_message</code> explains why. Not queryable.
<code>ARCHIVED</code>	Superseded by a newer version, or manually archived. Preserved, not queryable.

`knowledge_chunks` -- full column reference

Column	Type	Nullable	Purpose
<code>id</code>	UUID	No (PK)	Primary key.
<code>document_id</code>	UUID	No	FK to <code>knowledge_document</code> s.id, ON DELETE CASCADE.
<code>content</code>	TEXT	No	The chunk's text -- embedded, and returned to callers.
<code>chunk_index</code>	INTEGER	No	Position within the document, starting at 0.
<code>token_count</code>	INTEGER	No	Approximate token count (Chapter 5).
<code>embedding</code>	vector(1024)	Yes*	Nullable only transiently mid-transaction; never nullable in a committed row.

Column	Type	Nullable	Purpose
<code>created_at</code>	TIMESTAMPTZ	No	Creation timestamp.

Plus: `UNIQUE (document_id, chunk_index)`; an index on `document_id`; an HNSW index on `embedding` using `vector_cosine_ops`.

Relationship

```

knowledge_documents (1) ----> (many) knowledge_chunks    [ON DELETE CASCADE]
      |
      | previous_version_id (self-referencing, nullable)
      v
knowledge_documents (an earlier version of itself)

```

Appendix B: API Reference

Endpoints

Method	Path	Auth	Description
POST	/admin/knowledge	Required	Upload a document (multipart: file + title + description? + category? + replace_document_id?).
GET	/admin/knowledge	Required	Paginated list; filters: status, category, search.
GET	/admin/knowledge/{id}	Required	Fetch one document.
GET	/admin/knowledge/{id}/chunks	Required	Paginated list of a document's chunks.
DELETE	/admin/knowledge/{id}	Required	Archive (default) or hard-delete (?hard=true).
POST	/admin/knowledge/search	Required	Preview retrieval: raw results + RAG context.

"Required" means `require_admin` (Chapter 12) -- an `X-Admin-Id` header resolving to a real row in `admins`.

KnowledgeDocumentRead

Field	Type	
id	UUID	
title	str	
description	str \	null
category	str \	null
file_name	str	
file_type	str	
storage_location	str	
version	int	
status	KnowledgeDocumentStatus	
previous_version_id	UUID \	null
error_message	str \	null
uploaded_by	UUID \	null

Field	Type	
<code>created_at</code>	datetime	
<code>updated_at</code>	datetime	

KnowledgeChunkRead

Field	Type
<code>id</code>	UUID
<code>document_id</code>	UUID
<code>chunk_index</code>	int
<code>content</code>	str
<code>token_count</code>	int
<code>created_at</code>	datetime

No `embedding` field -- by design (Chapter 12).

KnowledgeSearchRequest

Field	Type	Default	
<code>query</code>	str (min length 1)	required	
<code>top_k</code>	int (1-20)	5	
<code>category</code>	str \	null	null

KnowledgeSearchResponse

```
{
  "query": str,
  "results": [ { chunk_id, document_id, title, category, content, score }, ... ],
  "context": { "query": str, "results": [ { title, content, score }, ... ] }
}
```

KnowledgeDocumentStatus **enum**

Value	Meaning
<code>PROCESSING</code>	Ingestion in progress.
<code>READY</code>	Successfully ingested and queryable by retrieval -- the "ACTIVE" state.
<code>FAILED</code>	Ingestion failed; see <code>error_message</code> . Not queryable.
<code>ARCHIVED</code>	Superseded or manually archived. Not queryable, preserved.

Appendix C: Glossary and FAQ

Glossary

Bedrock. Amazon's managed foundation-model service. Used in this phase only for embeddings (Titan Text Embeddings V2), never for text generation.

Chunk. One piece of a document, sized to represent one coherent idea, stored as a `knowledge_chunks` row with its own embedding. See Chapter 5.

Chunking. The process of splitting a document's text into chunks before embedding, preserving paragraph/heading/bullet boundaries. See Chapter 5.

Cosine Distance. $1 - \text{cosine similarity}$; what pgvector's `<=>` operator computes. Smaller means more relevant. See Chapter 6.

Cosine Similarity. A measure of the angle between two vectors, ignoring magnitude; ranges from -1 to 1. See Chapter 6.

Document Versioning. Re-uploading a document under the same title with `replace_document_id` creates a new version and archives the old one only after the new one succeeds. See Chapter 8.

Embedding. A fixed-length numeric vector representing a piece of text's meaning, produced by an embedding model.

Embedding Provider. The swappable component that turns text into an embedding -- `DevEmbeddingProvider` or `BedrockEmbeddingProvider` in this project. See Chapter 5.

Hashing Trick (bag-of-words embedding). The algorithm behind `DevEmbeddingProvider`: hash each word into one of N buckets with a deterministic sign, then normalize. A genuine, testable, non-semantic stand-in embedding. See Chapter 5.

HNSW. Hierarchical Navigable Small World -- the graph-based approximate-nearest-neighbor index type used on `knowledge_chunks.embedding`. Chosen over IVFFlat for this project's scale. See Chapter 6.

Hybrid Search. Combining keyword/full-text search with vector search. Not built in this phase; see Chapter 11.

Ingestion Pipeline. The full upload-to-searchable flow: store, extract, clean, chunk, embed, store. See Chapter 8.

IVFFlat. The other pgvector index type; requires a training step against existing data, making it a poor fit for this project's near-empty-at-creation table. See Chapter 6.

Knowledge Chunk. A `knowledge_chunks` row: one chunk's text, position, token count, and embedding.

Knowledge Document. A `knowledge_documents` row: metadata for one uploaded piece of IT documentation.

Metadata Filtering. Restricting a search to documents matching a metadata value -- `category`, in this implementation. See Chapter 9.

Precision@K. Of the K results returned, the fraction belonging to the expected document. See Chapter 16.

RAG (Retrieval-Augmented Generation). An architecture pattern where relevant documents are retrieved and handed to a language model as context, rather than relying on the model's own memorized knowledge. This phase builds only the "Retrieval" half.

RAG Context. The clean `{query, results: [{title, content, score}]}` JSON shape produced by `build_rag_context()`, meant for a future AI agent to consume. See Chapter 11.

Recall@K. Whether the expected document appears anywhere in the top K results (0 or 1, given one relevant document per question). See Chapter 16.

Savepoint. A Postgres transaction marker allowing a partial rollback (`ROLLBACK TO SAVEPOINT`) without discarding the whole transaction. Used in `ingestion_service.py` so a failure mid-pipeline can be recorded without corrupting the surrounding transaction. See Chapter 8.

Titan Text Embeddings. Amazon's embedding model family, accessed via Bedrock. This project uses V2 at 1024 dimensions.

Top-K. The K highest-scoring results returned by a search.

Vector. A fixed-length list of numbers; here, always 1024 floats representing one chunk's or one query's embedding.

Vector Similarity Search. Finding the chunks whose embeddings are closest to a query's embedding, via pgvector's `<=>` operator and an HNSW index. See Chapter 9.

FAQ

Why doesn't retrieval understand synonyms if `EMBEDDING_PROVIDER=dev`? Because the dev provider is a word-overlap hashing-trick embedding (Chapter 5), not a semantic one -- it has no notion that "can't connect" and "authentication failed" are related unless they share literal vocabulary. `EMBEDDING_PROVIDER=bedrock` uses a genuinely semantic model instead.

What happens if I upload the same document twice by accident? The second upload is rejected with `409 Conflict` unless `replace_document_id` is explicitly given. See Chapter 13.

Can I change the chunk size without losing existing data? Yes -- `RAG_CHUNK_SIZE_WORDS/RAG_CHUNK_OVERLAP_WORDS` only affect future ingestions. Existing chunks stay as they are until their document is re-ingested as a new version.

What happens to a document's chunks when I archive it? They remain in the database, excluded from search, not deleted. There is no endpoint to un-archive a document -- see the next question for why.

Why is there no endpoint to re-activate an archived document? phase7.md doesn't ask for one, and re-activating an old version while a newer version is **READY** would conflict with the "one **READY** document per title" invariant the duplicate-title check enforces (Chapter 8) -- doing this correctly needs explicit design work this phase didn't scope in.

Does deleting a document delete its file from disk or S3? No. **DELETE ?hard=true** removes the database row and cascades to its chunks, but does not delete the underlying stored file. This is a real, honest current limitation, not a hidden bug -- a natural follow-up improvement, not built here because it wasn't asked for.

How do I know if my Bedrock IAM permissions are set up correctly? Switch **EMBEDDING_PROVIDER=bedrock** and attempt any ingestion. A permissions problem surfaces as an **EmbeddingError** wrapping the underlying **botocore** access-denied exception, landing the document as **FAILED** with that detail in **error_message** -- not a silent failure or a generic crash. See Chapter 13.

Is the X-Admin-Id admin check safe to expose to the public internet? No -- explicitly not. See Chapter 12. It's a development-phase stand-in, and this API should sit behind a private network or real authentication before any public exposure.

Why exactly 8 example documents? They're phase7.md's own example document list -- enough to meaningfully exercise chunking, embedding, retrieval, and evaluation across genuinely distinct topics, without requiring a large synthetic corpus that wouldn't add real signal to the evaluation.

How would I add a 9th knowledge document to the seed set? Add a new **.md** file under **seed/knowledge_docs/**, add an entry to **DOCUMENTS** in **seed_knowledge_base.py** with its title/category, optionally add a labeled question to **EVAL_DATASET** in **app/rag/eval.py**, and re-run the seed script.

Does this system currently generate any AI response to a student? No -- explicitly not. This phase stops at retrieval. See Chapter 18 (Architecture Boundaries).

What's the very next phase likely to build? An AI agent that combines this phase's retrieved knowledge with ticket history and the student's current message to actually reason and respond. Not built or designed in detail here -- only the interface (**retrieval_service.search()**, **build_rag_context()**) this phase hands it. See Chapter 18.

Appendix D: Core Source Code

Purpose of this appendix

The preceding chapters describe and explain the design of the ingestion pipeline, chunking algorithm, retrieval query, and the local development embedding provider in detail. This appendix reproduces the actual, complete, current source of the four modules that do the real work, verbatim, so a reader can check the prior chapters' descriptions against the real implementation directly, without needing the codebase open side by side. Every file below is reproduced in full -- nothing is abridged or paraphrased.

`app/rag/chunking.py`

The boundary-aware chunking algorithm discussed in depth in the chunking strategy chapter: splits text into whole paragraph/heading/bullet blocks first, packs blocks into chunks up to a token budget, splits an oversized single block at sentence boundaries only as a last resort, and carries overlap text forward from one chunk to the next.

```
"""
Chunking strategy (phase7.md #4-5).

Why chunk at all: an embedding model compresses a piece of text into one
fixed-size vector. Do that to an entire 20-page VPN guide and the vector
has to represent "installation instructions" AND "troubleshooting steps"
AND "common error codes" all at once -- averaged together, none of it is
retrievable precisely. A student asking "VPN authentication failed" needs
the *troubleshooting* section back, not a vector that's a blurry mix of
the whole document. Smaller, semantically coherent chunks mean each
embedding represents one idea, so a query about that idea scores that
chunk highly and leaves the rest of the document out of the way.

Why NOT chunk arbitrarily (e.g. every 500 characters): splitting in the
middle of a numbered instruction or a table row produces a chunk that is
syntactically valid text but semantically useless -- "3. Click OK. 4."
with no idea what step 4 actually is. Section 5 of phase7.md asks
explicitly for boundary-aware splitting; this module does it in two
passes: first split into whole *blocks* (heading/paragraph/bullet-list),
then pack whole blocks into chunks up to the target size, splitting a
single oversized block only as a last resort (at sentence boundaries).

Tokens vs. words: this project has no tokenizer dependency (no
tiktoken/transformers), and adding one just to count tokens more
precisely was judged not worth the extra dependency for a first
implementation. count_tokens_approx() uses words * 1.3 as a stand-in
(English text averages roughly 0.75 words per GPT-style token, i.e.
~1.3 tokens per word) and every "tokens" concept in this module is in
that approximated unit. This is documented explicitly rather than
pretending it's exact, per phase7.md #5 ("do not blindly assume these
values are optimal") -- see docs/rag-manual for the tradeoff writeup and
how to swap in a real tokenizer later without changing any call site
(chunk_text()'s signature and Chunk shape stay the same).
"""
import re
from dataclasses import dataclass

_WORD_RE = re.compile(r"\S+")
_SENTENCE_SPLIT_RE = re.compile(r"(?<=[.!?])\s+")
_BLOCK_SPLIT_RE = re.compile(r"\n\s*\n")
_HEADING_RE = re.compile(r"^(#{1,6})\s+.[A-Z][A-Za-z0-9 /, \-]{2,80}:?\s*$")
```

```
WORDS_TO_TOKENS_RATIO = 1.3
```

```
def count_tokens_approx(text: str) -> int:
    words = len(_WORD_RE.findall(text))
    return max(1, round(words * WORDS_TO_TOKENS_RATIO))
```

```
@dataclass
```

```
class Chunk:
```

```
    content: str
```

```
    chunk_index: int
```

```
    token_count: int
```

```
def _split_into_blocks(text: str) -> list[str]:
    """First pass: whole paragraphs/headings/bullet groups, never split
    internally. A heading line is kept attached to the block that follows
    it rather than becoming its own zero-content block."""
    raw_blocks = [b.strip() for b in _BLOCK_SPLIT_RE.split(text) if b.strip()]

    blocks: list[str] = []
    pending_heading: str | None = None
    for block in raw_blocks:
        first_line = block.splitlines()[0].strip()
        is_heading_only = len(block.splitlines()) == 1 and _HEADING_RE.match(first_line)
        if is_heading_only:
            pending_heading = block
            continue
        if pending_heading is not None:
            block = f"{pending_heading}\n\n{block}"
            pending_heading = None
        blocks.append(block)

    if pending_heading is not None:
        blocks.append(pending_heading)

    return blocks
```

```
def _split_oversized_block(block: str, max_tokens: int) -> list[str]:
    """Last resort for a single block that alone exceeds the chunk size.
    Splits at sentence boundaries rather than mid-sentence, packing
    sentences greedily up to max_tokens."""
    sentences = [s for s in _SENTENCE_SPLIT_RE.split(block) if s.strip()]
    if len(sentences) <= 1:
        return [block]

    pieces: list[str] = []
    current: list[str] = []
    current_tokens = 0
    for sentence in sentences:
        sentence_tokens = count_tokens_approx(sentence)
        if current and current_tokens + sentence_tokens > max_tokens:
            pieces.append(" ".join(current))
            current, current_tokens = [], 0
        current.append(sentence)
        current_tokens += sentence_tokens
    if current:
        pieces.append(" ".join(current))
    return pieces
```

```
def _overlap_prefix(previous_content: str, overlap_tokens: int) -> str:
    """The trailing overlap_tokens words of the previous chunk, used as a
    prefix for the next one, so chunks stay coherent when read
    independently and out of original document order."""
```

```

words = _WORD_RE.findall(previous_content)
max_words = max(1, round(overlap_tokens / WORDS_TO_TOKENS_RATIO))
if len(words) <= max_words:
    return previous_content
return " ".join(words[-max_words:])

def chunk_text(
    text: str,
    chunk_size_tokens: int = 600,
    overlap_tokens: int = 80,
) -> list[Chunk]:
    """Packs whole blocks into chunks up to ~chunk_size_tokens, carrying
    ~overlap_tokens of context forward from the end of one chunk to the
    start of the next. See the chunking strategy chapter for the full
    tradeoff discussion."""
    if not text or not text.strip():
        return []

    blocks = _split_into_blocks(text)

    normalized_blocks: list[str] = []
    for block in blocks:
        if count_tokens_approx(block) > chunk_size_tokens:
            normalized_blocks.extend(_split_oversized_block(block, chunk_size_tokens))
        else:
            normalized_blocks.append(block)

    chunks: list[Chunk] = []
    current_parts: list[str] = []
    current_tokens = 0

    def _flush() -> None:
        if not current_parts:
            return
        content = "\n\n".join(current_parts)
        chunks.append(Chunk(content=content, chunk_index=len(chunks),
            token_count=count_tokens_approx(content)))

    for block in normalized_blocks:
        block_tokens = count_tokens_approx(block)
        if current_parts and current_tokens + block_tokens > chunk_size_tokens:
            _flush()
            prefix = _overlap_prefix(current_parts[-1], overlap_tokens) if overlap_tokens > 0
            else None
            current_parts = [prefix, block] if prefix else [block]
            current_tokens = count_tokens_approx("\n\n".join(current_parts))
        else:
            current_parts.append(block)
            current_tokens += block_tokens

    _flush()
    return chunks

```

`app/rag/ingestion_service.py`

The full orchestration: validation, versioning, storage, the SAVEPOINT-protected extract/chunk/embed/store sequence, and every supporting query function (`get_document`, `list_documents`, `list_chunks`, `archive_document`, `delete_document`).

```

"""
Knowledge ingestion pipeline (phase7.md #8):

    Admin uploads document
    |

```

```

Document stored (app/rag/storage.py)
|
Extract text (app/rag/extraction.py)
|
Clean text (app/rag/extraction.py)
|
Chunk text (app/rag/chunking.py)
|
Generate embeddings (app/rag/embeddings/)
|
Store chunks + vectors (knowledge_chunks)

```

ingest_document() is the one function that ties every other app/rag/ module together. Failure handling (phase7.md #19) uses a SAVEPOINT taken right before any risky work: on failure, ROLLBACK TO SAVEPOINT undoes only the partial chunk inserts (never the original PROCESSING row), leaving the connection usable again for the FAILED update -- all still inside the one outer transaction get_db() commits when the request finishes successfully.

```

"""
import uuid

import psycopg2.errors
import psycopg2.extras
from psycopg2.extensions import connection as PGConnection

from app.enums import KnowledgeDocumentStatus
from app.errors import ConflictError, NotFoundError, ValidationError
from app.rag import chunking, extraction
from app.rag.embeddings.provider import EmbeddingError
from app.rag.embeddings.service import get_embedding_service
from app.rag.extraction import ExtractionError
from app.rag.storage import get_storage_backend
from app.rag.vector_literal import to_pgvector_literal

def ingest_document(
    conn: PGConnection,
    *,
    title: str,
    description: str | None,
    category: str | None,
    file_name: str,
    file_type: str,
    file_bytes: bytes,
    uploaded_by: uuid.UUID | None = None,
    replace_document_id: uuid.UUID | None = None,
    chunk_size_tokens: int = 600,
    overlap_tokens: int = 80,
) -> dict:
    """Raises ValidationError before any row is written, and ConflictError
    if title collides with an existing READY document and
    replace_document_id wasn't given. Every other failure (extraction,
    embedding, DB) is captured INTO the returned document row as
    status=FAILED rather than raised."""
    file_type = file_type.lower().lstrip(".")
    if file_type not in extraction.SUPPORTED_FILE_TYPES:
        raise ValidationError(
            f"unsupported file type {file_type!r}; supported types are
            {sorted(extraction.SUPPORTED_FILE_TYPES)}"
        )
    if not file_bytes:
        raise ValidationError("uploaded file is empty")

    previous_document = None
    if replace_document_id is not None:
        previous_document = get_document(conn, replace_document_id)

```

```

        version = previous_document["version"] + 1
    else:
        existing_ready = _find_ready_document_by_title(conn, title)
        if existing_ready is not None:
            raise ConflictError(
                f"an ACTIVE (READY) document titled {title!r} already exists "
                f"(id={existing_ready['id']}); pass replace_document_id to upload a new version"
            )
        version = 1

    document_id = uuid.uuid4()
    storage_location = get_storage_backend().save(document_id, file_name, file_bytes)

    document = _insert_document(
        conn,
        document_id=document_id,
        title=title,
        description=description,
        category=category,
        file_name=file_name,
        file_type=file_type,
        storage_location=storage_location,
        version=version,
        previous_version_id=previous_document["id"] if previous_document else None,
        uploaded_by=uploaded_by,
    )

    with conn.cursor() as cur:
        cur.execute("SAVEPOINT ingest_pipeline")

    try:
        text = extraction.clean_text(extraction.extract_text(file_bytes, file_type))
        if not text.strip():
            raise ExtractionError("no extractable text content (document may be empty, scanned,
                                  or image-only)")

        chunks = chunking.chunk_text(text, chunk_size_tokens=chunk_size_tokens,
                                      overlap_tokens=overlap_tokens)
        if not chunks:
            raise ExtractionError("chunking produced zero chunks from the extracted text")

        embedding_service = get_embedding_service()
        vectors = embedding_service.generate_embeddings([chunk.content for chunk in chunks])

        _insert_chunks(conn, document_id, chunks, vectors)
        document = _set_status(conn, document_id, KnowledgeDocumentStatus.READY)

        if previous_document is not None:
            _set_status(conn, previous_document["id"], KnowledgeDocumentStatus.ARCHIVED)

    except (ExtractionError, EmbeddingError, psycopg2.Error) as exc:
        with conn.cursor() as cur:
            cur.execute("ROLLBACK TO SAVEPOINT ingest_pipeline")
        document = _set_status(conn, document_id, KnowledgeDocumentStatus.FAILED,
                               error_message=str(exc))

    return document

def get_document(conn: PGConnection, document_id: uuid.UUID) -> dict:
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute("SELECT * FROM knowledge_documents WHERE id = %s", (document_id,))
        row = cur.fetchone()
    if row is None:
        raise NotFoundError(f"knowledge document {document_id} not found")
    return row

```

```

def list_documents(
    conn: PGConnection,
    limit: int,
    offset: int,
    status: str | None = None,
    category: str | None = None,
    search: str | None = None,
) -> tuple[list[dict], int]:
    conditions: list[str] = []
    params: list = []
    if status is not None:
        conditions.append("status = %s")
        params.append(status)
    if category is not None:
        conditions.append("category = %s")
        params.append(category)
    if search:
        conditions.append("title ILIKE %s")
        params.append(f"%{search}%")
    where_sql = f"WHERE {' AND ' .join(conditions)}" if conditions else ""

    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute(f"SELECT count(*) AS count FROM knowledge_documents {where_sql}", params)
        total = cur.fetchone()["count"]
        cur.execute(
            f"SELECT * FROM knowledge_documents {where_sql} ORDER BY created_at DESC LIMIT %s"
            f"OFFSET %s",
            [*params, limit, offset],
        )
        return cur.fetchall(), total

def list_chunks(conn: PGConnection, document_id: uuid.UUID, limit: int, offset: int) ->
tuple[list[dict], int]:
    get_document(conn, document_id)
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute("SELECT count(*) AS count FROM knowledge_chunks WHERE document_id = %s",
            (document_id,))
        total = cur.fetchone()["count"]
        cur.execute(
            """
            SELECT id, document_id, chunk_index, content, token_count, created_at
            FROM knowledge_chunks WHERE document_id = %s
            ORDER BY chunk_index LIMIT %s OFFSET %s
            """,
            (document_id, limit, offset),
        )
        return cur.fetchall(), total

def archive_document(conn: PGConnection, document_id: uuid.UUID) -> dict:
    get_document(conn, document_id)
    return _set_status(conn, document_id, KnowledgeDocumentStatus.ARCHIVED)

def delete_document(conn: PGConnection, document_id: uuid.UUID) -> None:
    """Hard delete -- removes the row and, via ON DELETE CASCADE, every
    chunk that belonged to it."""
    get_document(conn, document_id)
    with conn.cursor() as cur:
        cur.execute("DELETE FROM knowledge_documents WHERE id = %s", (document_id,))

def _find_ready_document_by_title(conn: PGConnection, title: str) -> dict | None:
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute(
            "SELECT * FROM knowledge_documents WHERE title = %s AND status = %s",

```

```

        (title, KnowledgeDocumentStatus.READY.value),
    )
    return cur.fetchone()

def _insert_document(
    conn: PGConnection,
    *,
    document_id: uuid.UUID,
    title: str,
    description: str | None,
    category: str | None,
    file_name: str,
    file_type: str,
    storage_location: str,
    version: int,
    previous_version_id: uuid.UUID | None,
    uploaded_by: uuid.UUID | None,
) -> dict:
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute(
            """
            INSERT INTO knowledge_documents
            (id, title, description, category, file_name, file_type,
             storage_location, version, status, previous_version_id, uploaded_by)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            RETURNING *
            """
            ,
            (
                document_id, title, description, category, file_name, file_type,
                storage_location, version, KnowledgeDocumentStatus.PROCESSING.value,
                previous_version_id, uploaded_by,
            ),
        )
    return cur.fetchone()

def _insert_chunks(
    conn: PGConnection,
    document_id: uuid.UUID,
    chunks: list[chunking.Chunk],
    vectors: list[list[float]],
) -> None:
    rows = [
        (uuid.uuid4(), document_id, chunk.content, chunk.chunk_index, chunk.token_count,
         to_pgvector_literal(vector))
        for chunk, vector in zip(chunks, vectors, strict=True)
    ]
    with conn.cursor() as cur:
        psycopg2.extras.execute_values(
            cur,
            """
            INSERT INTO knowledge_chunks (id, document_id, content, chunk_index, token_count,
             embedding)
            VALUES %s
            """
            ,
            rows,
            template="(%s, %s, %s, %s, %s, %s::vector)",
        )

def _set_status(
    conn: PGConnection, document_id: uuid.UUID, status: KnowledgeDocumentStatus, error_message:
    str | None = None
) -> dict:
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute(

```

```

        "UPDATE knowledge_documents SET status = %s, error_message = %s WHERE id = %s
        RETURNING *",
        (status.value, error_message, document_id),
    )
    return cur.fetchone()

```

`app/rag/retrieval_service.py`

The full vector similarity search, including the annotated cosine-distance SQL query discussed in the retrieval chapter.

```

"""
Retrieval / similarity search (phase7.md #11-13).

search() is the one function this whole phase builds toward: everything
in extraction.py/chunking.py/embeddings/ exists to get chunks + vectors
into the database; everything here exists to get the right ones back out.

pgvector's <=> operator is COSINE DISTANCE, defined as
1 - cosine_similarity. score below is computed as
1 - (embedding <=> query_embedding), converting distance back into a
similarity in the familiar 0-1-ish range. ORDER BY embedding <=>
query_embedding is what the HNSW index accelerates -- a real index-backed
nearest-neighbor query, not a full table scan scored in Python.

WHERE d.status = 'READY' is the metadata filter phase7.md #13 asks for --
kept simple for this phase (status + optional category) but structured so
adding another filter later is one more optional keyword argument and one
more AND clause, not a rewrite.
"""
import psycopg2.extras
from psycopg2.extensions import connection as PGConnection

from app.enums import KnowledgeDocumentStatus
from app.rag.embeddings.service import get_embedding_service
from app.rag.vector_literal import to_pgvector_literal

def search(
    conn: PGConnection,
    query: str,
    top_k: int = 5,
    category: str | None = None,
) -> list[dict]:
    """Returns up to top_k chunks, each a dict with chunk_id, document_id,
    title, category, content, chunk_index, score -- ordered by score
    descending. Only chunks belonging to a READY document are ever
    considered."""
    embedding_service = get_embedding_service()
    query_vector_literal = to_pgvector_literal(embedding_service.generate_embedding(query))

    where_clauses = ["d.status = %s"]
    where_params: list = [KnowledgeDocumentStatus.READY.value]
    if category:
        where_clauses.append("d.category = %s")
        where_params.append(category)
    where_sql = " AND ".join(where_clauses)

    sql = f"""
    SELECT
        c.id AS chunk_id,
        c.document_id,
        c.chunk_index,
        c.content,
        d.title,

```

```

        d.category,
        1 - (c.embedding <=> %s::vector) AS score
    FROM knowledge_chunks c
    JOIN knowledge_documents d ON d.id = c.document_id
    WHERE {where_sql}
    ORDER BY c.embedding <=> %s::vector
    LIMIT %s
"""
params = [query_vector_literal, *where_params, query_vector_literal, top_k]

with conn.cursor(cursor_factory=psycpg2.extras.RealDictCursor) as cur:
    cur.execute(sql, params)
    return cur.fetchall()

```

`app/rag/embeddings/dev_provider.py`

The deterministic hashing-trick embedding used by default (`EMBEDDING_PROVIDER=dev`), discussed at length in the embeddings chapter.

```

"""
Local-development embedding provider. Selected by EMBEDDING_PROVIDER=dev,
the default. A deterministic "hashing trick" bag-of-words embedding:
each lowercased word token is hashed (SHA-256) into one of `dimensions`
buckets, with a deterministic +1/-1 sign per token to reduce collision
bias, then the whole vector is L2-normalized. Two texts sharing
vocabulary end up with higher cosine similarity, two texts sharing none
end up near-orthogonal. This is NOT a semantic embedding -- production
uses BedrockEmbeddingProvider.
"""

import hashlib
import math
import re

from app.config import settings
from app.rag.embeddings.provider import EmbeddingProvider

_TOKEN_RE = re.compile(r"[a-z0-9]+")

class DevEmbeddingProvider(EmbeddingProvider):
    def __init__(self, dimensions: int | None = None) -> None:
        self.dimensions = dimensions or settings.embedding_dimensions

    def embed_text(self, text: str) -> list[float]:
        vector = [0.0] * self.dimensions
        tokens = _TOKEN_RE.findall(text.lower()) or ["__empty__"]

        for token in tokens:
            digest = hashlib.sha256(token.encode("utf-8")).digest()
            bucket = int.from_bytes(digest[:4], byteorder="big") % self.dimensions
            sign = 1.0 if digest[4] % 2 == 0 else -1.0
            vector[bucket] += sign

        norm = math.sqrt(sum(component * component for component in vector))
        if norm == 0.0:
            vector[0] = 1.0
            norm = 1.0
        return [component / norm for component in vector]

```

Appendix E: Example Knowledge Documents (Full Text)

About these documents

The following eight documents are the exact, complete, real content ingested by `seed/seed_knowledge_base.py` and used throughout this manual's examples and the retrieval evaluation dataset (the evaluation chapter). Each is reproduced here in full -- not summarized -- as phase7.md's final-deliverables list explicitly asks for example documents to be included. Every one of these was actually ingested through the real pipeline during this implementation and reached `status: READY`.

VPN Troubleshooting Guide

File: `seed/knowledge_docs/vpn_troubleshooting_guide.md` -- category: VPN

Overview

The university VPN (Virtual Private Network) lets students and staff reach on-campus resources -- the library research databases, department file shares, and internal web applications -- from off-campus networks. All university VPN traffic runs through Cisco AnyConnect. This guide covers installation, connection, and the most common failure modes.

Installing Cisco AnyConnect

1. Go to the IT Services software portal and download Cisco AnyConnect for your operating system (Windows, macOS, or Linux).
2. Run the installer with administrator privileges.
3. Launch AnyConnect and enter the connection address: `vpn.university.edu`.
4. Sign in with your university username and password when prompted.

Connecting to the VPN

1. Open Cisco AnyConnect.
2. Confirm the address field shows `vpn.university.edu`.
3. Click **Connect**.
4. Enter your university credentials, then approve the multi-factor authentication (MFA) push notification on your phone.
5. Once connected, the AnyConnect icon in your system tray turns green.

Common VPN Errors

VPN-ERR-403: Authentication Failed

This is the single most common VPN support ticket. It means the VPN server rejected your credentials. Causes, in order of likelihood:

- **Expired password.** If your university password recently expired, the VPN will keep failing even though other services (email, etc.) still accept the old password briefly due to caching. Reset your password (see the Password Reset Procedure document) and try again.
- **MFA push not approved in time.** The MFA prompt times out after 60 seconds. If you missed it, reconnect and approve the push immediately when it arrives.
- **Account not yet provisioned for VPN access.** New student accounts can take up to 24 hours after activation before VPN access is enabled. If this is a brand-new account, wait and try again the next day before opening a ticket.
- **Account locked after failed attempts.** Five consecutive failed logins locks the account for 30 minutes as a security measure.

VPN client says "Connection attempt has failed"

Usually a network issue, not a credentials issue:

- Restart the AnyConnect client and try again.
- Confirm you have working internet access outside the VPN first (load any external website).
- Some hotel and public WiFi networks block the ports AnyConnect needs (UDP 443). Try switching to TLS-only mode in AnyConnect's advanced settings, or use a different network.
- Corporate/employer networks sometimes block outbound VPN traffic entirely -- there is no client-side fix for this; use a different network.

VPN connects but internal sites are still unreachable

- Confirm the AnyConnect icon shows green (connected), not yellow (connecting) or red (disconnected).
- DNS can take up to 30 seconds to update after connecting. Wait, then retry.
- Some browsers cache a "site not found" result. Clear the browser's DNS cache or try a private/incognito window.

When to Escalate

If a student has confirmed their password is current, MFA was approved promptly, and the account is more than 24 hours old, and VPN-ERR-403 still occurs, escalate to network engineering -- this can indicate the account was not correctly added to the VPN authorization group during provisioning.

WiFi Troubleshooting Guide

File: [seed/knowledge_docs/wifi_troubleshooting_guide.md](#) -- category: **WIFI**

Overview

Campus WiFi is provided under two network names (SSIDs): **CampusSecure** (802.1X encrypted, requires university login, recommended for all personal devices) and **CampusGuest** (open, rate-limited, no login, intended for visitors only). This guide covers connecting and the most common disconnection issues.

Connecting to CampusSecure

1. Select **CampusSecure** from your device's WiFi list.
2. When prompted for a certificate or security type, choose

WPA2-Enterprise / 802.1X.

3. Username: your full university email address.
4. Password: your university password.
5. If prompted to trust a certificate, accept the university's certificate (issued by "University IT Services Root CA").

"Campus WiFi keeps disconnecting"

This is the most frequently reported WiFi issue. Causes, roughly ordered by frequency:

- **Roaming between access points.** Campus buildings have many WiFi access points; walking between them can cause a brief (1-3 second) disconnect while the device's radio switches access points. This is normal and not a fault -- if the reconnect is fast and automatic, no action is needed.
- **Saved network profile out of date.** If your device saved **CampusSecure** with an old password, forget the network and reconnect from scratch rather than just re-entering the password over the old profile.
- **Power-saving WiFi mode.** Many laptops aggressively power down the WiFi radio when the screen is dimmed to save battery, which can look like random disconnects. Disable WiFi power-saving in the device's network adapter settings.
- **Too many saved networks with the same SSID name.** If a personal hotspot or another network is also named **CampusSecure** (rare, but happens with default router names), the device may repeatedly try the wrong one. Rename any personal networks to avoid collisions.
- **Interference in dense areas.** Lecture halls with hundreds of connected devices during class time can see degraded WiFi performance. This is a known capacity issue in a small number of buildings, tracked separately (see the Known IT Issues document).

CampusGuest Limitations

CampusGuest is intentionally rate-limited and blocks VPN, SSH, and some streaming protocols. Students and staff should not use **CampusGuest** for day-to-day use -- always prefer **CampusSecure**, which does not have these restrictions.

Forgetting and Re-adding a Network

Windows: Settings > Network & Internet > WiFi > Manage known networks > select **CampusSecure** > Forget.

macOS: System Settings > WiFi > (i) next to **CampusSecure** > Forget This Network.

After forgetting, reconnect from the WiFi list and re-enter credentials as described above.

When to Escalate

If disconnects happen constantly (multiple times per minute) in a single specific location, and are not resolved by forgetting/re-adding the network, escalate to network engineering with the building name and room number -- this may indicate a failing access point rather than a device-side issue.

Password Reset Procedure

File: seed/knowledge_docs/password_reset_procedure.md -- category: PASSWORD

Overview

University accounts use a single password across email, WiFi (CampusSecure), VPN, and the student portal. This document covers self-service password reset and multi-factor authentication (MFA) device changes.

Self-Service Password Reset

1. Go to password.university.edu from any browser.
2. Click **Forgot Password**.
3. Enter your university username (not your full email address).
4. Verify your identity using one of:
 - An MFA push notification to your registered phone, or
 - A one-time code sent to your personal recovery email (set during account setup), or
 - Your security questions (if configured).
5. Choose a new password. Requirements: at least 12 characters, at least one number, at least one symbol, and it must not match any of your last 5 passwords.
6. Allow up to 15 minutes for the new password to propagate to all systems (email, WiFi, VPN). Signing in immediately after a reset may briefly still fail on some systems while this propagates.

"I forgot my student password"

Direct the student to password.university.edu and the steps above. If they cannot complete identity verification (no MFA device registered, recovery email inaccessible, no security questions set), this requires in-person identity verification at the IT Services help desk with a photo ID -- self-service reset cannot bypass identity verification.

Resetting MFA (Multi-Factor Authentication)

If a student has a new phone, lost their phone, or their MFA app was uninstalled, their old MFA device needs to be de-registered and a new one added.

1. Go to mfa.university.edu and sign in with username and password (not MFA -- signing in to the MFA portal itself only requires the password).
2. Under **My Devices**, remove the old/lost device.
3. Click **Add a Device** and follow the QR code enrollment flow with an authenticator app (Microsoft Authenticator or Google Authenticator are both supported).
4. Test the new device immediately by triggering a push (the portal has a **Send Test Push** button) before leaving the page.

If a student cannot sign in to mfa.university.edu at all because they also forgot their password, the password must be reset FIRST (see above), then MFA re-enrollment can proceed.

Account Lockout

Five consecutive failed password attempts locks the account for 30 minutes. This is a fixed security policy and cannot be manually unlocked early by IT staff -- advise the student to wait out the lockout window rather than continuing to retry, since each attempt during lockout resets the 30-minute timer.

Related Issues

A locked-out or recently-reset account can also cause VPN-ERR-403 (see the VPN Troubleshooting Guide) for up to 15 minutes after a reset while the new password propagates -- this is expected and not a separate VPN problem.

Student Account Setup

File: [seed/knowledge_docs/student_account_setup.md](#) -- *category:* ACCOUNT

Overview

Every admitted student receives a university account after enrollment deposit confirmation. This document covers first-time activation and initial configuration.

First-Time Activation

1. Within 48 hours of enrollment confirmation, an activation email is sent to the personal email address provided on the application. It contains a one-time activation link (valid for 7 days).

2. Click the activation link and set an initial password (12+ characters, at least one number and one symbol).
3. Register for multi-factor authentication (MFA) immediately -- the activation flow will not let you skip this step. Use an authenticator app (Microsoft Authenticator or Google Authenticator).
4. Set a personal recovery email address (used later for self-service password resets -- see the Password Reset Procedure document).

"How do I set up my new student account?"

Walk the student through First-Time Activation above. If they never received the activation email, check:

- Spam/junk folder of the personal email on file.
- Whether the enrollment deposit has actually cleared (activation emails only send after deposit confirmation, which can lag 24-48 hours behind payment processing).
- That the personal email on file is spelled correctly in the admissions system -- a typo there means the activation email went nowhere; this requires an admissions office correction, not an IT fix, and should be escalated to the registrar's office.

What Gets Provisioned on Activation

- University email address (firstname.lastname@university.edu)
- Access to the student portal
- WiFi access (CampusSecure -- see the WiFi Troubleshooting Guide)
- Access to Microsoft 365 (Office apps, OneDrive) -- see the Microsoft Office Installation document
- VPN access -- note this can take up to 24 hours AFTER activation to become active; a VPN-ERR-403 in the first day is often just this delay, not a real problem (see the VPN Troubleshooting Guide)

Changing Your Preferred Name

Students may set a preferred first name, which appears on class rosters, email display name, and the student portal, separate from their legal name (which remains on official transcripts and financial documents). This is done at portal.university.edu under Profile > Preferred Name, and takes up to 24 hours to propagate to all systems.

Deactivation

Accounts are automatically deactivated 180 days after a student's last active term with no new enrollment, per university data retention policy. A deactivated account can be reactivated by the registrar's office if the student re-enrolls; IT Services cannot reactivate an account directly.

Microsoft Office Installation

File: [seed/knowledge_docs/microsoft_office_installation.md](#) -- category: SOFTWARE

Overview

All active students and staff are entitled to a free Microsoft 365 subscription (Word, Excel, PowerPoint, OneNote, and 1TB of OneDrive storage) for the duration of their enrollment/employment, installable on up to 5 personal devices.

"How do I install Microsoft Office on my laptop?"

1. Go to [portal.office.com](#) and sign in with your full university email address and password.
2. Click **Install Office** in the top-right corner of the Office portal.
3. Choose **Office 365 apps** (not "Office 2021" or any one-time-purchase option -- those are not covered by the university license).
4. Run the downloaded installer. Installation typically takes 10-20 minutes depending on internet speed.
5. Open any Office app (e.g. Word) and sign in again with your university email when prompted -- this activates the license on this specific device.

Common Installation Issues

"Something went wrong" during activation

Usually means the sign-in step (step 5) used a personal Microsoft account instead of the university account. Sign out of any personal Microsoft account on the device first, then retry activation with the university email.

Installer hangs at a specific percentage

Corporate/campus network proxies occasionally interrupt the Office installer's connection. Try again on CampusSecure WiFi (not CampusGuest, which rate-limits large downloads) or a home network.

"You've reached the device limit"

The license allows 5 devices. To free up a slot, go to [portal.office.com](#) > Account settings > find "Install status" and deactivate an old/unused device from the list, then retry installation on the new device.

Office apps show "Unlicensed Product" after working previously

This means the subscription lapsed -- almost always because the student's account was deactivated (see Student Account Setup document's Deactivation section) or the employee's employment status changed. If the account itself is active in the student/staff portal, this may be a sync delay of up to 24 hours; if it persists beyond that, escalate to software licensing.

Mac vs. Windows

The installer and activation steps above are identical on macOS and Windows -- both are covered by the same university Microsoft 365 license. Office for Mac does have some minor feature differences from Office for Windows (mainly in advanced Excel macros), which is a Microsoft product limitation, not something IT Services can change.

Campus Network Guide

File: [seed/knowledge_docs/campus_network_guide.md](#) -- category: NETWORK

Overview

This document covers wired (ethernet) networking in dorms and campus buildings, network naming conventions, and general network architecture that other troubleshooting guides (WiFi, VPN) reference.

Wired Ethernet in Dorms

Every dorm room has at least one active ethernet wall jack, providing a faster and more stable connection than WiFi -- recommended for gaming, large downloads, and video calls.

1. Connect an ethernet cable from your device to the wall jack.
2. Most devices configure automatically via DHCP -- no manual setup is needed. If the connection doesn't come up automatically:
 - Confirm the device's network adapter is set to "Obtain an IP address automatically" (DHCP), not a static/manual IP.
 - Try a different ethernet cable -- damaged cables are a very common cause of "no connection" reports in dorms.
3. Wired connections do not require 802.1X sign-in the way CampusSecure WiFi does -- registration happens automatically based on the port and device the first time it connects each semester.

If a dorm room's ethernet jack shows no link light at all on both ends, the jack itself may be inactive (not all jacks in a room are always enabled) or the in-wall cabling may be faulty -- this requires a physical inspection by facilities/network engineering, not something resolvable remotely.

Network Names (SSIDs) Reference

- **CampusSecure** -- primary WiFi network, 802.1X encrypted, requires university login (see the WiFi Troubleshooting Guide).
- **CampusGuest** -- open guest WiFi, rate-limited, visitors only.
- **CampusIoT** -- separate network for smart devices (game consoles, smart speakers, printers) that can't do 802.1X login; requires pre-registering the device's MAC address at [iot.university.edu](#).

VPN and Off-Campus Access

The VPN (vpn.university.edu, via Cisco AnyConnect -- see the VPN Troubleshooting Guide) is the only supported way to reach internal campus resources (file shares, internal web apps) from outside the campus network. Off-campus students should expect anything that requires VPN access on-campus to also require it off-campus.

Network Architecture Notes (For Escalation Context)

The campus network is split into building-level VLANs; a device physically moved between buildings on wired ethernet will receive a new IP address on reconnect (this is normal, not a fault). WiFi access points are managed centrally and roam a device between physical access points transparently within **CampusSecure**, which is why brief 1-3 second WiFi drops while walking are expected and not a bug (see the WiFi Troubleshooting Guide).

When to Escalate

Any report of a fully dead ethernet jack (no link light with a known-good cable and device) should be escalated to facilities/network engineering with the building, room number, and jack label (printed on the wall plate) -- this cannot be diagnosed or fixed remotely.

Printer Troubleshooting

File: [seed/knowledge_docs/printer_troubleshooting.md](#) -- *category:* **HARDWARE**

Overview

This document covers the campus print system (**PrintSmart**), used in libraries, computer labs, and department printers. Students print by sending a job from any registered device; the job releases when the student badges in at any printer with their student ID.

Setting Up PrintSmart

1. Install the PrintSmart client from the software portal (same portal used for Cisco AnyConnect -- see the VPN Troubleshooting Guide).
2. Sign in with your university credentials.
3. Print from any application as normal; select the **PrintSmart** printer.

The job is held on the server, not sent directly to a printer.

4. At any campus printer, tap your student ID card on the badge reader.
- All held jobs for your account appear on the printer's screen; select which to print.

"The library printer says paper jam but there's no paper stuck"

This is one of the most common printer complaints and is almost always one of the following, in order of likelihood:

- **A jam earlier in the paper path that isn't visible from the tray.**

Open every access panel/door on the printer (not just the main tray) -- most jams are in the fuser area or a roller further inside, not visible without opening the internal doors. The printer's screen usually shows a

diagram indicating which door to open.

- **A torn corner of paper left behind from a previous jam.** Even after a jam looks cleared, a small torn piece can remain lodged in a roller and keep triggering the sensor. Run a hand along the visible paper path to check for a torn fragment.
- **Humidity-swollen paper.** In humid weather, paper from an opened ream can swell slightly and jam a sensor that a fresh ream wouldn't. Try a fresh ream if available.
- **A worn paper-feed sensor giving a false reading.** If the paper path is confirmed completely clear and the jam error persists after a full power cycle, this indicates a hardware sensor fault -- escalate to facilities/printer maintenance rather than continuing to open and close trays.

Job Stuck "Pending" and Never Appears at the Printer

- Confirm the printer you badged at is online (a red or amber light usually indicates an offline/error state).
- Held jobs expire after 24 hours if never released -- if it's been longer than that, the job is gone and needs to be resent.
- Confirm you badged with the correct ID -- a student with two cards (e.g. a replacement issued after a lost card) occasionally still carries the deactivated old one.

Print Quota

Each student receives 500 pages/semester at no charge; additional pages are billed to the student account at \$0.10/page. Quota resets at the start of each semester and does not roll over. Quota balance is visible in the PrintSmart client.

When to Escalate

Any hardware fault confirmed after a full power cycle (persistent jam error with a visibly clear paper path, print quality defects like repeating marks/streaks, unusual noises) should be escalated to facilities/printer maintenance with the printer's asset tag number (printed on a sticker on the printer's side).

Known IT Issues

File: [seed/knowledge_docs/known_it_issues.md](#) -- category: OTHER

Overview

This document tracks currently known, ongoing issues that affect multiple students/staff, so that a ticket about a widespread issue can be answered immediately ("this is a known issue, here is the status") instead of independently re-investigated. This document is updated by IT Services as issues are confirmed and resolved -- an admin should re-upload a new version (see the knowledge-base versioning behavior in the RAG manual) when an issue's status changes rather than editing history silently.

"Is there a known outage affecting campus email today?"

Check this section first before investigating an individual email complaint as a one-off account issue. As of this document's most recent version, there are no active, ongoing email outages. If a student reports email being down, treat it as an individual account issue (see the Password Reset Procedure document) unless multiple independent reports arrive within a short window, in which case escalate to infrastructure immediately and update this document.

Currently Known Issues

Lecture hall WiFi capacity (multiple buildings, ongoing)

Several large lecture halls see degraded **CampusSecure** WiFi performance during peak class-change times (a few minutes before/after the hour) due to a high concentration of simultaneously-connecting devices. This is a known capacity limitation being addressed by a phased access-point upgrade; the WiFi Troubleshooting Guide's "interference in dense areas" section reflects this. No individual-device fix resolves this; do not have students repeatedly forget/re-add the network for this specific symptom.

VPN slow during finals week (seasonal, recurring)

VPN throughput is measurably slower during finals week each semester due to elevated concurrent usage (students working on projects/theses from off-campus). This is a known, expected, seasonal load pattern -- not a fault -- and does not require escalation on its own; only escalate if a student sees the connection actually fail (VPN-ERR-403 or "Connection attempt has failed") rather than just being slow.

Resolved Issues (Recent History)

Office 365 licensing sync delay (resolved)

For a period, newly activated student accounts saw up to a 48-hour delay before Microsoft 365 licensing activated (longer than the normal 24-hour window described in the Microsoft Office Installation document). This was traced to a licensing sync job running only once daily instead of hourly, and was resolved by increasing the sync frequency. If a student describes exactly this symptom now, it should be treated as a new individual issue, not this resolved one -- do not assume it has recurred without checking current sync job health first.

How to Use This Document During Triage

1. Check whether the student's symptom matches an entry under "Currently Known Issues" above.
2. If it matches, respond with the known cause/status/ETA rather than re-diagnosing from scratch, and do not open a duplicate infrastructure ticket for it -- link the student's ticket to the existing tracking ticket instead.
3. If it does NOT match anything here, proceed with the relevant individual troubleshooting guide (VPN, WiFi, Password Reset, etc.) as normal.

Appendix F: Example Chunks and Example Retrieval Results

Purpose of this appendix

phase7.md's final-deliverables list asks for example chunks and example retrieval results alongside the example documents in the previous appendix. Everything in this appendix was captured by actually querying the real, live database used throughout this implementation -- the VPN Troubleshooting Guide document exactly as it exists after real ingestion, and a real `retrieval_service.search()` call against it. Nothing here is invented or hand-constructed to look plausible.

Example: how the VPN Troubleshooting Guide was actually chunked

Querying `knowledge_chunks` for the ingested VPN Troubleshooting Guide document (`RAG_CHUNK_SIZE_WORDS=600`, `RAG_CHUNK_OVERLAP_WORDS=80`, the project defaults) returns exactly **two** chunks:

chunk_index	token_count (approx.)	word count
0	573	441
1	140	108

Chunk 0 (441 words) covers the document's "Overview," "Installing Cisco AnyConnect," "Connecting to the VPN," "VPN-ERR-403: Authentication Failed," the unnamed "Connection attempt has failed" section, and most of "VPN connects but internal sites are still unreachable" -- packed together because their combined size (573 approximate tokens) still sits under the 600-token budget, and because `chunk_text()`'s block-packing loop keeps adding whole blocks to the current chunk as long as they fit, only starting a new chunk once the *next* block would push the running total over budget.

Chunk 1 (108 words) is where this is worth reading closely, because it's a real, live example of the overlap mechanism described in the chunking strategy chapter, not a constructed illustration of it. Its actual content, in full:

```
### VPN connects but internal sites are still unreachable

- Confirm the AnyConnect icon shows green (connected), not yellow
  (connecting) or red (disconnected).
- DNS can take up to 30 seconds to update after connecting. Wait, then
  retry.
- Some browsers cache a "site not found" result. Clear the browser's DNS
  cache or try a private/incognito window.

## When to Escalate

If a student has confirmed their password is current, MFA was approved
promptly, and the account is more than 24 hours old, and VPN-ERR-403
still occurs, escalate to network engineering -- this can indicate the
account was not correctly added to the VPN authorization group during
provisioning.
```

Notice the heading `### VPN connects but internal sites are still unreachable` and its three bullet points appear in **both** chunk 0 (as the tail of that chunk) and chunk 1 (as the head of this one). This is exactly `_overlap_prefix()` doing its job: when chunk 0 filled up and a new chunk had to start, the last block that had just been added to chunk 0 (this heading + its bullets) was carried forward as chunk 1's opening context, before chunk 1's own new content ("When to Escalate") begins. A reader who only ever retrieves chunk 1 in isolation -- which is exactly what happens at query time, since chunks are retrieved independently and ranked by score, not read in original document order -- still gets the full "VPN connects but internal sites are still unreachable" guidance intact, not a fragment starting mid-list.

Example: a real retrieval result

Using phase7.md's own literal example query -- a student's message, "My VPN is not connecting" -- against the real seeded 8-document knowledge base, with `top_k=3`:

Rank	Score	Document	Chunk
1	0.3279	VPN Troubleshooting Guide	chunk 1 ("VPN connects but internal sites are still unreachable" / "When to Escalate")
2	0.2764	VPN Troubleshooting Guide	chunk 0 (Overview through most of the errors section)
3	0.2023	WiFi Troubleshooting Guide	chunk 0 (Overview through "CampusGuest Limitations")

Both of the VPN Troubleshooting Guide's chunks outrank every chunk of every other document, which is the correct, expected outcome for a VPN-specific query -- and the WiFi Troubleshooting Guide's chunk ranks third rather than being excluded outright, which is also correct and expected: `retrieval_service.search()` doesn't have a relevance *threshold* below which results are dropped, it returns the top `top_k` by score regardless of how low the lowest of those scores is (see the retrieval chapter for the reasoning: filtering by an absolute score threshold in a hashing-trick vocabulary-overlap embedding space is not a reliable signal, since the *meaning* of a given score value shifts depending on how sparse or common the shared vocabulary between two pieces of text happens to be -- ranking is reliable here, an absolute cutoff would not be).

The full, real `build_rag_context()` output for this exact query -- what a future AI agent would actually receive:

```
{
  "query": "My VPN is not connecting",
  "results": [
    {
      "title": "VPN Troubleshooting Guide",
      "content": "### VPN connects but internal sites are still unreachable...\n[full text is chunk 1, reproduced above]",
      "score": 0.3279
    },
    {
      "title": "VPN Troubleshooting Guide",
      "content": "## Overview\n\nThe university VPN (Virtual Private Network)...\n[full text is chunk 0, reproduced in Appendix E]",
      "score": 0.2764
    },
    {
      "title": "WiFi Troubleshooting Guide",
      "content": "## Overview\n\nThe university WiFi (Wireless Fidelity)...\n[full text is chunk 0, reproduced in Appendix E]",
      "score": 0.2023
    }
  ]
}
```

```

    "score": 0.2764
  },
  {
    "title": "WiFi Troubleshooting Guide",
    "content": "## Overview\n\nCampus WiFi is provided under two network names\n(SSIDs)...\n[full text in Appendix E]",
    "score": 0.2023
  }
]
}

```

(Every **content** field is shown truncated above purely for this appendix's print layout -- the real response contains each chunk's complete, untruncated text with real embedded newlines, exactly as reproduced in full for chunk 1 in the fenced block earlier in this appendix, and for every document's full text in Appendix E. Note also this is a *different* real query from the "VPN still does not connect, auth failed" example used in the admin API reference and testing chapters -- the two queries word themselves differently enough that they produce different score values and, for this one, a different top-ranked chunk, which is itself a useful illustration that retrieval is sensitive to actual query phrasing, not just topic.)

What this demonstrates end to end

Between this appendix and Appendix E, a reader now has: the complete, real source text of a real knowledge document; the complete, real chunks that document was actually split into, including a genuine (not constructed-for-effect) example of the overlap mechanism carrying context across a chunk boundary; and a complete, real ranked retrieval result and RAG context object for a realistic student query against that document, sitting alongside seven other real documents in the same knowledge base. Every number, every piece of text, and every ranking above came from actually running this system, not from describing what it should do.